U.S. DOE H2O Wave Hindcast

Open-Source Data Resource Characterization Example

Table of contents

This example characterizes the wave energy resource at U.S. marine energy resource assessment points using the U.S. DOE H2O High-Resolution Wave Hindcast. Data is accessed using through the `us-marine-energy-resource` library, which finds the hindcast grid node nearest a user supplied coordinate and downloads its record. The data source is MHKDR Submission 326: High Resolution Ocean Surface Wave Hindcast (US Wave) Data and data is downloaded from the H2O Hindcast OpenEI Data Lake. The underlying data is `.h5` files that are standardized from the original model output and are the datasource for the NLR Marine Energy Atlas. This analysis covers aggregate statistics, monthly climatology, joint probability, extreme sea states, exceedance, directionality, and a multi point comparison, using MHKiT for the using the wave hindcast data.

1 Platform Setup

This section setups this notebook code for multiple different notebook types including Quarto, Jupyter Notebook, and Kaggle. Behind the scenes each of these outputs has different ways for producing images, interactive elements (maps), and handling importing of bundled software. This section handles all of the behind the scenes to make each platform produce similar output and the details are in `h2o_examples/environment.py`.

```
import os
import sys
from pathlib import Path

# On Kaggle the h2o_examples helpers ship in the companion dataset; make them
# importable before anything else. Kaggle mounts datasets a few levels deep
# (e.g. /kaggle/input/datasets/<user>/<slug>/), so search recursively rather
```

```
# than assuming */h2o_examples.
_pkg = next(Path("/kaggle/input").rglob("h2o_examples/__init__.py"), None)
if _pkg is not None:
    sys.path.insert(0, str(_pkg.parent.parent))
elif os.environ.get("KAGGLE_KERNEL_RUN_TYPE"):
    _listing = sorted(str(p) for p in Path("/kaggle/input").rglob("*"))
    raise RuntimeError(
        "h2o_examples not found under /kaggle/input -- is the "
        "h2o-wave-hindcast-cache dataset attached? /kaggle/input contains:\n"
        + "\n".join(_listing[:80])
    )

from h2o_examples.environment import CACHE_DIR
```

2 Python Preamble

This section handles all of the python library initialization and specific initialization for downloading, caching, analyzing and visualizing the wave hindcast data.

2.1 Local User Setup

This example leverages `uv` for installing python packages and environments. This is setup automatically on some platforms, but some platforms will need to install python and its dependencies:

2.1.a Clone the repository

```
git clone https://github.com/US-Marine-Energy-Resource/examples
```

2.1.b Install `uv`

UV Installation Instructions

2.1.c Install dependencies

```
uv sync
```

2.1.d Start Jupyter Notebook (local users only)

Using `uv` with Jupyter

```
uv run --with jupyter jupyter lab
```

Open the notebook in a browser

2.2 Python Library Imports

```
import inspect

import calendar
```

```
import textwrap
import matplotlib.pyplot as plt
from matplotlib.patches import Patch
import numpy as np
import pandas as pd
import seaborn as sns

from mhkit.wave import contours, graphics, performance
from us_marine_energy_resource import wave_hindcast
from us_marine_energy_resource.wave_hindcast.errors import InvalidAttributeError

from h2o_examples.cache import memoize_pickle
from h2o_examples.download_statistics import download_statistics, write_json
from h2o_examples.figures import FIG_DIR, FIG_WIDTH, plot_wave_matrix, savefig, show_table
from h2o_examples.mapping import render_points_map_outputs
from h2o_examples.points import PACWAVE_POINTS, make_points_map, point_coord
```

3 Wave Resource Assessment Specification

This section defines the point level inputs to download time-series data from the H2O wave hindcast.

3.1 Inputs

The user needs to provide 3 inputs,

- Coordinates as lat/lon decimal
  - Within the modeled area
- Years
  - 1979 to 2020
- Variables
  - Full list below

3.1.a Coordinates

Coordinates must be supplied as decimal latitude and longitude:

- Humboldt Bay:
  - Latitude: `40.8418`
  - Longitude: `-124.2480`

Note: The model resolution more detailed near the shore, and coordinate precision of 3-6 decimal places (approximately 110m to 11cm) is reasonable for the level of precision within the underlying model. Internally the `get_data_at_point` function will select the closest point to the user specified coordinates and reports how far the user input point is from the modeled point. Modeled points are fixed for the 42 year hindcast.

```
humboldt_bay = {
    "lat": 40.8398,
```

```
    "lon": -124.25,
}

# Simplified example, downloads most recent year, all variables
df = wave_hindcast.get_data_at_point(humboldt_bay["lat"], humboldt_bay["lon"], years=[2020],
variables=["significant_wave_height", "energy_period"])
```

3.1.b Years

- Year Range
  - 1979 to 2020 (Hawaii and Atlantic are up to 2010)

3.1.c Variables

The following variables can be specified using the Hindcast Variable Id. Variables map directly to the underlying SWAN models "Bulk Parameters".

Name	SWAN Name	Symbol	Units	Hindcast Variable Id	Description
Significant Wave Height	HSIGN	H_{m_0}	meters	significant_wave_height	Significant wave height
Energy Period	TMM10	T_e, T_{-10}	seconds	energy_period	Energy period
Omni-Directional Wave Power		J	watts per meter	omni-directional_wave_power	Omni-directional wave power
Mean Wave Direction	DIR		from degrees clockwise from true north	mean_wave_direction	Mean wave direction
Peak Period	RTP	T_p	seconds	peak_period	Relative peak period of the variance density spectrum (in the absence of currents)
Maximum Energy Direction		θ_J	from degrees clockwise from true north	maximum_energy_direction	Direction of maximum wave energy

Name	SWAN Name	Symbol	Units	Hindcast Variable Id	Description
Directionality Coefficient		d	unitless, range from 0 to 1	`directionality_coefficient`	Directionality coefficient
Spectral Width		ϵ_0	nitless, range from 0 to 1	`spectral_width`	Spectral Width

Note: Symbol are defined in IEC 62600-101, which the standard calls “parameters”.

3.1.d Additional Variables

The following variables can be specified using the Hindcast Variable Id. Variables are map directly the SWAN model “Bulk Parameters”.

Name	SWAN Name	Symbol	Units	Hindcast Variable Id	Description
Mean Period	`PER`		seconds	`mean_average_period`	Mean absolute wave period, equivalent to $T_{m_{01}}
Mean Absolute Zero Crossing Period	`TM02`	T_z, T_{02}	seconds	`mean_zero_crossing_period`	Mean absolute zero crossing period
Peak Wave Direction	`PDIR`		from degrees clockwise from true north	`peak_wave_direction`	Peak wave direction, determined by wave power

3.1.e Metadata

For each point there an associated metadata entry in the hindcast data identifies non time varying data.

Name	SWAN Name	Symbol	Units	Hindcast Variable Id
X Coordinate	`XP`		decimal degrees longitude	`longitude`
Y Coordinate	`YP`		decimal degrees latitude	`latitude`

Name	SWAN Name	Symbol	Units	Hindcast Vari-able Id
Depth	DEPTH	h	meters	depth

3.2 Selected Coordinates

The H2O wave hindcast is a has an underlying unstructured grid that follows the coast. The underlying model provides latitude and longitude coordinates for each result. Behind the scenes the wpto_hindcast module selects the nearest model grid node and returns that node's time series data. The node's position, index (gid), and distance from the requested coordinate are returned as additional data (Section 5).

```
# --- Assessment coordinates (edit me) ----------------------------------------
# name -> (latitude, longitude, label). PacWave South/North take their centerpoints
# from the published corners in h2o_examples/points.py; the rest are inline.
_pw_s = PACWAVE_POINTS["US_Oregon_PacWave_South"]
_pw_n = PACWAVE_POINTS["US_Oregon_PacWave_North"]

POINTS = {
    "US_Oregon_PacWave_South": (_pw_s["lat"], _pw_s["lng"], "PacWave South"),
    "US_Oregon_PacWave_North": (_pw_n["lat"], _pw_n["lng"], "PacWave North"),
    "US_Hawaii_Oahu_WETS": (21.46488, -157.751524, "WETS"),
    # The Atlantic 2011 to 202 points are chunked (12, 10294) and are impossible to download
efficiently.
    # Every other domain is (1500, 500)
    # An atlantic year touches 243 scattered chunks instead of 2 -- ~120 MB per variable-year
    # against 5.8, and minutes rather than seconds. Both omitted below.
    "US_North_Carolina_Jenettes_Pier": (35.91036, -75.59239, "Jennette's Pier"),
    "US_North_Carolina_Pioneer_WEC": (35.943117, -74.88035, "Pioneer WEC"),
    "US_California_Scripps_Pier": (32.867633, -117.263167, "Scripps Pier"),
    "US_California_San_Luis_Obispo": (
        35.1672960606959,
        -120.740349224586,
        "San Luis Obispo",
    ),
    "US_California_Humboldt_Bay": (40.8398, -124.25, "Humboldt Bay"),
    "US_Alaska_Kodiak": (57.0, -152.5, "Kodiak (open water)"),
    "US_PuertoRico_North_Shore": (18.60, -66.10, "N. of San Juan (open water)"),
    # add or remove points here
}

# --- Water depth per point [m] -----------------------------------------------
# Read once from the hindcast's own `meta` table (the `water_depth` field at the
# point's grid node, 2020 files) and recorded here as a constant rather than re-read
# at render time: it is a property of the grid node, it never changes between runs,
```

```
# and baking it in keeps the render offline-safe on Kaggle.
#
# It is not simply read from `df.attrs` because the package drops it. The S3 backend
# looks for a `meta` field named `depth`, but the published files name it
# `water_depth` (verified on West_Coast, Hawaii and Alaska), so the lookup misses and
# the key silently never appears. Candidate package issue.
#
# The 67.7 m below is a good check on the source: PacWave publishes 65-78 m MLLW for
# the South site (see h2o_examples/points.py), and the node agrees.
# Points absent from this dict have an unknown depth and are treated as unconstrained.
POINT_DEPTHS = {
    "US_Oregon_PacWave_South": 67.7,        # published site depth 65-78 m MLLW
    "US_California_San_Luis_Obispo": 14.0,  # 0.2 km from shore, nearshore node
    "US_Hawaii_Oahu_WETS": 353.1,           # Hawaii's shelf drops off fast
    "US_Alaska_Kodiak": 143.9,
}
```

```
# --- Selection: which points drive which analysis ----------------------------
SELECTED_POINT = "US_Oregon_PacWave_South"   # drives the single-point analysis
POINTS_TO_COMPARE = [                         # drives the multi-point comparison
    "US_Oregon_PacWave_South",                 # West_Coast
    "US_California_San_Luis_Obispo",           # West_Coast
    "US_Hawaii_Oahu_WETS",                     # Hawaii
    "US_Alaska_Kodiak",                        # Alaska
]
```

```
# --- Working point dict, built from POINTS -----------------------------------
# Coordinates come from POINTS; PacWave points also carry their polygon geometry
# and color from points.py so the map can draw their berth boundaries.
POINT_INFO = {}
for _name, (_lat, _lng, _label) in POINTS.items():
    POINT_INFO[_name] = {"lat": _lat, "lng": _lng, "label": _label,
                         "depth": POINT_DEPTHS.get(_name)}
    if _name in PACWAVE_POINTS:
        POINT_INFO[_name]["corners"] = PACWAVE_POINTS[_name]["corners"]
        POINT_INFO[_name]["color"] = PACWAVE_POINTS[_name]["color"]

# Single-point identity, derived from the selection above. Quarto renders these
# inline in the narrative below (e.g. `{python} POINT_LABEL`). Figure captions name
# the point literally, because Quarto does not evaluate inline code in a `fig-cap`;
# if you change SELECTED_POINT, update the single-point captions to match. Captions
# name YEAR and CONTOUR_YEARS literally too, for the same reason.
lat_lon = point_coord(POINT_INFO[SELECTED_POINT])
```

```
POINT_LABEL = POINT_INFO[SELECTED_POINT]["label"]
POINT_COORDS = f"{lat_lon[0]:.4f}, {lat_lon[1]:.4f}"
```

Every point as a lat,lon pair (Table 1). For the two PacWave points this is the centerpoint computed from the published corners. The `Domain` column is resolved from the coordinates by `wave_hindcast.describe_point`, an offline lookup against the hindcast's grid-node index. It needs no download and no API key and directly access data stored in OpenEI AWS S3 Storage.

```
point_coordinates = pd.DataFrame(
    [
        {
            "Point": name,
            "Label": point["label"],
            "Domain": wave_hindcast.describe_point(*point_coord(point))["domain"],
            "Latitude": point["lat"],
            "Longitude": point["lng"],
        }
        for name, point in POINT_INFO.items()
    ]
)
show_table(point_coordinates)
```

Table 1: Wave resource assessment points and their hindcast domains

The first map (Figure 1) visualizes only `SELECTED_POINT`. The multi-point map appears with the comparison in Section 13.

```
_ = render_points_map_outputs(
    {SELECTED_POINT: POINT_INFO[SELECTED_POINT]},
    make_points_map,
    output_dir=FIG_DIR,
    stem="selected_point_map",
    title=POINT_LABEL,
)
```

```
<folium.folium.Map at 0x11f026d10>
```

Figure 1: Selected point for the single-point analysis below

3.3 Years

`YEAR` is the single-year record the single-point analysis reads; `CONTOUR_YEARS` is the multi-year record the extreme-value contours draw from (Section 14).

```
YEAR = 2020  # the newest published year, served by the S3 backend in every domain
CONTOUR_YEARS = [2016, 2017, 2018, 2019, 2020]  # multi-year record for contours
```

3.4 Variables

The eight variables this example uses, in the hindcast dataset explicit variable names

```
VARIABLES = [
    "significant_wave_height",
    "energy_period",
    "omni-directional_wave_power",
    "mean_wave_direction",
    "peak_period",
    "maximum_energy_direction",
    "directionality_coefficient",
    "spectral_width",
]

COLUMN_NAMES = {
    "significant_wave_height": "Hm0",
    "energy_period": "Te",
    "omni-directional_wave_power": "J",
    "mean_wave_direction": "Dir",
    "peak_period": "Tp",
    "maximum_energy_direction": "MaxEDir",
    "directionality_coefficient": "DirCoeff",
    "spectral_width": "SpecWidth",
}

# The multi-year contour pull (@sec-contour-records) reads hm0 and te.
CONTOUR_VARIABLES = ["significant_wave_height", "energy_period"]

# Sentinel written into the data for timesteps the model did not produce.
FILL_VALUE = -9.0
```

4 Downloading Data

We read the 3-hour bulk-parameter hindcast [1] through `wave_hindcast.get_data_at_point(lat, lon, ...)`, which finds the hindcast grid node nearest a coordinate and returns its record as a DataFrame.

4.1 Backends

Two backends serve the data picks and the wpto_hindcast library selects the appropriate backed by request sizs. `get_data_at_point` allows 3 options, `auto`, `s3` and `api`. `auto` mode switches from S3 to the API past 30 variable-years (n_variables * n_years). The S3 backend reads the published files anonymously (free, no key, no request limits, but raw file access that downloads slowly at volume). The NLR API backend builds an archive server side (free API key required, suited to large pulls).

Every query in this example is intentionally small enough to query with S3 and `load_point` specifies `backend="s3"` to enforce this the api free backend queries. Every download is cached on disk under

`~/.mer_wave_cache`, so repeat renders read from disk. It is possible to download a large volume of data for free from S3 but the queries are relatively slow vs the api backend.

4.2 The `mer` command-line application

The `us-marine-energy-resource` package also installs a command-line application, `mer`, so the same workflow is available without writing Python. `mer` uses the same python under the hood and provides two commands `mer wave` and `mer tidal` to query the two U.S. DOE H2O hindcast datasets, `mer sources` lists the open-data sources, and `mer ls` / `mer info` / `mer explore` / `mer download` browse and pull the raw files straight from `s3://wpto-pds-us-wave`.

`mer wave <lat,lon>` finds the nearest grid node and, by default, fetches the four high-impact variables for the most recent served year – a quick first look. `--info` resolves the node offline, with no download and no key:

```
# `mer` is installed by the package, normally next to the interpreter running this
# example; fall back to PATH so this also works where the two differ (e.g. Kaggle).
#
# We run using subprocess rather than the `!` shell magic, and force plain text.

import re
import shutil
import subprocess

_mer = Path(sys.executable).parent / "mer"
MER = str(_mer) if _mer.exists() else (shutil.which("mer") or "mer")
_info = subprocess.run(
    [MER, "wave", "44.5670,-124.2290", "--info"],
    capture_output=True,
    text=True,
    # env={os.environ, "NO_COLOR": "1", "TERM": "dumb"},
    env={"NO_COLOR": "1", "TERM": "dumb"},
)
print(re.sub(r"\x1b\[[0-9;]*m", "", _info.stdout or _info.stderr))
```

```
──────────────────────── Traceback (most recent call last) ─────────────────────────
/Users/asimms/Desktop/Programming/us-marine-energy-resource/examples/.venv/l
ib/python3.11/site-packages/us_marine_energy_resource/gis.py:140 in
_load_spatial

   137 |   |   con.execute(f"LOAD '{wheel}'")
   138 |   |   return
   139 |   try:
❱  140 |   |   con.execute("INSTALL spatial")
   141 |   |   con.execute("LOAD spatial")
   142 |   except Exception as exc:
```

```
│   143 │   │   raise SpatialUnavailableError(                                │
└─────────────────────────────────────────────────────────────────────────────┘
IOException: IO Error: Can't find the home directory at ''
Specify a home directory using the SET home_directory='/path/to/dir' option.

The above exception was the direct cause of the following exception:

┌──────────────────────── Traceback (most recent call last) ────────────────────────┐
│ /Users/asimms/Desktop/Programming/us-marine-energy-resource/examples/.venv/l │
│ ib/python3.11/site-packages/us_marine_energy_resource/cli/wave.py:852 in     │
│ wave_query                                                                   │
│                                                                              │
│   849 │                                                                      │
│   850 │   explicit_name = name is not None                                   │
│   851 │   with _handle_errors():                                             │
│ ❱ 852 │   │   point = hindcast.describe_point(lat, lon, domain=domain,       │
│       backend=backend)                                                       │
│   853 │   │   _describe_table(point)                                         │
│   854 │   │   if backend != "s3":                                            │
│   855 │   │   │   _warn_outage(point["domain"], backend)                     │
│                                                                              │
│ /Users/asimms/Desktop/Programming/us-marine-energy-resource/examples/.venv/l │
│ ib/python3.11/site-packages/us_marine_energy_resource/wave_hindcast/hindcast │
│ .py:139 in describe_point                                                    │
│                                                                              │
│   136 │   PointOutsideDomainError                                            │
│   137 │   │   The point is outside every hindcast domain.                    │
│   138 │   """                                                                │
│ ❱ 139 │   node = nodes.nearest(lat, lon, domain=domain)                      │
│   140 │   assert isinstance(node, nodes.WaveNode)                            │
│   141 │   view = "s3" if backend == "auto" else backend                      │
│   142 │   info = _backend.get_backend(view).describe(node)                   │
│                                                                              │
│ /Users/asimms/Desktop/Programming/us-marine-energy-resource/examples/.venv/l │
│ ib/python3.11/site-packages/us_marine_energy_resource/wave_hindcast/nodes.py │
│ :217 in nearest                                                              │
│                                                                              │
│   214 │   """                                                                │
│   215 │   # Stage one: narrow to covering domains against the small          │
│       occupancy table.                                                       │
│   216 │   # Stage two, below, only touches those domains' node files.        │
│ ❱ 217 │   candidates = [domain] if domain else within(lat, lon)              │
│   218 │   if not candidates:                                                 │
│   219 │   │   raise errors.PointOutsideDomainError(                          │
│   220 │   │   │   f"({lat}, {lon}) is outside every hindcast domain",        │
│                                                                              │
```

```
| /Users/asimms/Desktop/Programming/us-marine-energy-resource/examples/.venv/l |
| ib/python3.11/site-packages/us_marine_energy_resource/wave_hindcast/nodes.py |
| :98 in within                                                                |
|                                                                              |
|    95 |                                                                      |
|    96 |   return [                                                           |
|    97 |   |   row[0]                                                         |
| ❱  98 |   |   for row in gis.connection()                                    |
|    99 |   |   .execute(                                                      |
|   100 |   |   |   f"""                                                       |
|   101 |   |   |   SELECT DISTINCT domain FROM                                |
|       read_parquet('{path.as_posix()}')                                      |
|                                                                              |
| /Users/asimms/Desktop/Programming/us-marine-energy-resource/examples/.venv/l |
| ib/python3.11/site-packages/us_marine_energy_resource/gis.py:180 in          |
| connection                                                                   |
|                                                                              |
|   177 |   if "con" not in _state:                                            |
|   178 |   |   duckdb = lazy_import("duckdb", "running spatial queries")      |
|   179 |   |   con = duckdb.connect()                                         |
| ❱ 180 |   |   _load_spatial(con, duckdb.__version__)                         |
|   181 |   |   _state["con"] = con                                            |
|   182 |   return _state["con"]                                               |
|   183                                                                        |
|                                                                              |
| /Users/asimms/Desktop/Programming/us-marine-energy-resource/examples/.venv/l |
| ib/python3.11/site-packages/us_marine_energy_resource/gis.py:143 in          |
| _load_spatial                                                                |
|                                                                              |
|   140 |   |   con.execute("INSTALL spatial")                                 |
|   141 |   |   con.execute("LOAD spatial")                                    |
|   142 |   except Exception as exc:                                           |
| ❱ 143 |   |   raise SpatialUnavailableError(                                 |
|   144 |   |   |   "could not load the DuckDB spatial extension. Install the  |
|       "                                                                      |
|   145 |   |   |   "duckdb-extension-spatial package for offline use, or run  |
|       "                                                                      |
|   146 |   |   |   "INSTALL spatial once with network access."                |
└──────────────────────────────────────────────────────────────────────────────┘
SpatialUnavailableError: could not load the DuckDB spatial extension. Install
the duckdb-extension-spatial package for offline use, or run INSTALL spatial
once with network access.
```

The flags that define a `wave` query mirror the Python arguments used below:

- `--years 2015-2020` and `--variables significant_wave_height,energy_period` narrow the pull; `--all` fetches every variable and year (slow).
- `--backend auto|s3|api` picks the source: `auto` reads small queries from S3 with no key and switches to the NLR API for large ones (a free key past ~30 variable-years), while `s3` and `api` force the choice.
- `--dry-run` prints the request without sending it, `--cache-only` keeps the result in `~/.mer_wave_cache` without writing a CSV, and `--cache` / `--clear` / `--clear-all` manage that cache.

The download of the record used below looks like this (shown, not run – the Python cells download it through the package instead):

```
mer wave 44.5670,-124.2290 --years 2020 \
                                --variables       significant_wave_height,energy_period,omni-
directional_wave_power,mean_wave_direction,peak_period
```

4.3 The Python query

`SELECTED_POINT`, `YEAR`, and `VARIABLES` come from the configuration section above. One query per point, through `load_point`, feeds every deliverable below.

```
def file_variable_names(requested, valid):
    """Map canonical dataset names onto one file's own spellings.

    The Gulf domain writes `omnidirectional_wave_power` where every other domain
    hyphenates `omni-directional_wave_power`; matching with the punctuation
    stripped absorbs that drift without a per-domain alias table.
    """
    def slug(name):
        return name.replace("-", "").replace("_", "")

    by_slug = {slug(name): name for name in valid}
    return [by_slug.get(slug(name), name) for name in requested]


def load_point(name, years=(YEAR,), variables=None):
    """One point's sea-state record via `wave_hindcast.get_data_at_point`.

    `variables` defaults to the full `VARIABLES` set; pass a subset to download only
    what an analysis consumes. Subsets cache under their own name, so the targeted
    record and the full one coexist rather than shadowing each other.

    The year span is baked into the cache name so a one-year record and a
    multi-year record of the same point never shadow each other in the package's
    cache. Pass `force=True` through to `get_data_at_point` to re-download.

    Returned as float64, and not as a harmless detail: five of mhkit's ten
    contour methods compute an internal default from the data
```

```
    (`max_x1 = x1.max() * 2`) and then reject it with `TypeError: max_x1 must be
    of type float` when handed float32, because `np.float32` is not a Python
    float — `np.float64` is. Casting once, here, keeps every method reachable.
    """
    variables = list(VARIABLES if variables is None else variables)
    years = sorted(years)
    span = str(years[0]) if len(years) == 1 else f"{years[0]}-{years[-1]}"
    lat = point_coord(POINT_INFO[name])[0]
    lon = point_coord(POINT_INFO[name])[1]
    # The package keys its cache on `name` alone, so a targeted record needs its
    # own name or it would shadow the full one for the same span (and vice versa).
    tag = "" if variables == VARIABLES else "_" + "-".join(COLUMN_NAMES[v] for v in variables)
    # Pinned to S3 rather than left on "auto". Auto switches to the NLR API past 30
    # variable-years (`backend.AUTO_SEAM_VARIABLE_YEARS`); the API needs a key and
    # serves a different attribute set per domain -- West_Coast omits
    # mean_wave_direction, which S3 does serve. Every query here is well under the
    # seam, and pinning keeps that true rather than merely coincidental.
    query = {
        "name": f"{name}_{span}{tag}",
        "years": years,
        "variables": variables,
        "backend": "s3",
    }

    def download(force=False):
        try:
            return wave_hindcast.get_data_at_point(*lat_lon, force=force, **query)
        except InvalidAttributeError as err:  # this file spells a name differently
            query["variables"] = file_variable_names(variables, err.valid)
            return wave_hindcast.get_data_at_point(*lat_lon, force=force, **query)
    #
    df = download()
    # The variable set is not part of the cache key either, so a record cached
    # before a VARIABLES change comes back with the old columns and the positional
    # rename below would mislabel them. Re-download when the width disagrees; an
    # unchanged set still reads straight from cache.
    columns = [COLUMN_NAMES[v] for v in variables]
    if df.shape[1] != len(columns):
        df = download(force=True)

    attrs = dict(df.attrs)
    # Columns come back Title Cased in the requested order; use the short names.
    df.columns = columns
    # Drop rows at the fill sentinel (-9 or -999), see the note below.
    df = df[(df > FILL_VALUE + 0.1).all(axis=1)]
    if "J" in df:
```

```
        df["J"] = df["J"] / 1000.0  # W/m -> kW/m
    df = df.astype("float64")
    df.attrs = attrs
    return df


def record_hours(df):
    """Hours the record covers: sea states times the sampling interval."""
    return len(df) * (df.index[1] - df.index[0]).total_seconds() / 3600


data = load_point(SELECTED_POINT)
```

ℹ Missing data is a fill value, not `NaN`

The model does not always return a valid result at every point over time. For non valid results the hindcast datasets use `-9`. This code section removes any rows with −9 from the downloaded dataet.

```
df = df[(df > FILL_VALUE + 0.1).all(axis=1)]
```

5 Exploring the Downloaded Data

5.1 Metadata

Every download provides metadata, accessible from `DataFrame.attrs`

```
metadata = wave_hindcast.describe_point(*lat_lon)
metadata
```

```
{'location_id': 479519,
 'domain': 'West_Coast',
 'endpoint': 's3://wpto-pds-us-wave',
 'requested_lat': 44.5670485,
 'requested_lon': -124.22896474999999,
 'node_lat': 44.5682,
 'node_lon': -124.228,
 'distance_m': 149.11513302759153,
 'years': [1979, 2020],
 'n_years': 42,
 'interval_minutes': 180,
 'direction_transform': None}
```

5.2 Data Preview

```
print(data.head().to_string())
```

```
                                Hm0        Te         J          Dir          Tp  MaxEDir
DirCoeff  SpecWidth
timestamp
2020-01-01 00:00:00+00:00  4.54951  12.9905  144.900  275.640991  22.766899    282.5  0.939274
0.49061
2020-01-01 03:00:00+00:00  5.16615  12.8580  184.976  273.678986  22.766899    282.5  0.926594
0.47974
2020-01-01 06:00:00+00:00  5.47224  13.2552  215.807  274.352997  22.766899    282.5  0.924255
0.44262
2020-01-01 09:00:00+00:00  5.72487  13.8130  249.203  277.634003  20.697201    282.5  0.940091
0.40394
2020-01-01 12:00:00+00:00  5.91118  14.3218  278.600  280.515015  20.697201    282.5  0.954117
0.36138
```

6 Aggregate Statistics

Sections 4 through 10 characterize one point, PacWave South at 44.5670, −124.2290, read for 2020 from the s3 direct backend of the H20 hindcast. Each single-point figure caption repeats the point and coordinates so a figure identifies its source on its own.

Aggregate wave energy statistics come from mhkit's `performance.statistics`; its docstring describes the fields.

```
print(inspect.getdoc(performance.statistics))
```

```
Calculates statistics, including count, mean, standard
deviation (std), min, percentiles (25%, 50%, 75%), and max.

Note that std uses a degree of freedom of N in accordance with
Formula D.5 of IEC TS 62600-100 Ed. 2.0 en 2024.

Parameters
------------
X: numpy array, pandas Series, pandas DataFrame, xarray DataArray, or xarray Dataset
    Data
to_pandas: bool (optional)
    Flag to output pandas instead of xarray. Default = True.

Returns
---------
```

```
stats: pandas Series or xarray DataArray
    Statistics
```

```
print(
    pd.DataFrame(
        {q: performance.statistics(data[q]) for q in ["Hm0", "Te", "J"]}
    ).to_string()
)
```

```
               Hm0           Te            J
index
count  2928.000000  2928.000000  2928.000000
mean      2.320099     9.866388    38.198468
std       1.029724     1.872971    46.197721
min       0.882500     5.692900     3.213000
25%       1.573435     8.485375    10.809500
50%       2.010700     9.678800    20.707500
75%       2.847345    10.968200    46.990750
max       7.529150    17.567600   449.944000
```

7 Time Series Analysis

The temporal data from the hindcast datasets provide the oppportunity understand how the wave resource changes throughout the year. Each subsection below is one view of the same single-point record, ordered from the least reduced to the most:

- **Monthly box plots** (Figure 2 through Figure 9) – the per-month spread of all eight downloaded variables.
- **Raw time series** (Figure 10 through Figure 13) – the unreduced 3-hourly record, which is what every metric and visualization derives from.
- **Distributions** (Figure 14 through Figure 21) – the same record with time set aside entirely, spread over value instead.
- **Daily climatology** (Figure 22 through Figure 24) and **rolling averages** (Figure 25, Figure 26) – day-of-year structure, and that structure smoothed until only the seasonal signal is left.
- **Monthly bars** (Figure 27 through Figure 30) – one labeled number per month, for the mean and for the 95th percentile.

```
QOIS = {
    "Hm0": "$H_{m0}$ [m]",
    "Te": "$T_e$ [s]",
    "J": "$J$ [kW/m]",
    "Tp": "$T_p$ [s]",
    "Dir": "Mean Wave\nDirection [deg]",
    "MaxEDir": "Maximum Energy\nDirection [deg]",
```

```
    "DirCoeff": "Directionality\nCoefficient [0-1]",
    "SpecWidth": "Spectral Width\n[0-1]",
}
months = np.arange(1, 13)
month_labels = [calendar.month_abbr[m] for m in months]  # 'Jan'..'Dec'
colors = sns.color_palette()  # default seaborn color cycle

doy = data.index.dayofyear
dgrp = data.groupby(doy)
month_starts = [pd.Timestamp(YEAR, m, 1).dayofyear for m in months]


def qcolor(q):
    """One fixed color per quantity, shared by every figure in this section."""
    return colors[list(QOIS).index(q)]


TS_LINEWIDTH = 1.5
MONTH_TICKS = [pd.Timestamp(YEAR, m, 1) for m in months]


def raw_timeseries(q, name):
    """The 3-hourly record of `q` as it comes: no aggregation, no smoothing.

    Every figure after this one reduces the record somehow -- per month, per day,
    per rolling window. This is the series those reductions are computed from, so
    the reader can see the storm-by-storm structure the aggregates smooth away.
    Deliberately bare: no band, no overlay, no legend (one line needs no key).
    """
    fig, ax = plt.subplots(figsize=(FIG_WIDTH, 3))
    ax.plot(data.index, data[q], color=qcolor(q), linewidth=TS_LINEWIDTH)
    ax.set(xlabel="Month", ylabel=QOIS[q])
    ax.margins(x=0)
    ax.set_xticks(MONTH_TICKS)
    ax.set_xticklabels(month_labels)
    fig.tight_layout()
    savefig(name)


def raw_timeseries_stack(qs, name):
    """The same raw records stacked on one shared time axis.

    Sharing the axis is the point: a storm shows up as a spike in all three at
    once, which is impossible to see when the series are separate figures. Only
    the bottom panel carries the month labels.
    """
```

```
    fig, axes = plt.subplots(
        len(qs), 1, figsize=(FIG_WIDTH, 2 * len(qs)), sharex=True
    )
    for ax, q in zip(axes, qs):
        ax.plot(data.index, data[q], color=qcolor(q), linewidth=TS_LINEWIDTH)
        ax.set_ylabel(QOIS[q])
        ax.margins(x=0)
    axes[-1].set_xlabel("Month")
    axes[-1].set_xticks(MONTH_TICKS)
    axes[-1].set_xticklabels(month_labels)
    fig.tight_layout()
    savefig(name)


def monthly_boxplot(q):
    """Box-and-whisker of the 3-hourly distribution of `q` per month."""
    fig, ax = plt.subplots(figsize=(FIG_WIDTH, 3))
    sns.boxplot(x=data.index.month, y=data[q], ax=ax, color=qcolor(q))
    ax.set(xlabel="Month", ylabel=QOIS[q])
    ax.set_xticks(range(12))
    ax.set_xticklabels(month_labels)
    fig.tight_layout()
    savefig(f"monthly_boxplot_{q.lower()}")


def daily_climatology(q):
    """Daily median with a 25-75% band and a 7-day average across the year."""
    fig, ax = plt.subplots(figsize=(FIG_WIDTH, 4))
    stats = dgrp[q].describe()
    days = stats.index
    ax.fill_between(days, stats["25%"], stats["75%"], color=qcolor(q),
                    alpha=0.3, label="Daily 25–75%")
    ax.plot(days, stats["50%"], color=qcolor(q), label="Daily Median")
    roll = stats["50%"].rolling(7, center=True, min_periods=1).mean()
    ax.plot(days, roll, color="0.4", linewidth=2, label="7-day avg")
    ax.set(xlabel="Month", ylabel=QOIS[q])
    ax.margins(x=0)
    ax.set_xticks(month_starts)
    ax.set_xticklabels(month_labels)
    ax.legend(loc="upper left", bbox_to_anchor=(1.02, 1), borderaxespad=0)
    fig.tight_layout()
    savefig(f"daily_climatology_{q.lower()}")


def rolling_median(q, window):
    """`window`-day moving average of the daily median of `q`."""
```

```
    fig, ax = plt.subplots(figsize=(FIG_WIDTH, 3.2))
    roll = dgrp[q].median().rolling(window, center=True, min_periods=1).mean()
    ax.plot(roll.index, roll, color=qcolor(q), label=f"{window}-day avg")
    ax.set(xlabel="Month", ylabel=QOIS[q])
    ax.margins(x=0)
    ax.set_xticks(month_starts)
    ax.set_xticklabels(month_labels)
    fig.tight_layout()
    savefig(f"rolling_{window}d_{q.lower()}")


HIST_BINS = 50


def distribution(q):
    """Histogram and KDE of the 3-hourly record of `q`, with its CDF on a twin axis.

    The time series above says when each sea state happened; this says how much
    of the year was spent at each value. Histogram and KDE share the left axis
    because both are densities: the bars are the binned record, the line is the
    same record smoothed, and disagreement between them means the bin width is
    doing the talking. The cumulative curve reads the record as a probability,
    and is the linear-axis complement of the log-axis survival functions in
    @sec-exceedance.

    `cut=0` stops the KDE at the observed range, which keeps it off physically
    impossible values (negative $J$, wave directions past 360 deg).
    """
    values = data[q].dropna()
    fig, ax = plt.subplots(figsize=(FIG_WIDTH, 3.2))
    ax.hist(values, bins=HIST_BINS, density=True, color=qcolor(q), alpha=0.55,
            label="Histogram")
    sns.kdeplot(x=values, ax=ax, color=qcolor(q), linewidth=2, cut=0, label="KDE")
    ax.set(xlabel=QOIS[q], ylabel="Probability density")
    ax.margins(x=0)

    cax = ax.twinx()
    ordered = np.sort(values.values)
    cdf = np.arange(1, ordered.size + 1) / ordered.size
    cax.plot(ordered, cdf, color="0.4", linewidth=2, label="Cumulative")
    cax.set_ylabel("Cumulative probability")
    cax.set_ylim(0, 1)
    cax.grid(False)  # seaborn's theme would otherwise draw a second grid

    handles = ax.get_legend_handles_labels()[0] + cax.get_legend_handles_labels()[0]
    ax.legend(handles, [h.get_label() for h in handles], loc="best")
```

```
    fig.tight_layout()
    savefig(f"distribution_{q.lower()}")
```

7.1 Monthly Box Plots

The box plots cover all eight downloaded variables: the three the rest of this section works from (H_{m0}, T_e, J), the peak period T_p, both direction variables, and the two spectral shape parameters. The direction box plots are raw dataset values in the meteorological convention.

```
monthly_boxplot("Hm0")
```

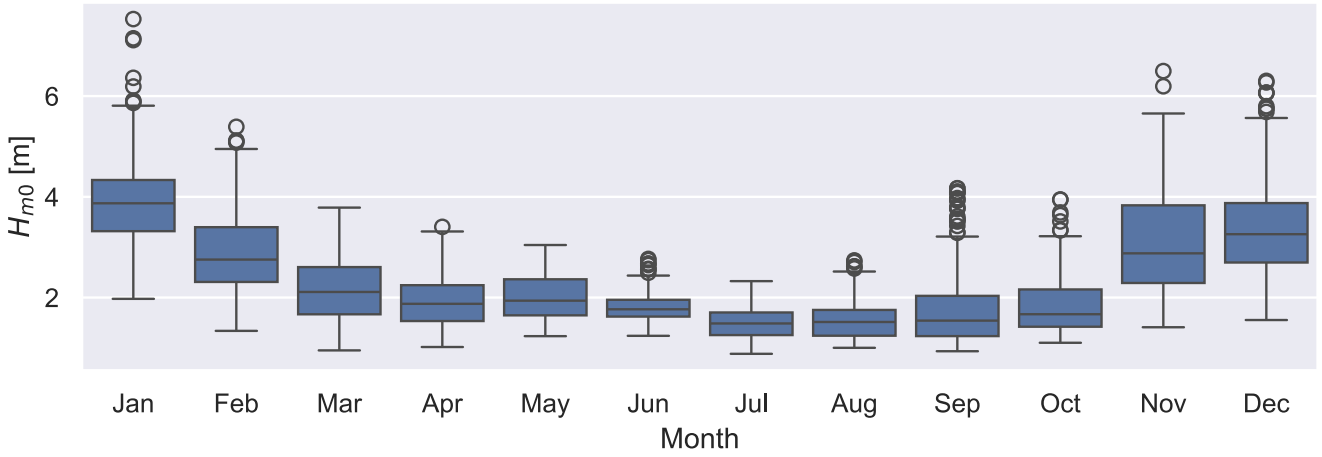

Figure 2: PacWave South (44.5670, −124.2290), monthly (2020) distribution of significant wave height (H_{m0}). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
monthly_boxplot("Te")
```

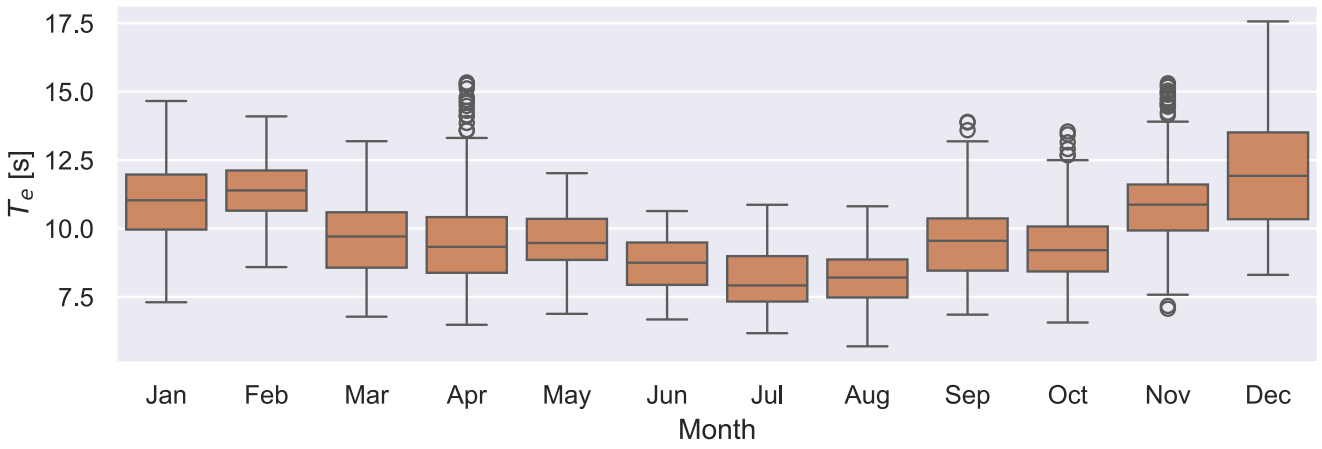

Figure 3: PacWave South (44.5670, −124.2290), monthly (2020) distribution of energy period (T_e). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
monthly_boxplot("J")
```

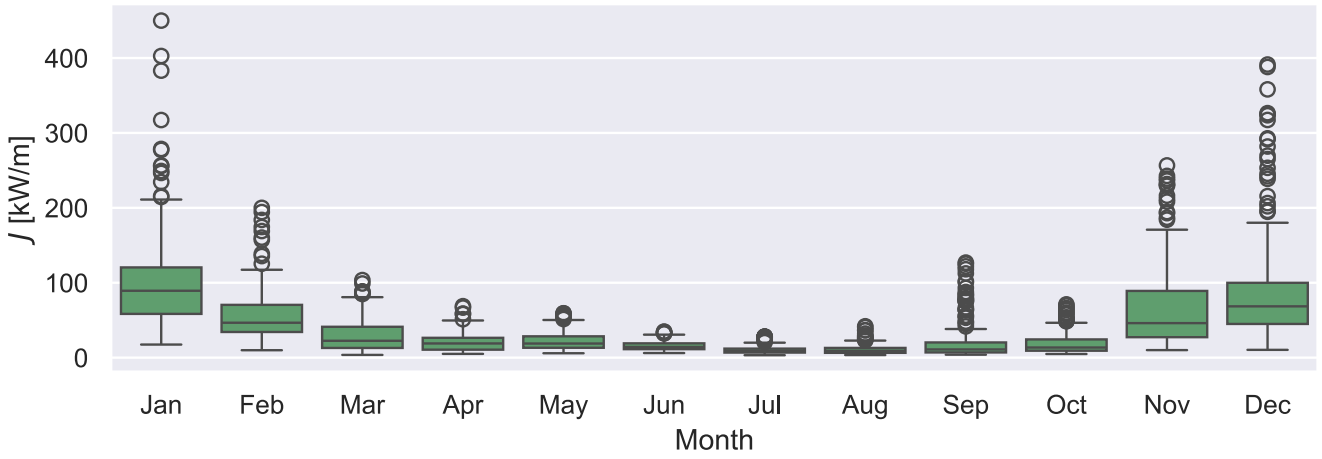

Figure 4: PacWave South (44.5670, −124.2290), monthly (2020) distribution of omnidirectional wave energy flux (J). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
monthly_boxplot("Tp")
```

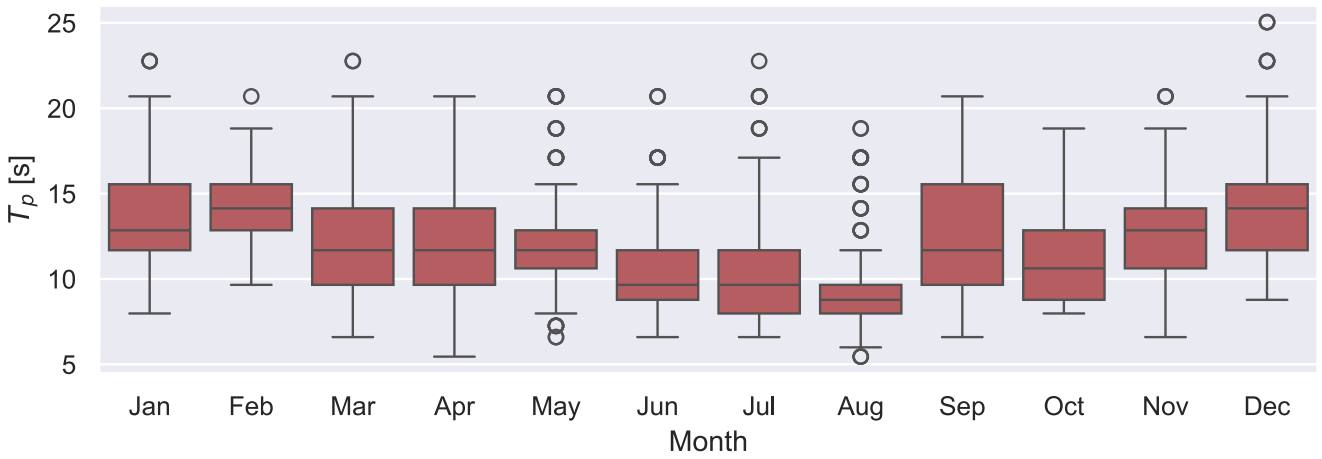

Figure 5: PacWave South (44.5670, −124.2290), monthly (2020) distribution of peak period (T_p). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
monthly_boxplot("Dir")
```

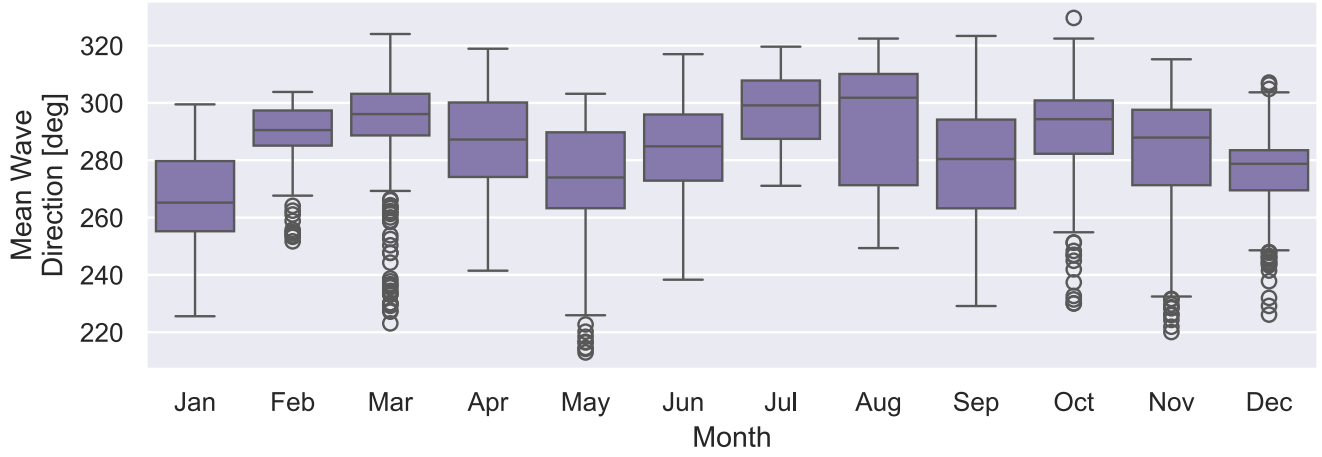

Figure 6: PacWave South (44.5670, −124.2290), monthly (2020) distribution of mean wave direction. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
monthly_boxplot("MaxEDir")
```

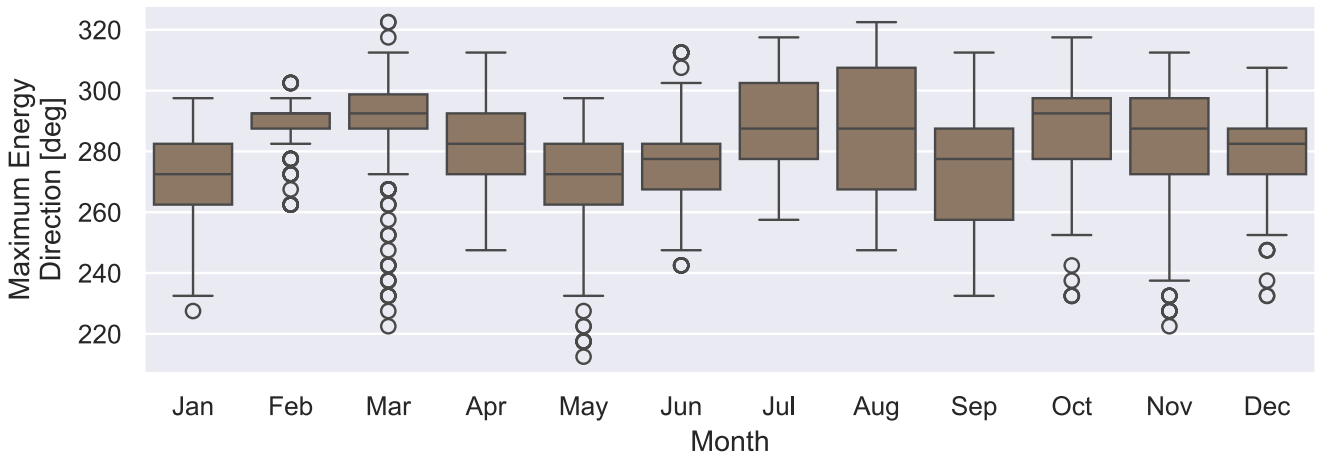

Figure 7: PacWave South (44.5670, −124.2290), monthly (2020) distribution of maximum energy direction. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
monthly_boxplot("DirCoeff")
```

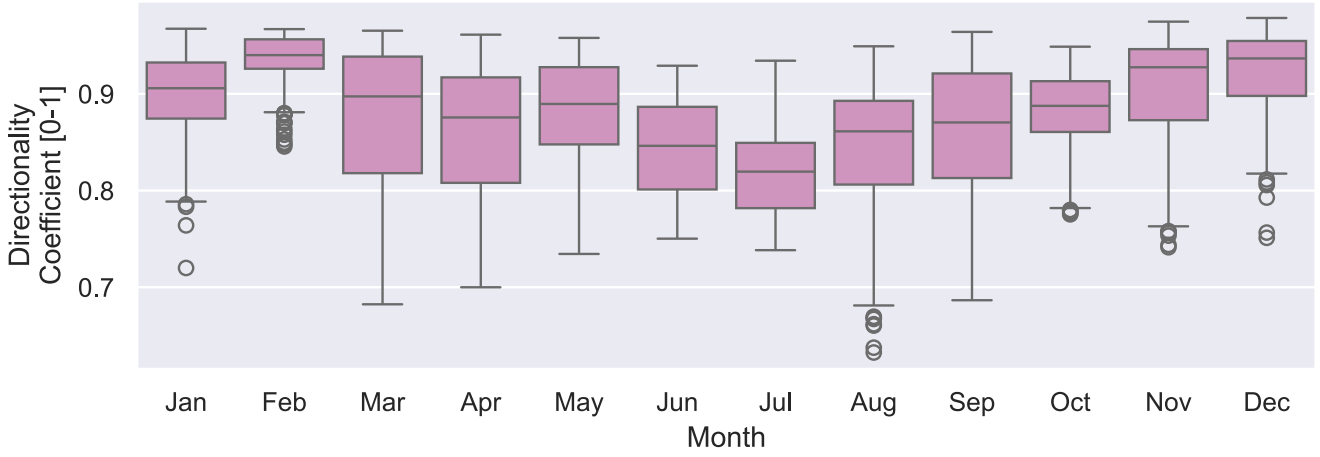

Figure 8: PacWave South (44.5670, −124.2290), monthly (2020) distribution of directionality coefficient. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
monthly_boxplot("SpecWidth")
```

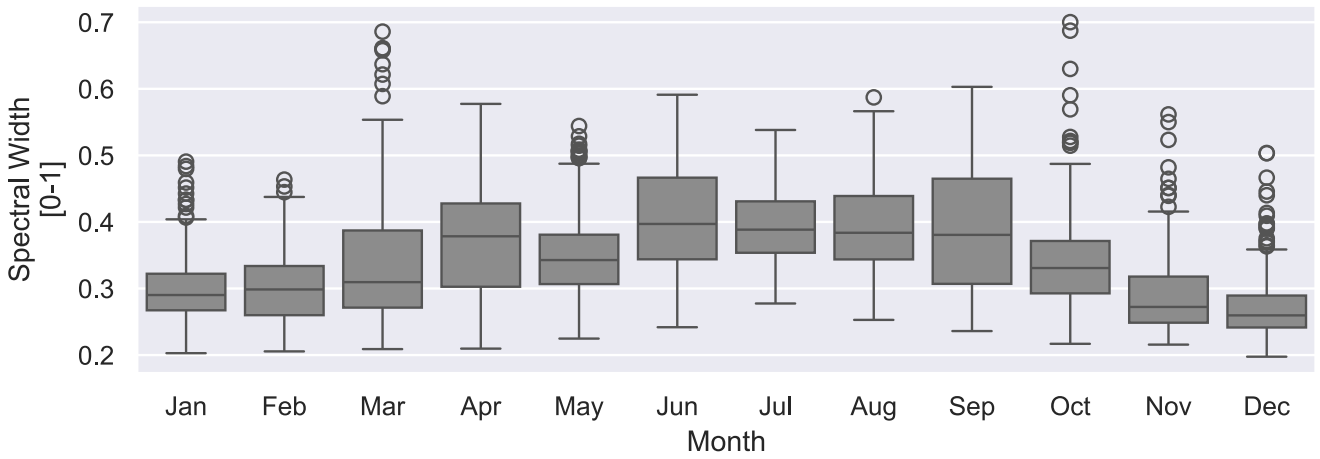

Figure 9: PacWave South (44.5670, −124.2290), monthly (2020) distribution of spectral width. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

7.2 Raw Time Series

Figure 10 through Figure 12 show the 3-hourly record as it comes, with no aggregation and no smoothing. Figure 13 puts the three on one shared time axis, where a storm reads as a simultaneous excursion in all three rather than three separate spikes.

```
raw_timeseries("Hm0", "timeseries_hm0")
```

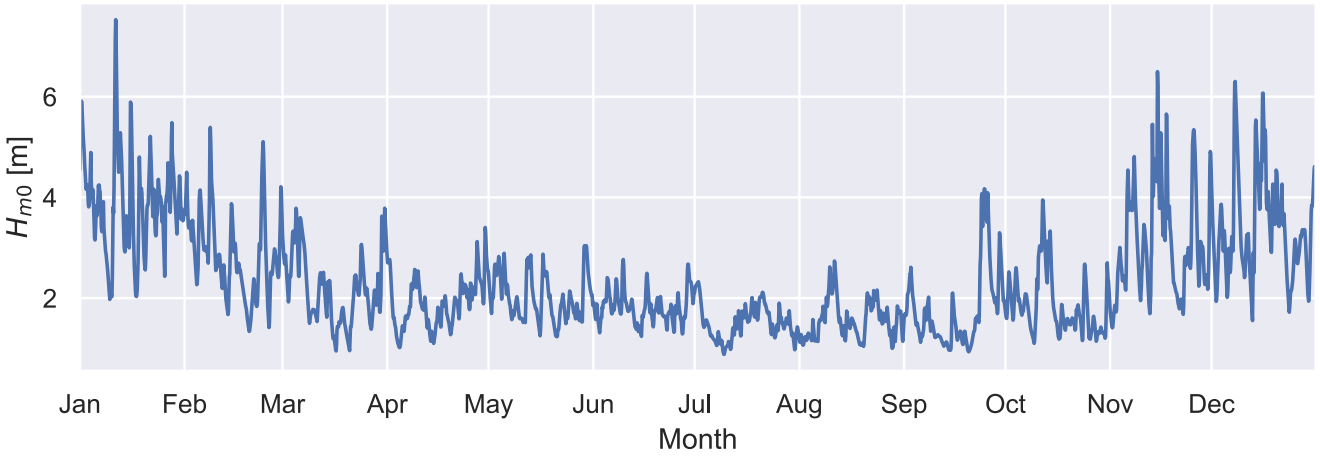

Figure 10: PacWave South (44.5670, −124.2290), 3-hourly (2020) record of significant wave height (H_{m0}). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
raw_timeseries("Te", "timeseries_te")
```

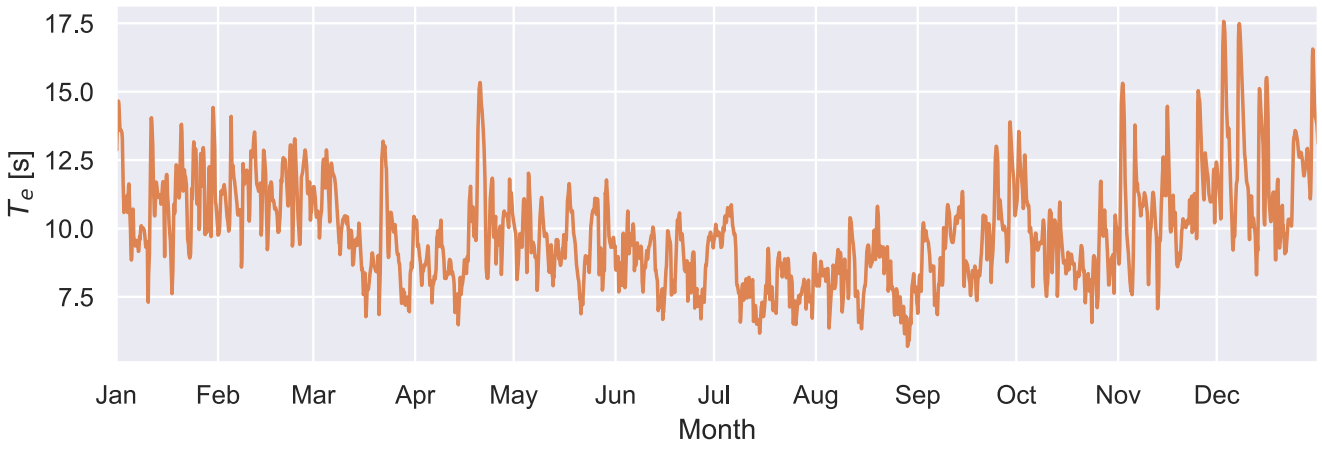

Figure 11: PacWave South (44.5670, −124.2290), 3-hourly (2020) record of energy period (T_e). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
raw_timeseries("J", "timeseries_j")
```

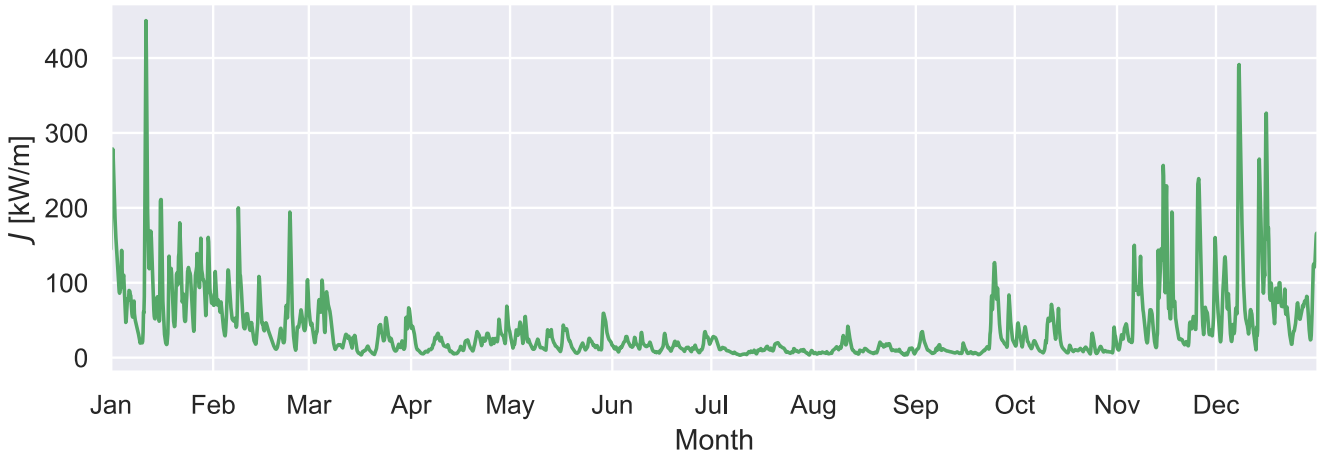

Figure 12: PacWave South (44.5670, −124.2290), 3-hourly (2020) record of omnidirectional wave energy flux (J). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
raw_timeseries_stack(["Hm0", "Te", "J"], "timeseries_stack")
```

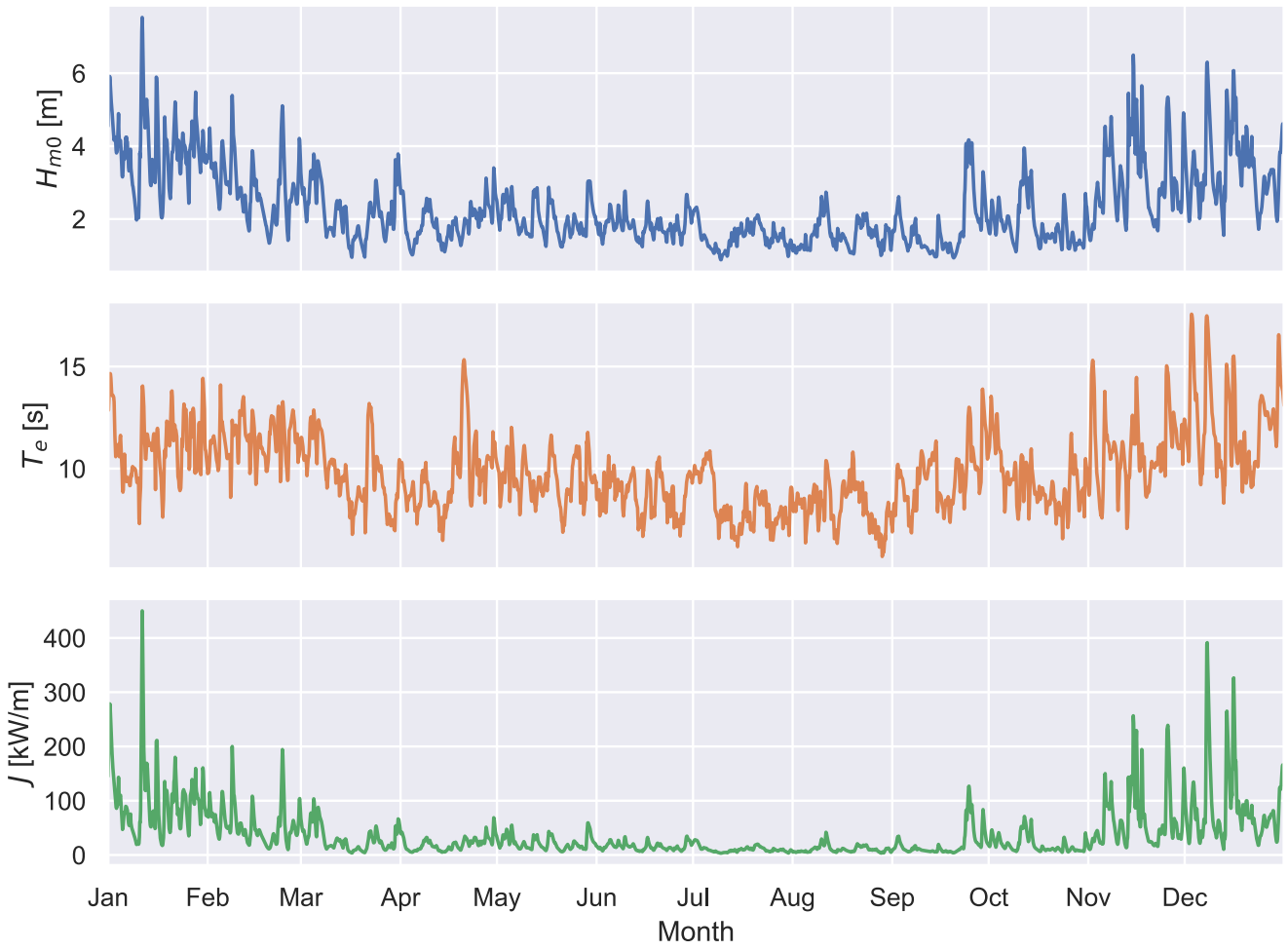

Figure 13: PacWave South (44.5670, −124.2290), 3-hourly (2020) records of significant wave height (H_{m0}), energy period (T_e), and omnidirectional wave energy flux (J) on a shared time axis. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

7.3 Distributions

Every figure above is indexed by time. These are not: each histogram bins the same 3-hourly record into 50 bins over the value of the quantity, so what it shows is how much of the year was spent at each sea state, not when. Bars and KDE line are both densities on the left axis – the binned record and its smoothed version – so where they disagree, the bin width is doing the talking. The grey cumulative curve on the twin axis reads that record as a probability, $P(q \leq x)$, on a linear axis. Its upper tail is the same information Section 10 re-plots as the survival function $P(q > x)$ on a log axis for H_{m0} and J, where the rare-but-large sea states that matter for survival loads are legible.

Like the box plots, this covers all eight downloaded variables (Figure 14 through Figure 21). The two direction variables are the ones to read carefully: direction is circular, so a KDE fitted on a linear 0-360

degree axis cannot wrap, and a distribution straddling north would show as two edge peaks rather than one. Section 11 plots those on a polar axis instead.

```
distribution("Hm0")
```

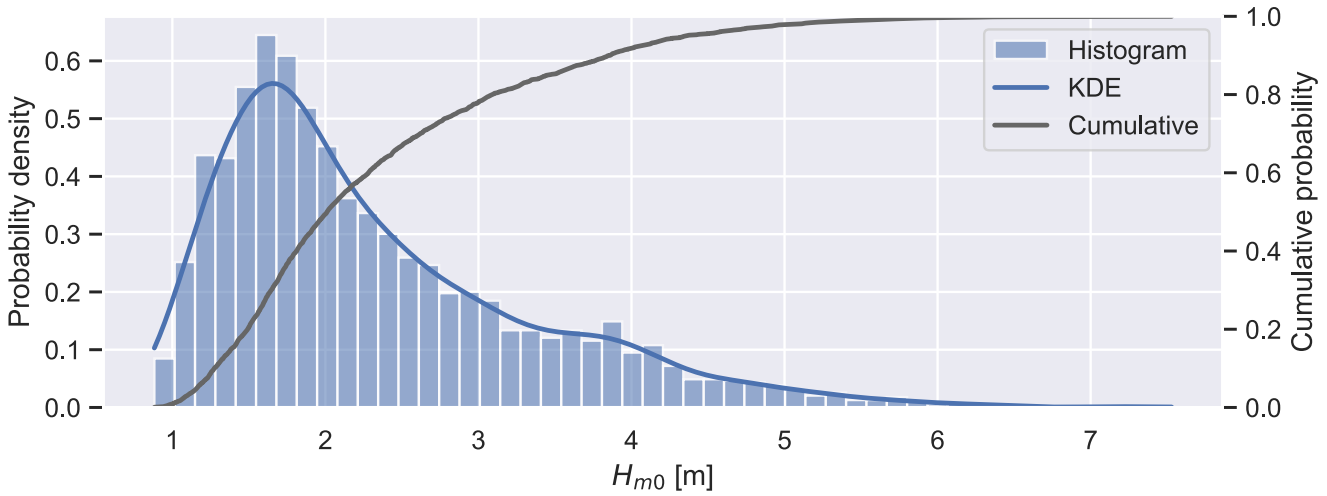

Figure 14: PacWave South (44.5670, −124.2290), distribution (50 bins) of the 3-hourly (2020) significant wave height (H_{m0}) record, with its kernel density estimate and cumulative distribution. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
distribution("Te")
```

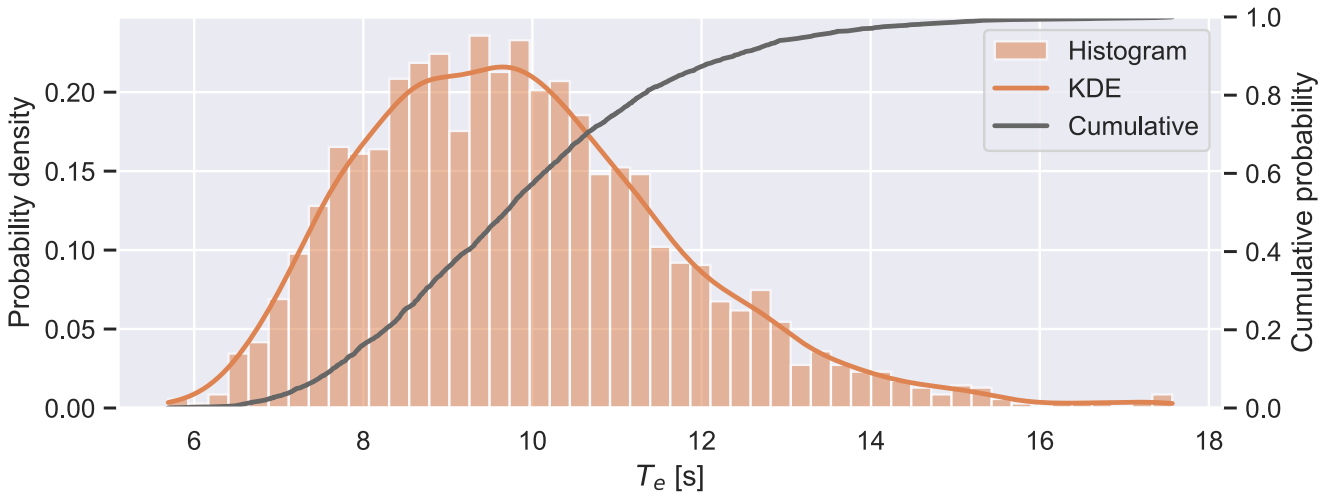

Figure 15: PacWave South (44.5670, −124.2290), distribution (50 bins) of the 3-hourly (2020) energy period (T_e) record, with its kernel density estimate and cumulative distribution. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
distribution("J")
```

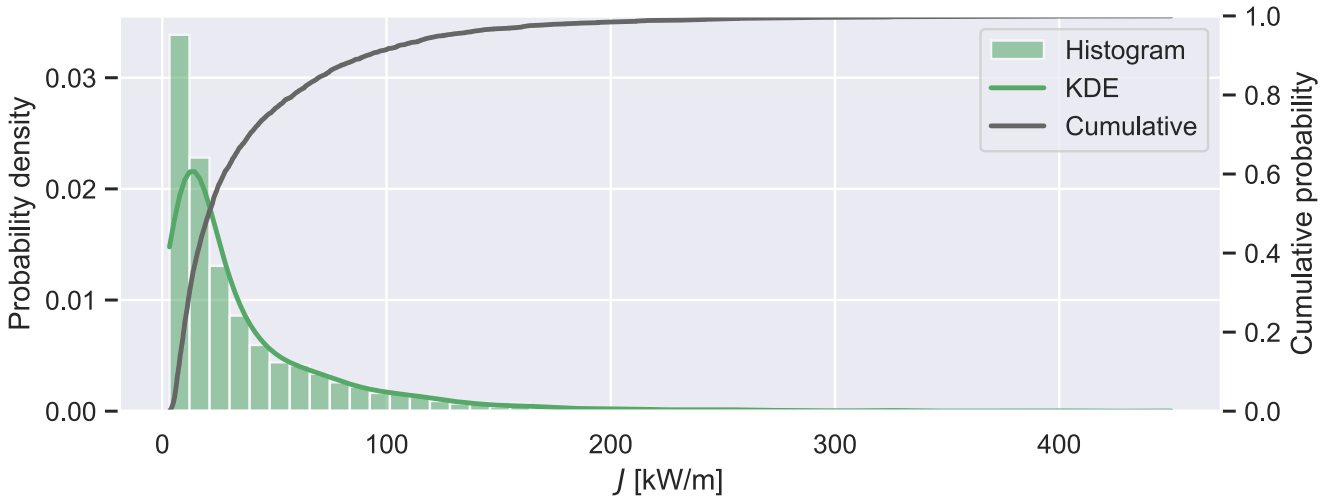

Figure 16: PacWave South (44.5670, −124.2290), distribution (50 bins) of the 3-hourly (2020) omnidirectional wave energy flux (J) record, with its kernel density estimate and cumulative distribution. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
distribution("Tp")
```

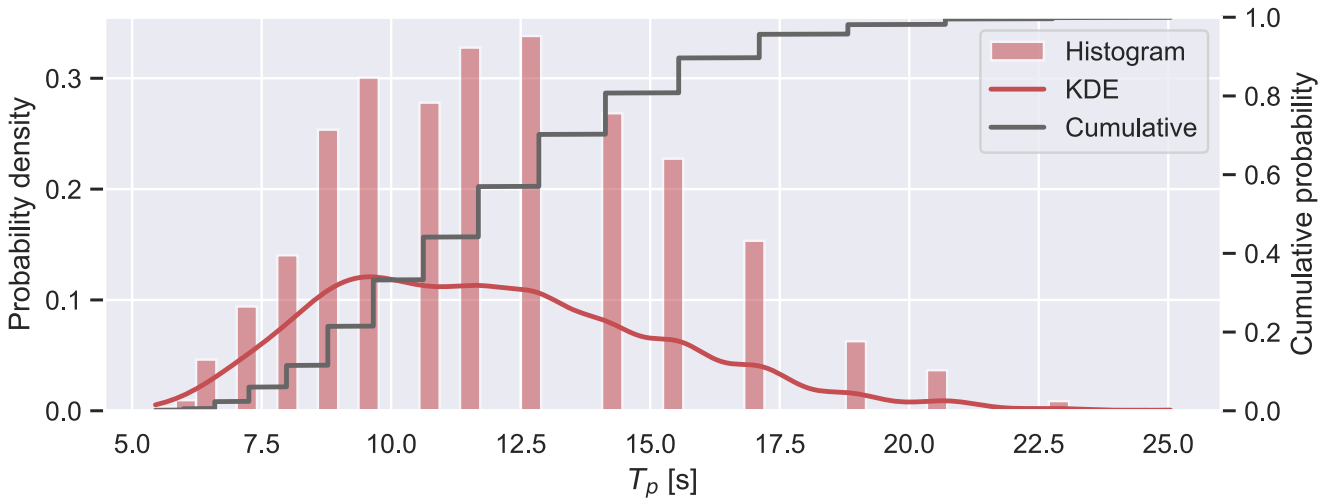

Figure 17: PacWave South (44.5670, −124.2290), distribution (50 bins) of the 3-hourly (2020) peak period (T_p) record, with its kernel density estimate and cumulative distribution. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
distribution("Dir")
```

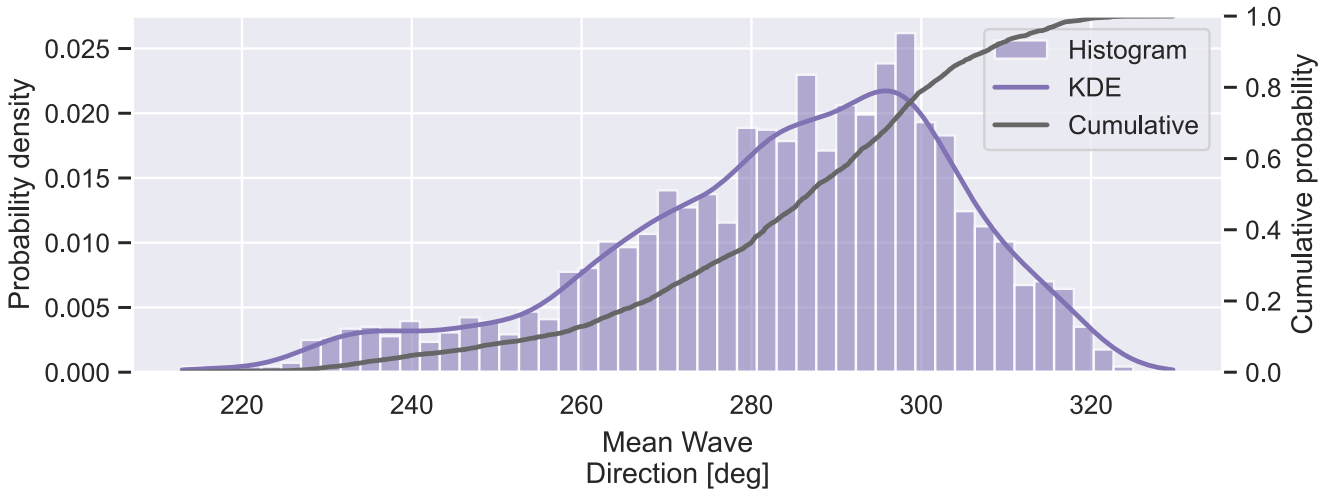

Figure 18: PacWave South (44.5670, −124.2290), distribution (50 bins) of the 3-hourly (2020) mean wave direction record, with its kernel density estimate and cumulative distribution. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
distribution("MaxEDir")
```

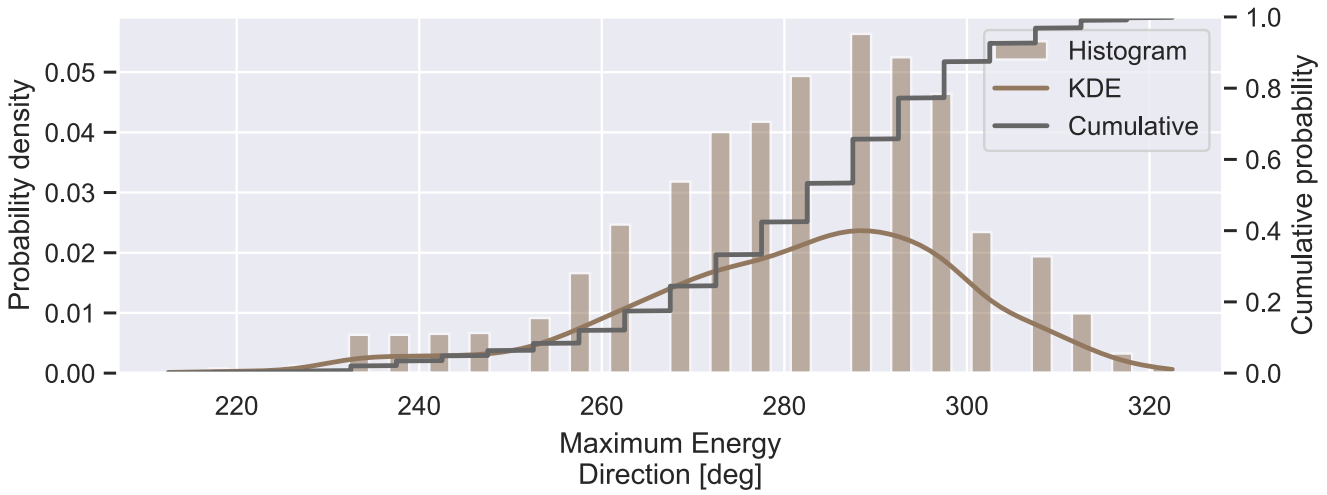

Figure 19: PacWave South (44.5670, −124.2290), distribution (50 bins) of the 3-hourly (2020) maximum energy direction record, with its kernel density estimate and cumulative distribution. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
distribution("DirCoeff")
```

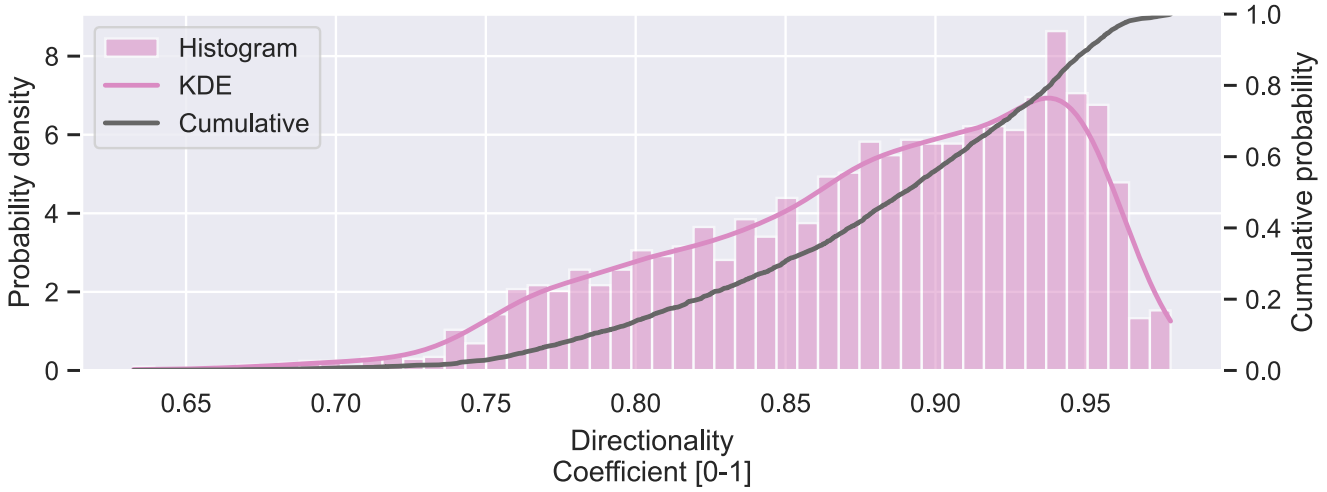

Figure 20: PacWave South (44.5670, −124.2290), distribution (50 bins) of the 3-hourly (2020) directionality coefficient record, with its kernel density estimate and cumulative distribution. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
distribution("SpecWidth")
```

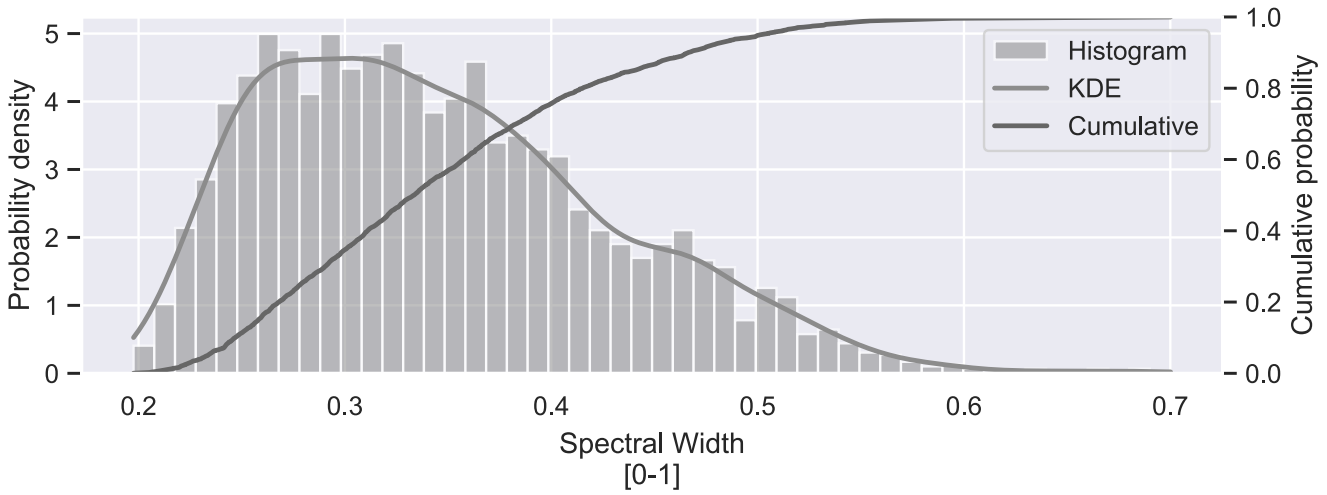

Figure 21: PacWave South (44.5670, −124.2290), distribution (50 bins) of the 3-hourly (2020) spectral width record, with its kernel density estimate and cumulative distribution. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

7.4 Daily Climatology

Grouping by day of year collapses the eight values in each day to a median and a 25-75% band, so the day-to-day structure survives but the within-day scatter does not.

```
daily_climatology("Hm0")
```

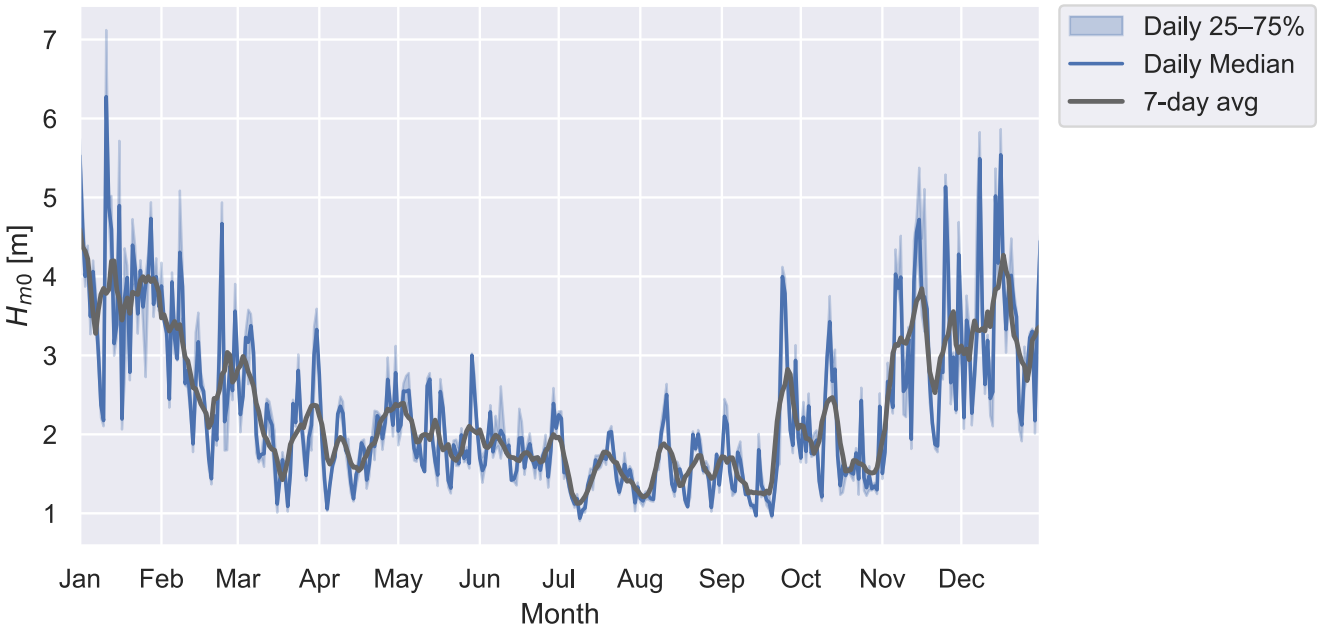

Figure 22: PacWave South (44.5670, −124.2290), daily (2020) significant wave height (H_{m0}): median, 25-75% band, and 7-day average. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
daily_climatology("Te")
```

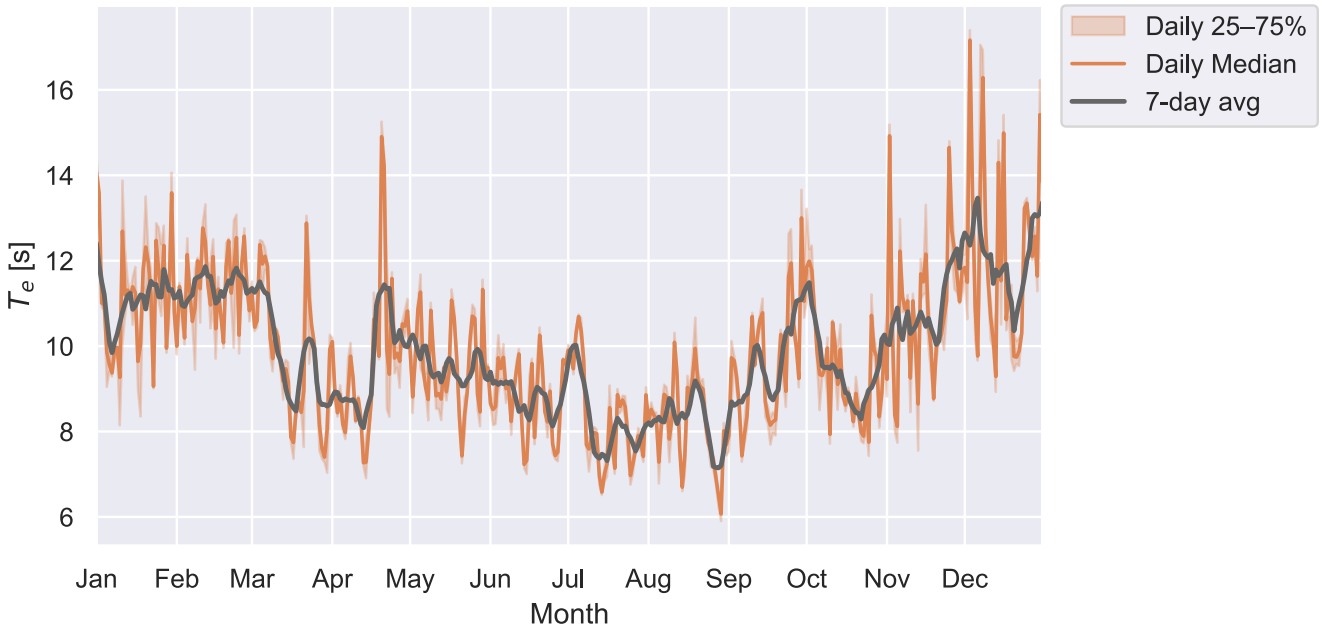

Figure 23: PacWave South (44.5670, −124.2290), daily (2020) energy period (T_e): median, 25-75% band, and 7-day average. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
daily_climatology("J")
```

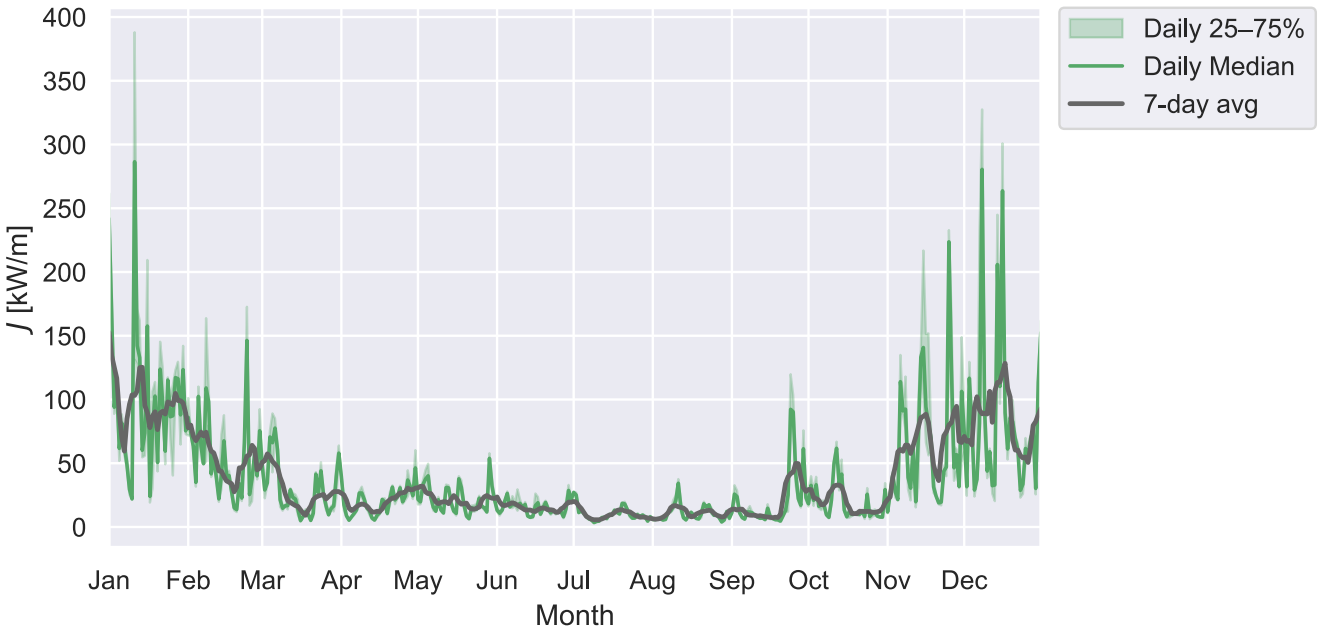

Figure 24: PacWave South (44.5670, −124.2290), daily (2020) omnidirectional wave energy flux (J): median, 25-75% band, and 7-day average. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

7.5 Rolling Averages

A 7-day average still tracks individual storm systems; a 30-day average smooths past them and leaves the seasonal signal on its own (Figure 25, Figure 26).

```
rolling_median("J", 7)
```

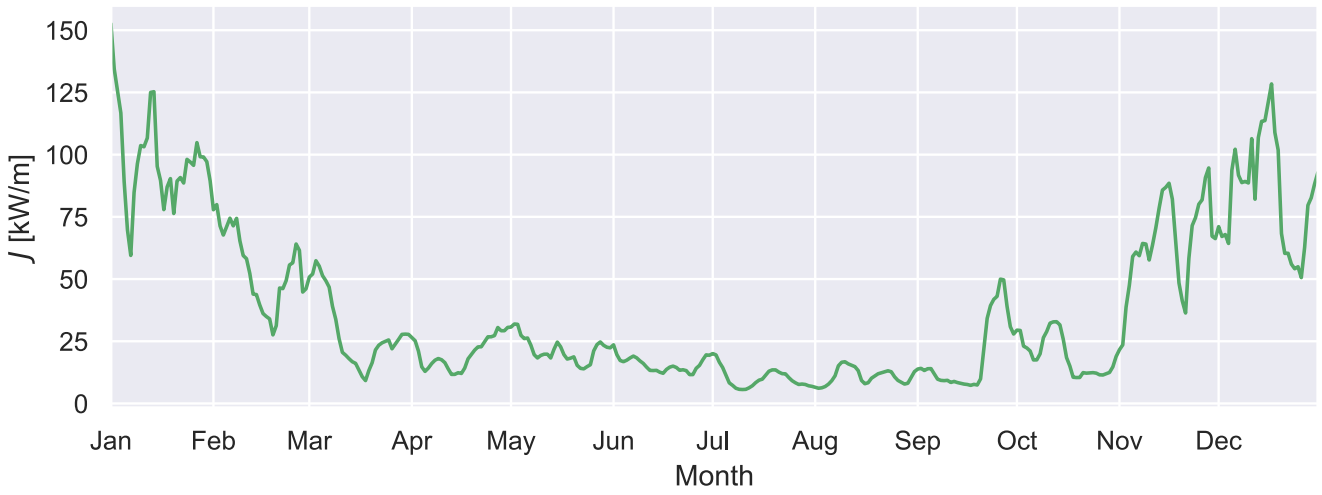

Figure 25: PacWave South (44.5670, −124.2290), 7-day moving average (2020) of the daily median omnidirectional wave energy flux (J). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
rolling_median("J", 30)
```

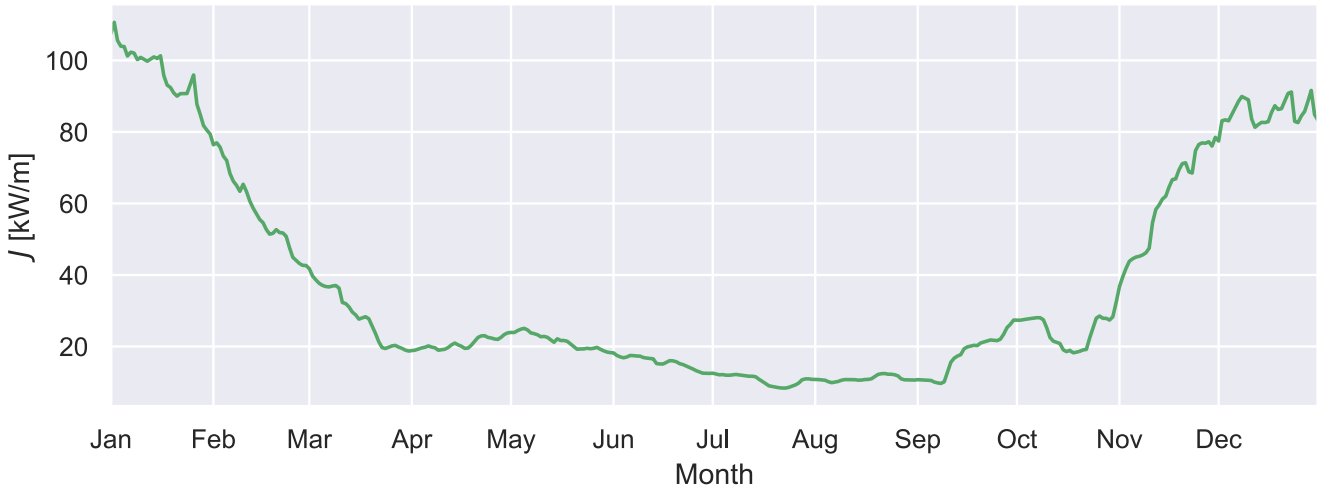

Figure 26: PacWave South (44.5670, −124.2290), 30-day moving average (2020) of the daily median omnidirectional wave energy flux (J). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

7.6 Monthly Bars

One labeled bar per month (Figure 27 through Figure 30). Mean and 95th percentile are separate figures: the mean is roughly what a device harvests in an average month, the 95th percentile is the tail it must survive, and paired bars on one axis would let the tail squash the mean flat.

```
def monthly_bar(q, stat_label, agg, fmt, name):
    """One labeled bar per month of `q`, aggregated by `agg`."""
    monthly = data[q].groupby(data.index.month).agg(agg)
    fig, ax = plt.subplots(figsize=(FIG_WIDTH, 4))
    bars = ax.bar(months, monthly.reindex(months).values, color=qcolor(q))
    ax.bar_label(bars, fmt=fmt, padding=2, fontsize=9, color="0.25")
    ax.set(xlabel="Month", ylabel=f"{stat_label} {QOIS[q]}")
    ax.set_xticks(months)
    ax.set_xticklabels(month_labels)
    ax.margins(y=0.14)  # headroom so the tallest bar's label is not clipped
    fig.tight_layout()
    savefig(name)


p95 = lambda s: s.quantile(0.95)
```

```
monthly_bar("Hm0", "Mean", "mean", "%.2f", "monthly_bar_hm0_mean")
```

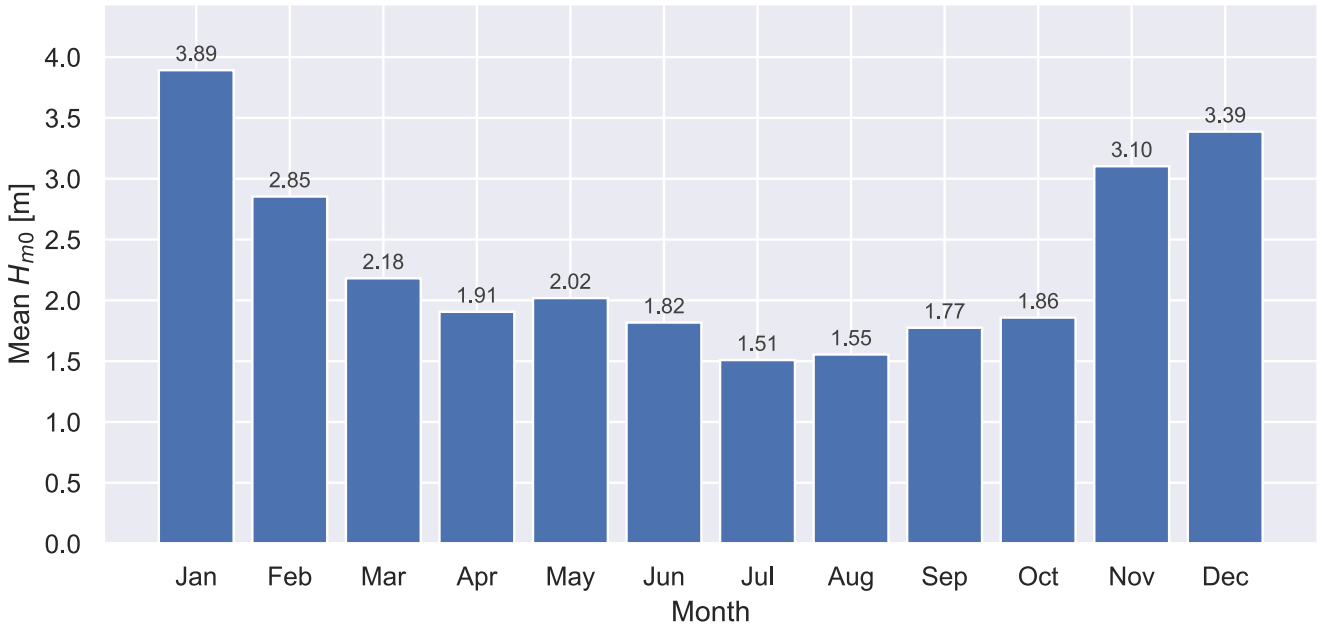

Figure 27: PacWave South (44.5670, −124.2290), mean significant wave height (H_{m0}) per month (2020). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
monthly_bar("Hm0", "95th percentile", p95, "%.2f", "monthly_bar_hm0_p95")
```

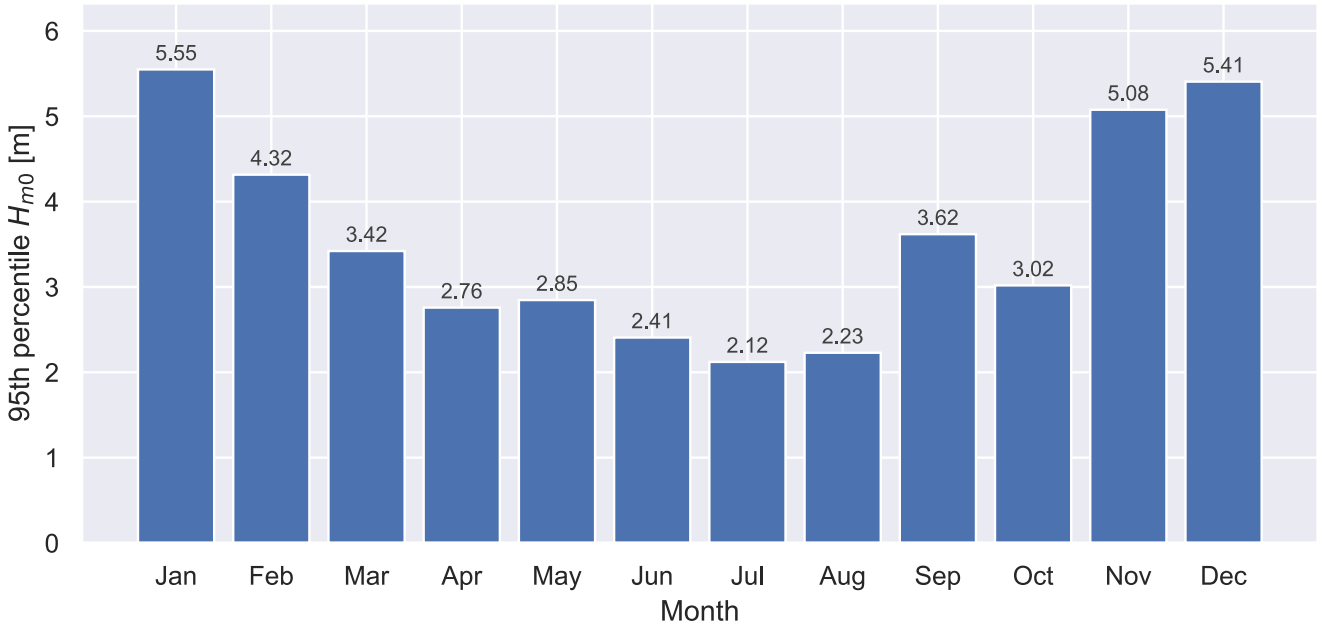

Figure 28: PacWave South (44.5670, −124.2290), 95th percentile significant wave height (H_{m0}) per month (2020). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
monthly_bar("J", "Mean", "mean", "%.1f", "monthly_bar_j_mean")
```

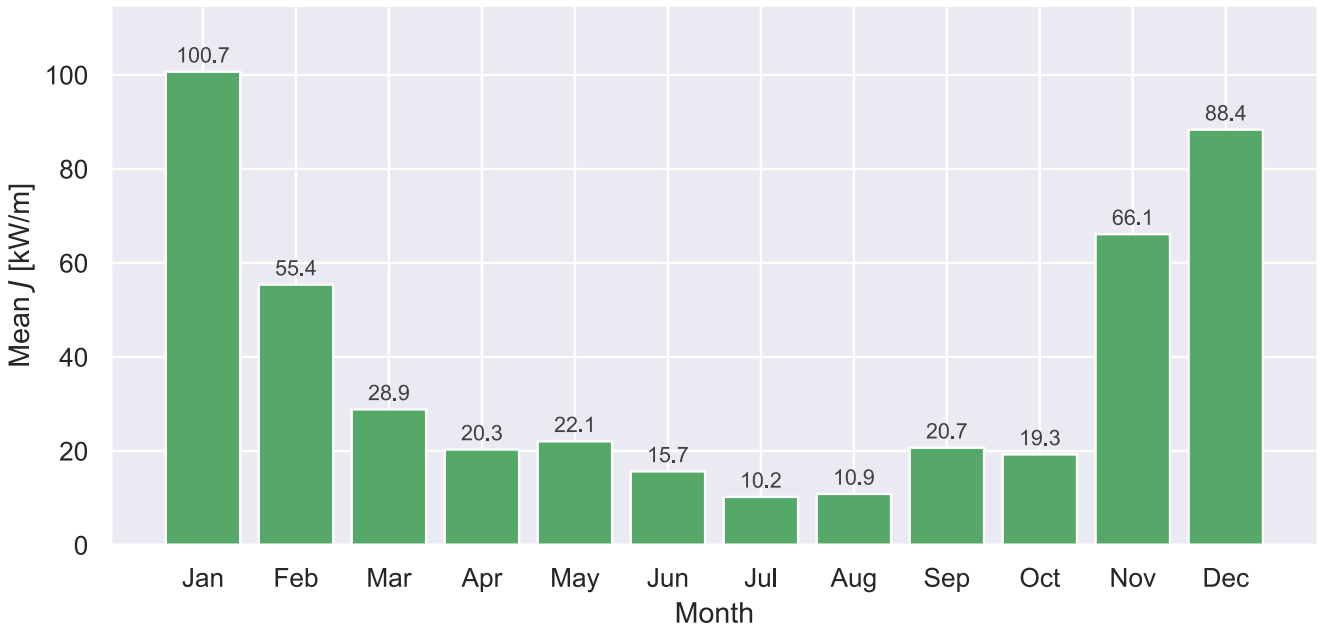

Figure 29: PacWave South (44.5670, −124.2290), mean omnidirectional wave energy flux (J) per month (2020). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
monthly_bar("J", "95th percentile", p95, "%.1f", "monthly_bar_j_p95")
```

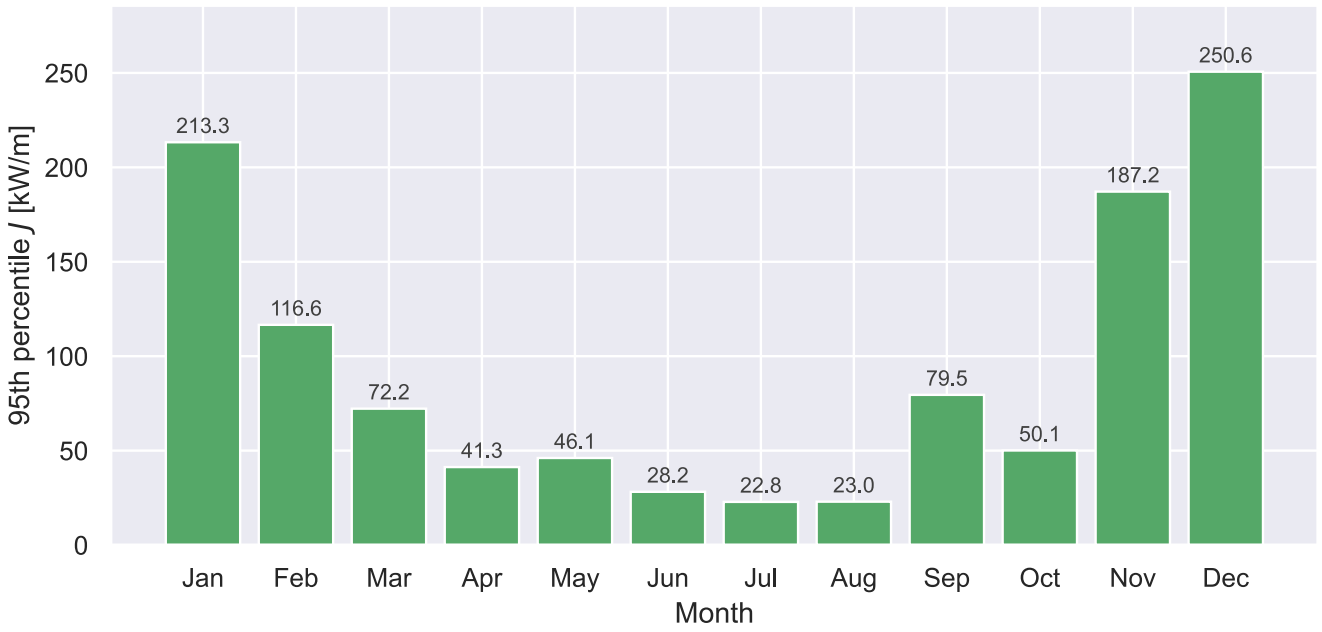

Figure 30: PacWave South (44.5670, −124.2290), 95th percentile omnidirectional wave energy flux (J) per month (2020). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

8 Joint Probability Distribution

Joint probability of occurrence (percent of the year) over the (H_{m0}, T_e) sea-state bins (Figure 31). This matrix is the resource half of device performance: paired with a device power matrix it becomes AEP in Section 12. Figure 32 is the same matrix without the per-cell numbers, for reading the distribution's shape rather than looking values up in it.

```
# Bin EDGES. `np.histogram2d` (the polar JPDs in @sec-directionality) takes edges.
Hm0_bins = np.arange(0, np.ceil(data.Hm0.max()) + 0.5, 0.5)
Te_bins = np.arange(0, np.ceil(data.Te.max()) + 1, 1)

# mhkit's matrix builders take bin CENTERS, not edges -- `wave_energy_flux_matrix`
# converts what it is handed from centers to edges internally. Passing the edge
# array straight in would silently offset every bin by half a width, putting the
# Hm0 boundaries at 0.25/0.75/... instead of the IEC 0/0.5/... So hand it the
# midpoints, which puts the bins where the edge arrays above say they are.
Hm0_centers = (Hm0_bins[:-1] + Hm0_bins[1:]) / 2
Te_centers = (Te_bins[:-1] + Te_bins[1:]) / 2

# Joint probability of occurrence (reused as the AEP frequency matrix in @sec-aep).
frequency = performance.wave_energy_flux_matrix(
    data.Hm0, data.Te, data.J, "frequency", Hm0_centers, Te_centers
```

```
)
print(frequency.to_string())
```

```
y_centers  0.5   1.5   2.5   3.5   4.5       5.5       6.5       7.5       8.5       9.5
10.5      11.5      12.5      13.5      14.5      15.5      16.5      17.5
x_centers
0.25         0.0   0.0   0.0   0.0   0.0  0.000000  0.000000  0.000000  0.000000  0.000000
0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000
0.75         0.0   0.0   0.0   0.0   0.0  0.000000  0.000000  0.003074  0.002391  0.001025
0.001025  0.001025  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000
1.25         0.0   0.0   0.0   0.0   0.0  0.001708  0.015710  0.056011  0.067964  0.028347
0.023566  0.003074  0.000342  0.000342  0.000342  0.000000  0.000000  0.000000
1.75         0.0   0.0   0.0   0.0   0.0  0.000000  0.012637  0.056352  0.082309  0.074112
0.039276  0.010929  0.005123  0.002049  0.002732  0.002391  0.000000  0.000000
2.25         0.0   0.0   0.0   0.0   0.0  0.000000  0.000342  0.012295  0.026639  0.061475
0.040642  0.017077  0.008880  0.006489  0.000683  0.000683  0.000000  0.000000
2.75         0.0   0.0   0.0   0.0   0.0  0.000000  0.000342  0.001366  0.010246  0.020492
0.033128  0.028005  0.013320  0.002732  0.000342  0.000000  0.001025  0.000342
3.25         0.0   0.0   0.0   0.0   0.0  0.000000  0.000000  0.000683  0.004440  0.008880
0.016052  0.022883  0.014003  0.003415  0.000000  0.000342  0.000342  0.001025
3.75         0.0   0.0   0.0   0.0   0.0  0.000000  0.000000  0.000683  0.003415  0.010929
0.014344  0.016393  0.014003  0.002049  0.002049  0.001025  0.000342  0.000342
4.25         0.0   0.0   0.0   0.0   0.0  0.000000  0.000000  0.000000  0.000000  0.005806
0.008197  0.008880  0.008197  0.005123  0.001708  0.000000  0.000000  0.000000
4.75         0.0   0.0   0.0   0.0   0.0  0.000000  0.000000  0.000000  0.000000  0.002049
0.004440  0.006148  0.002732  0.004440  0.002049  0.000342  0.000342  0.000000
5.25         0.0   0.0   0.0   0.0   0.0  0.000000  0.000000  0.000000  0.000000  0.000683
0.001025  0.002391  0.001025  0.001366  0.004098  0.001025  0.000342  0.000000
5.75         0.0   0.0   0.0   0.0   0.0  0.000000  0.000000  0.000000  0.000000  0.000000
0.000000  0.001708  0.000000  0.000683  0.001025  0.001025  0.000683  0.000342
6.25         0.0   0.0   0.0   0.0   0.0  0.000000  0.000000  0.000000  0.000000  0.000000
0.000000  0.000683  0.000342  0.000342  0.000000  0.000683  0.000000  0.001025
6.75         0.0   0.0   0.0   0.0   0.0  0.000000  0.000000  0.000000  0.000000  0.000000
0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000
7.25         0.0   0.0   0.0   0.0   0.0  0.000000  0.000000  0.000000  0.000000  0.000000
0.000000  0.000000  0.000000  0.000342  0.000342  0.000000  0.000000  0.000000
7.75         0.0   0.0   0.0   0.0   0.0  0.000000  0.000000  0.000000  0.000000  0.000000
0.000000  0.000000  0.000000  0.000000  0.000342  0.000000  0.000000  0.000000
```

```
plot_wave_matrix(frequency * 100, zlabel="Joint Probability [% of year]")
savefig("jpd_frequency_matrix")
```

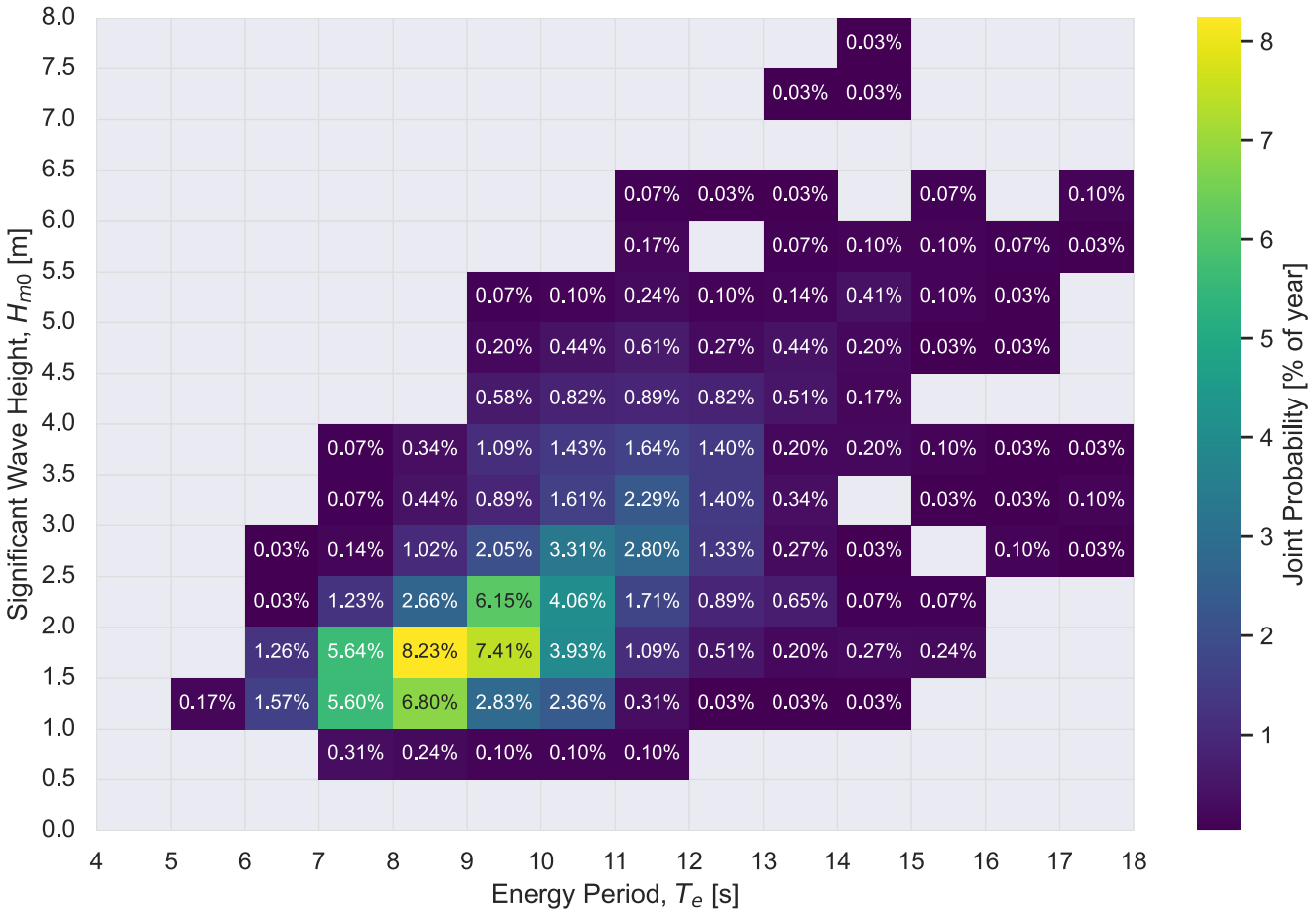

Figure 31: PacWave South (44.5670, −124.2290), joint probability of (H_{m0}, T_e) sea states, percent of the year (2020). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
plot_wave_matrix(
    frequency * 100, zlabel="Joint Probability [% of year]", annotate=False
)
savefig("jpd_frequency_matrix_plain")
```

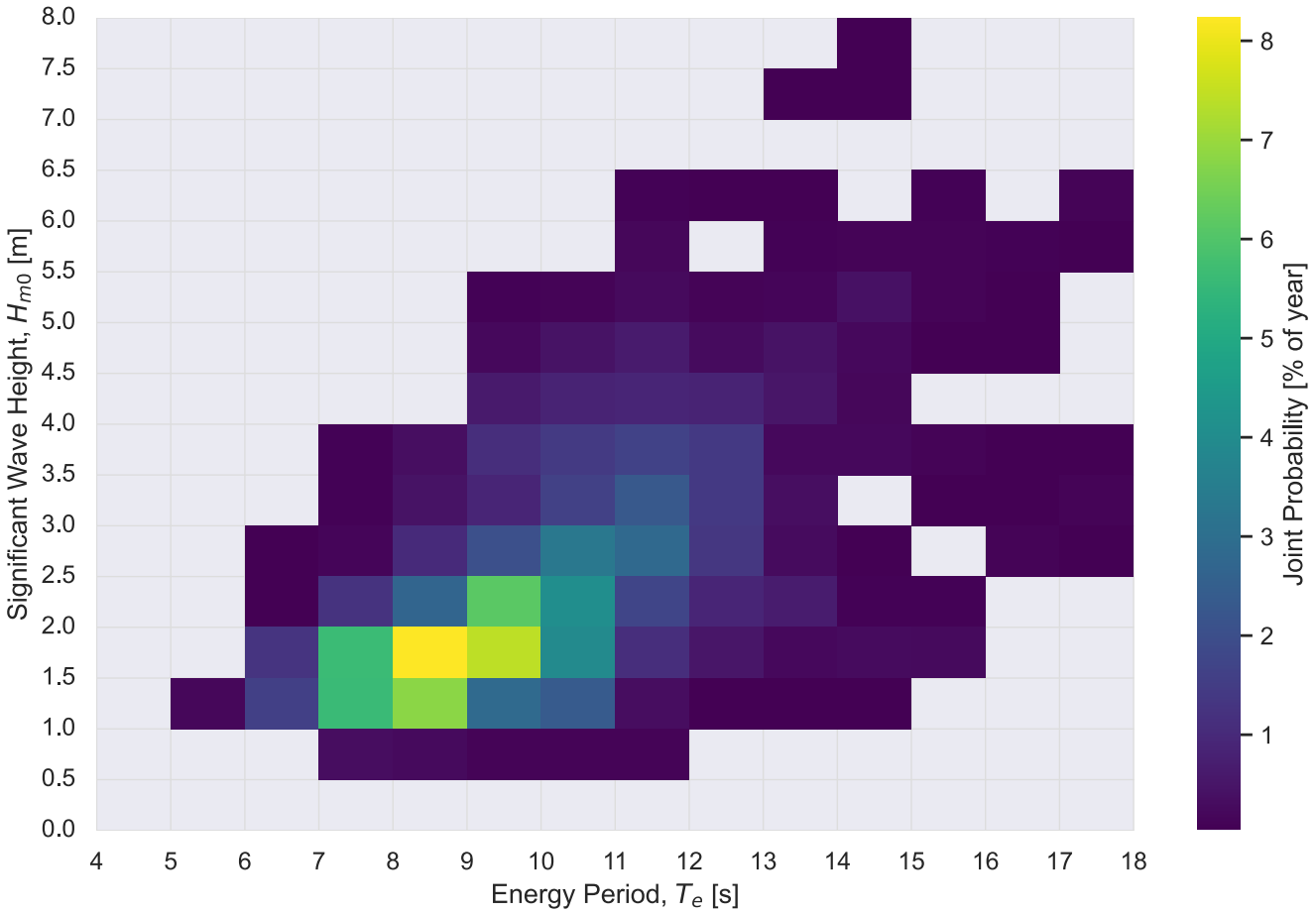

Figure 32: PacWave South (44.5670, −124.2290), joint probability of (H_{m0}, T_e) sea states, percent of the year (2020), unannotated. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

9 Extreme Sea States

Fifty- and one-hundred-year environmental contours bound the extreme sea states a device must survive (Figure 33). This example uses the `PCA` method throughout, following the Wave Energy Converter Design Response Toolbox [2]: it applies principal component analysis to (H_{m0}, T_e), fits an inverse Gaussian in the rotated frame, and reads the contour off a circle of iso-probability using the inverse first-order reliability method (I-FORM) [3].

Two caveats. `environmental_contours` offers ten methods, and the choice moves the 100-year H_{m0} by meters, more than the point or the record length does. The values below are PCA's answer, not the only answer (the full ten-method comparison lives in the experiment version of this example). A single hindcast year is illustrative, not a formal extreme-value estimate. Section 14 quantifies how much the record length moves the result.

```
# Seconds between sea-state samples (3-hour hindcast -> 10800 s).
dt = (data.index[1] - data.index[0]).total_seconds()

copulas50 = contours.environmental_contours(data.Hm0, data.Te, dt, 50, method="PCA")
copulas100 = contours.environmental_contours(data.Hm0, data.Te, dt, 100, method="PCA")

Hm0_contours = [copulas50["PCA_x1"], copulas100["PCA_x1"]]
Te_contours = [copulas50["PCA_x2"], copulas100["PCA_x2"]]

fig, ax = plt.subplots(figsize=(FIG_WIDTH, 4))
graphics.plot_environmental_contour(
    data.Te.values,
    data.Hm0.values,
    Te_contours,
    Hm0_contours,
    data_label=POINT_INFO[SELECTED_POINT]["label"],
    contour_label=["50-year contour", "100-year contour"],
    x_label="Energy Period, $T_e$ [s]",
    y_label="Significant Wave Height, $H_{m0}$ [m]",
    ax=ax,
)
ax.legend(loc="upper left")
fig.tight_layout()
savefig("extreme_contours")
```

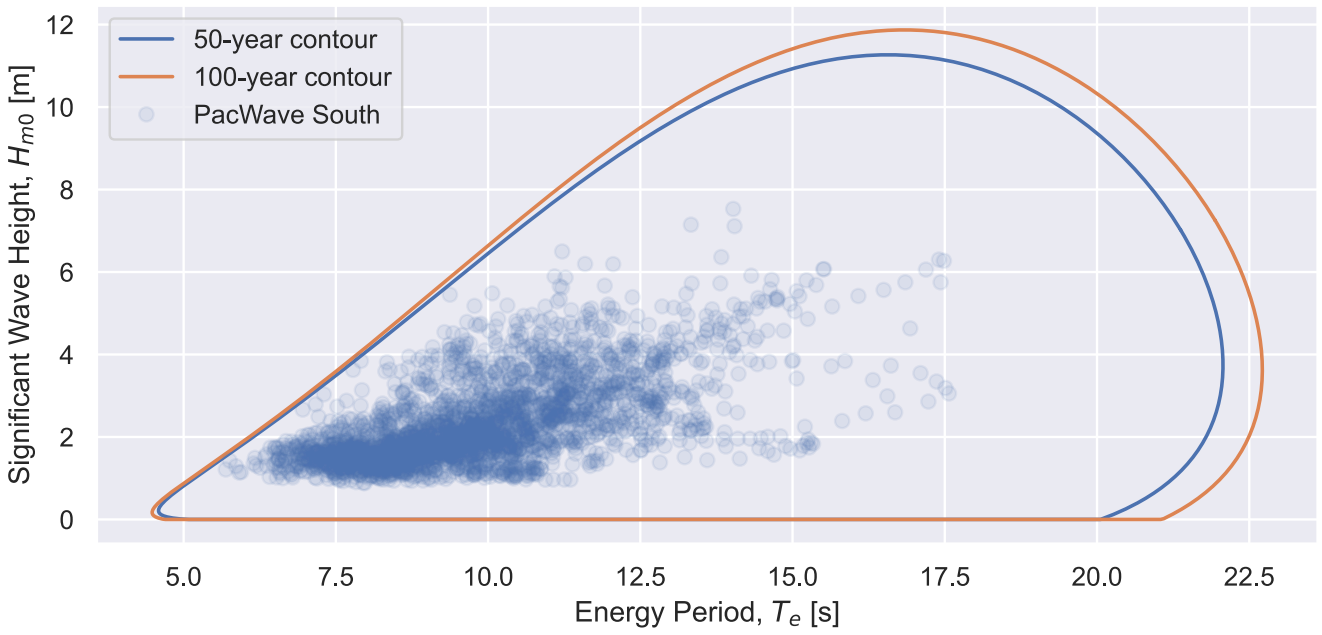

Figure 33: PacWave South (44.5670, −124.2290), 50- and 100-year environmental contours (H_{m0} vs T_e, PCA) from a 1-year record (2020). Method: principal component analysis of (H_{m0}, T_e), an inverse Gaussian fit in the rotated frame, and the contour read off a circle of iso-probability by the inverse first-order reliability method (I-FORM); computed with `mhkit.wave.contours.environmental_contours(Hm0, Te, dt, return_period, method='PCA')`. Method Source: Eckert-Gallup, Aubrey C., Cédric J. Sallaberry, Ann R. Dallman, and Vincent S. Neary. 2016. "Application of Principal Component Analysis (PCA) and Improved Joint Probability Distributions to the Inverse First-Order Reliability Method (I-FORM) for Predicting Extreme Sea States." Ocean Engineering 112: 307–19. https://doi.org/10.1016/j.oceaneng.2015.12.018. Coe, Ryan, Carlos Michelen, Aubrey Eckert-Gallup, Yi-Hsiang Yu, and Jennifer Van Rij. 2016. WEC Design Response Toolbox v. 1.0. Computer software. https://www.osti.gov//servlets/purl/1312743. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
# Peak of each contour: the largest H_m0 and the T_e it occurs at.
for label, cop in (("50-year", copulas50), ("100-year", copulas100)):
    i = cop["PCA_x1"].argmax()
    print(f"{label}: H_m0 max {cop['PCA_x1'].max():.1f} m at T_e {cop['PCA_x2'][i]:.1f} s")
```

```
50-year: H_m0 max 11.3 m at T_e 16.6 s
100-year: H_m0 max 11.9 m at T_e 16.8 s
```

10 Exceedance Probability

Exceedance probability is how often the point exceeds a given sea state, the survivability view of the same record. Figure 34 is the empirical survival function $P(H_{m0} > x)$, and Figure 35 the same construction for the energy flux, $P(J > x)$. The two answer different questions: the H_{m0} tail is what the structure must survive, the J tail is where the resource actually sits, and J's is the longer one because flux goes as $H_{m0}^2 T_e$.

```
def exceedance_plot(q, name, fmt="{:.1f}"):
    """Empirical survival function of `q`: P(q > x) against x, log y.

    The dashed markers are the 99th percentile and the record maximum. Axis text
    is derived from QOIS[q], so the symbol and unit follow the variable.
    """
    values = np.sort(data[q].values)
    exceedance = 1.0 - np.arange(1, values.size + 1) / (values.size + 1)
    symbol = QOIS[q].split("[")[0].strip().strip("$")  # "$H_{m0}$ [m]" -> "H_{m0}"
    unit = QOIS[q].split("[")[-1].rstrip("]").strip()  # -> "m"

    fig, ax = plt.subplots(figsize=(7, 3))
    ax.semilogy(values, exceedance, color=qcolor(q))
    for pct in (0.99, 1.0):
        val = np.quantile(data[q], pct) if pct < 1.0 else data[q].max()
        ax.axvline(val, ls="--", color="0.5")
        ax.text(
            val, 1e-3,
            f"{'max' if pct == 1.0 else '99%'}\n{fmt.format(val)} {unit}",
            ha="center",
        )
    ax.set(xlabel=QOIS[q], ylabel=f"$P({symbol} > x)$")
    ax.grid(True, which="both", alpha=0.3)
    savefig(name)
```

```
exceedance_plot("Hm0", "exceedance_hm0")
```

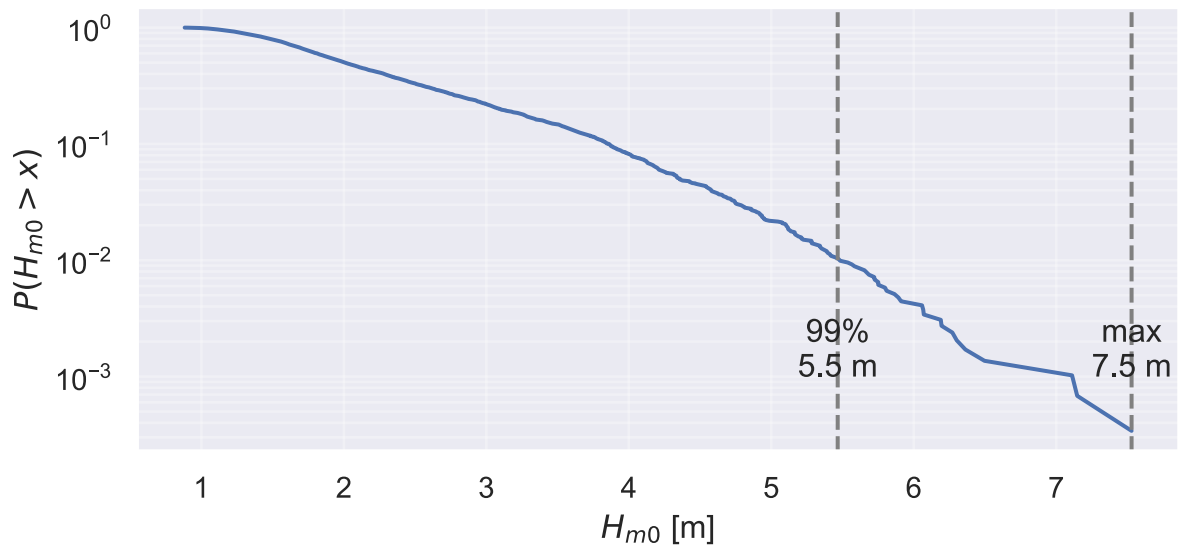

Figure 34: PacWave South (44.5670, −124.2290), exceedance probability (2020) of significant wave height (H_{m0}). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
exceedance_plot("J", "exceedance_j", fmt="{:.0f}")
```

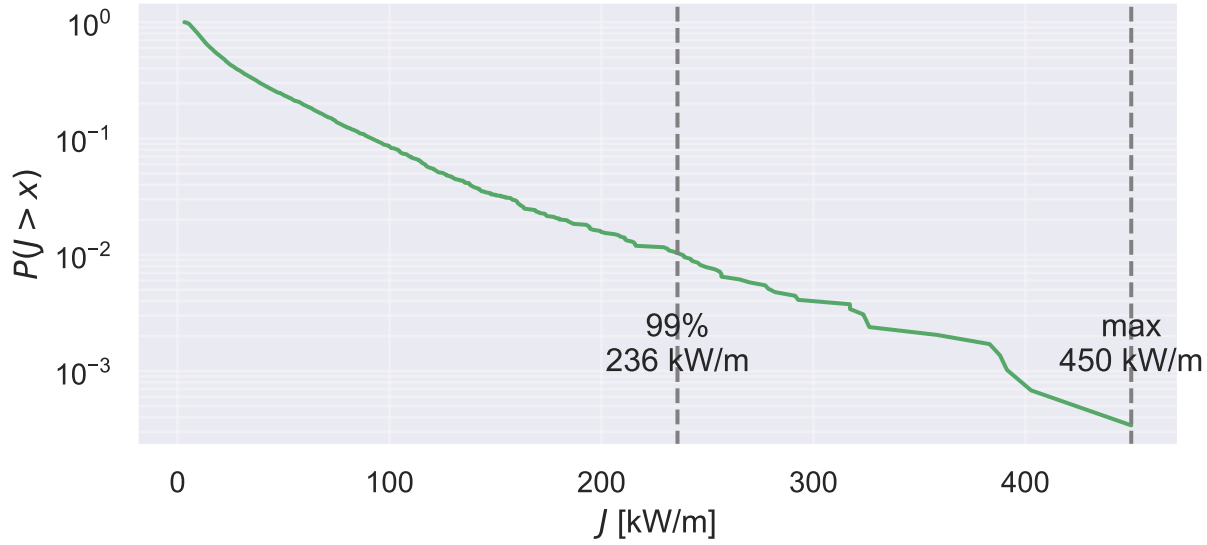

Figure 35: PacWave South (44.5670, −124.2290), exceedance probability (2020) of omnidirectional wave energy flux (J). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

11 Directionality

A wave rose (Figure 36) shows how often each direction sector occurs, stacked by H_{m0}; the polar joint probability panels (Figure 37, Figure 38, Figure 39) resolve the same directions against each variable; and Figure 40 visualizes the predominant direction of mean power [kW/m].

> **i** Direction conventions
>
> Directions come back in the meteorological convention (degrees clockwise from true north, the direction waves arrive *from*) for every domain.

```
Dir_bins = np.arange(0, 360 + 15, 15)
```

```
rose_bands = [0, 1, 2, 3, 4, np.inf]  # Hm0 [m]
band_labels = ["0-1 m", "1-2 m", "2-3 m", "3-4 m", ">4 m"]

rose_sector = pd.cut(data.Dir, Dir_bins, right=False)
band = pd.cut(data.Hm0, rose_bands, labels=band_labels, right=False)
# reindex both ways: crosstab drops empty sectors and empty bands, but the
# stacked bars need all 24 directions and every band.
rose = (
    pd.crosstab(rose_sector, band)
    .reindex(index=rose_sector.cat.categories, columns=band_labels, fill_value=0)
    / len(data) * 100  # % of year
)

theta = np.deg2rad(Dir_bins[:-1] + 7.5)
band_colors = sns.color_palette("viridis", n_colors=len(band_labels))

fig, ax = plt.subplots(figsize=(7, 7), subplot_kw={"projection": "polar"})
bottom = np.zeros(len(theta))
for label, color in zip(band_labels, band_colors):
    values = rose[label].to_numpy()
    ax.bar(theta, values, width=np.deg2rad(15), bottom=bottom,
           color=color, edgecolor="white", linewidth=0.4, label=label)
    bottom += values
ax.set_theta_zero_location("N")
ax.set_theta_direction(-1)
ax.legend(title="$H_{m0}$", loc="upper left", bbox_to_anchor=(1.08, 1.0))
fig.tight_layout()
savefig("directionality_wave_rose")
```

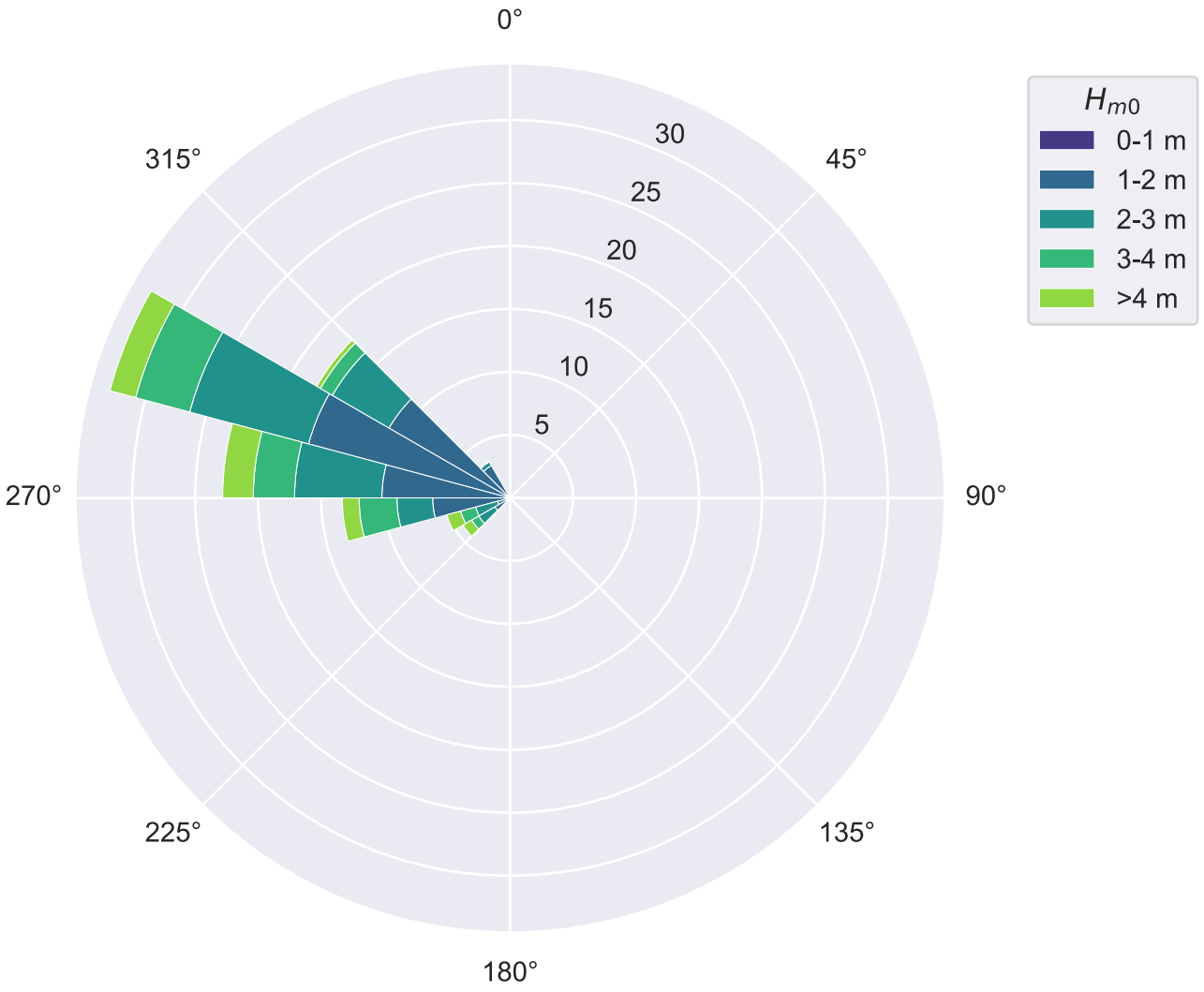

Figure 36: PacWave South (44.5670, −124.2290), wave rose: percent of the year (2020) by direction, stacked by significant wave height (H_{m0}) band. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
def polar_jpd(radius, r_bins, r_label, name):
    """Polar joint probability of direction against `radius`, % of year.

    Zero-probability cells are blanked rather than drawn at the bottom of the
    colour scale: a sea state that never occurred is not the same as one that
    occurred rarely, and filling them paints the whole disc the lowest colour so
    the populated sector reads against solid purple instead of the background.

    The rings carry `r_label`'s unit. The radius is a different quantity in each
    of these figures, so a bare number on the rings does not say what it measures.
```

```
    """
    counts, _, _ = np.histogram2d(data.Dir, radius, bins=[Dir_bins, r_bins])
    counts = counts / counts.sum() * 100  # % of year per (direction, radius) bin
    counts = np.where(counts == 0, np.nan, counts)  # NaN draws transparent
    th, rr = np.meshgrid(np.deg2rad(Dir_bins), r_bins, indexing="ij")
    fig, ax = plt.subplots(figsize=(7, 6), subplot_kw={"projection": "polar"})
    pcm = ax.pcolormesh(th, rr, counts, shading="auto", cmap="viridis")
    ax.set_theta_zero_location("N")
    ax.set_theta_direction(-1)
    # Name the angular axis on the 0 deg label, the same gap the rings had: the
    # compass numbers alone never say the angle is a wave direction. Derived from
    # QOIS["Dir"] ("Mean Wave\nDirection [deg]") so the name has one source.
    dir_name = QOIS["Dir"].replace("\n", " ").split("[")[0].strip()
    theta_ticks = np.arange(0, 360, 45)
    theta_labels = [f"{t:g}°" for t in theta_ticks]
    theta_labels[0] = f"0°, {dir_name}"
    ax.set_thetagrids(theta_ticks, theta_labels)
    # Ring labels moved off the data (the record sits west) and given the unit
    # carried in r_label, e.g. "$H_{m0}$ [m]" -> "2 m". 67.5 deg sits between two
    # compass labels, so the outermost ring label does not collide with one; the
    # full set of rings is thinned to four, because ring labels are collinear and
    # eight of them run together into an unreadable line.
    ax.set_rlabel_position(67.5)
    unit = r_label.split("[")[-1].rstrip("]").strip() if "[" in r_label else ""
    ticks = [t for t in ax.get_yticks() if 0 < t <= ax.get_rmax()]
    ticks = ticks[:: max(1, len(ticks) // 4)]
    ax.set_yticks(ticks)
    ax.set_yticklabels([f"{t:g} {unit}".strip() for t in ticks], fontsize=9)
    plt.colorbar(pcm, ax=ax, label="% of year", pad=0.1)
    fig.tight_layout()
    savefig(name)
```

```
polar_jpd(data.Hm0, Hm0_bins, QOIS["Hm0"], "polar_jpd_hm0")
```

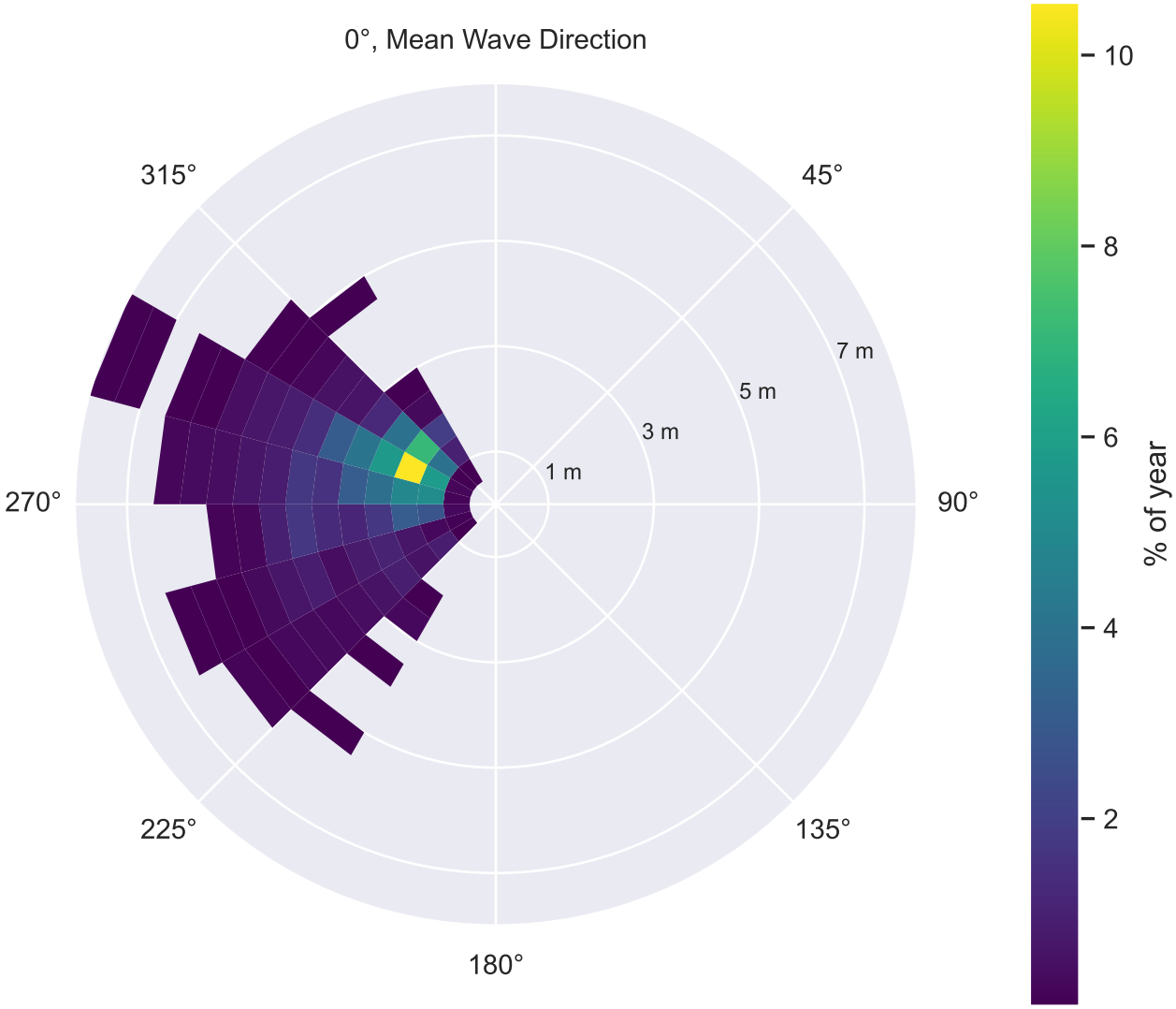

Figure 37: PacWave South (44.5670, −124.2290), joint probability (2020) of direction and significant wave height (H_{m0}). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
polar_jpd(data.Te, Te_bins, QOIS["Te"], "polar_jpd_te")
```

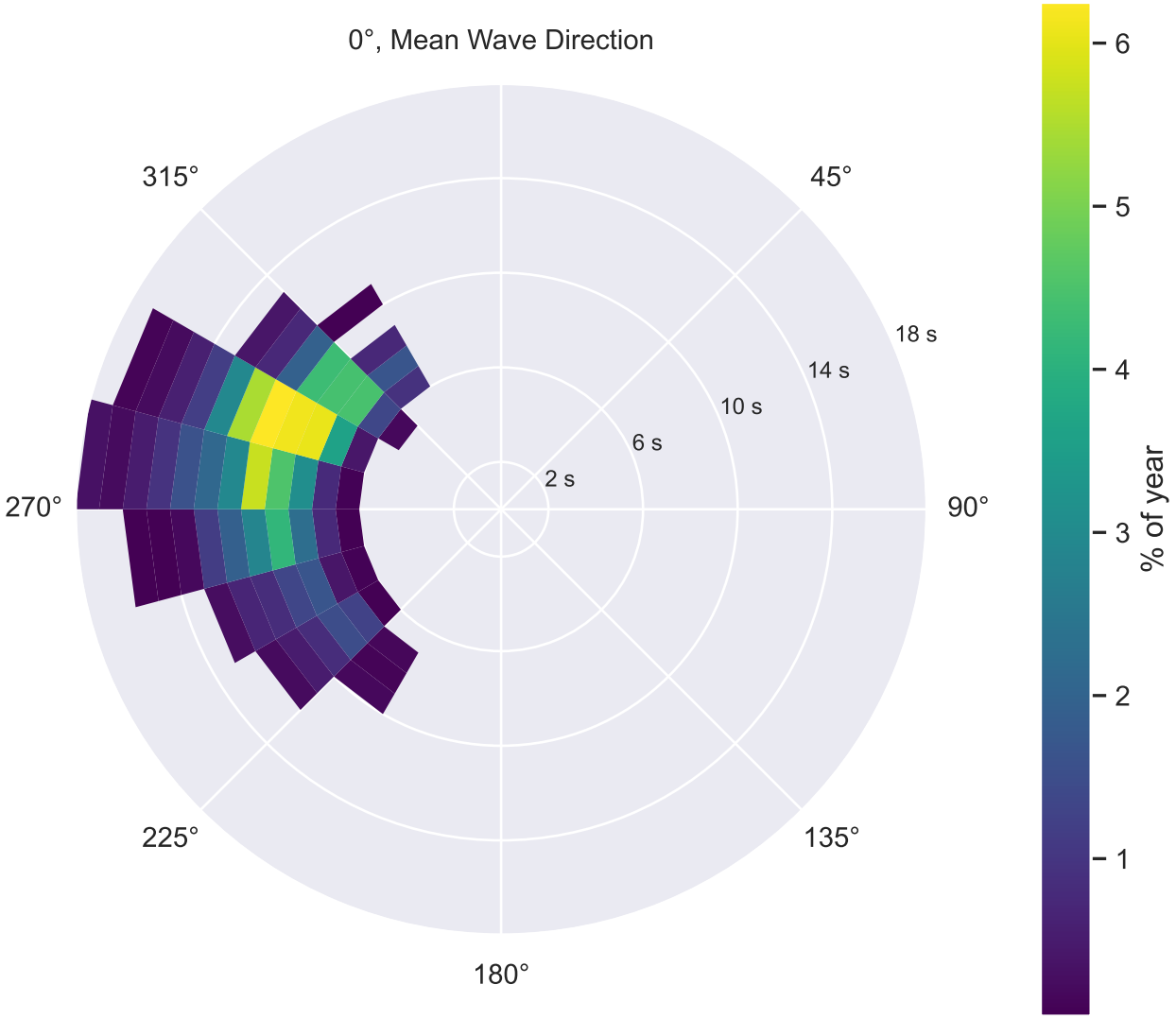

Figure 38: PacWave South (44.5670, −124.2290), joint probability (2020) of direction and energy period (T_e). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
polar_jpd(data.J, np.linspace(0, data.J.max(), len(Hm0_bins)), QOIS["J"], "polar_jpd_j")
```

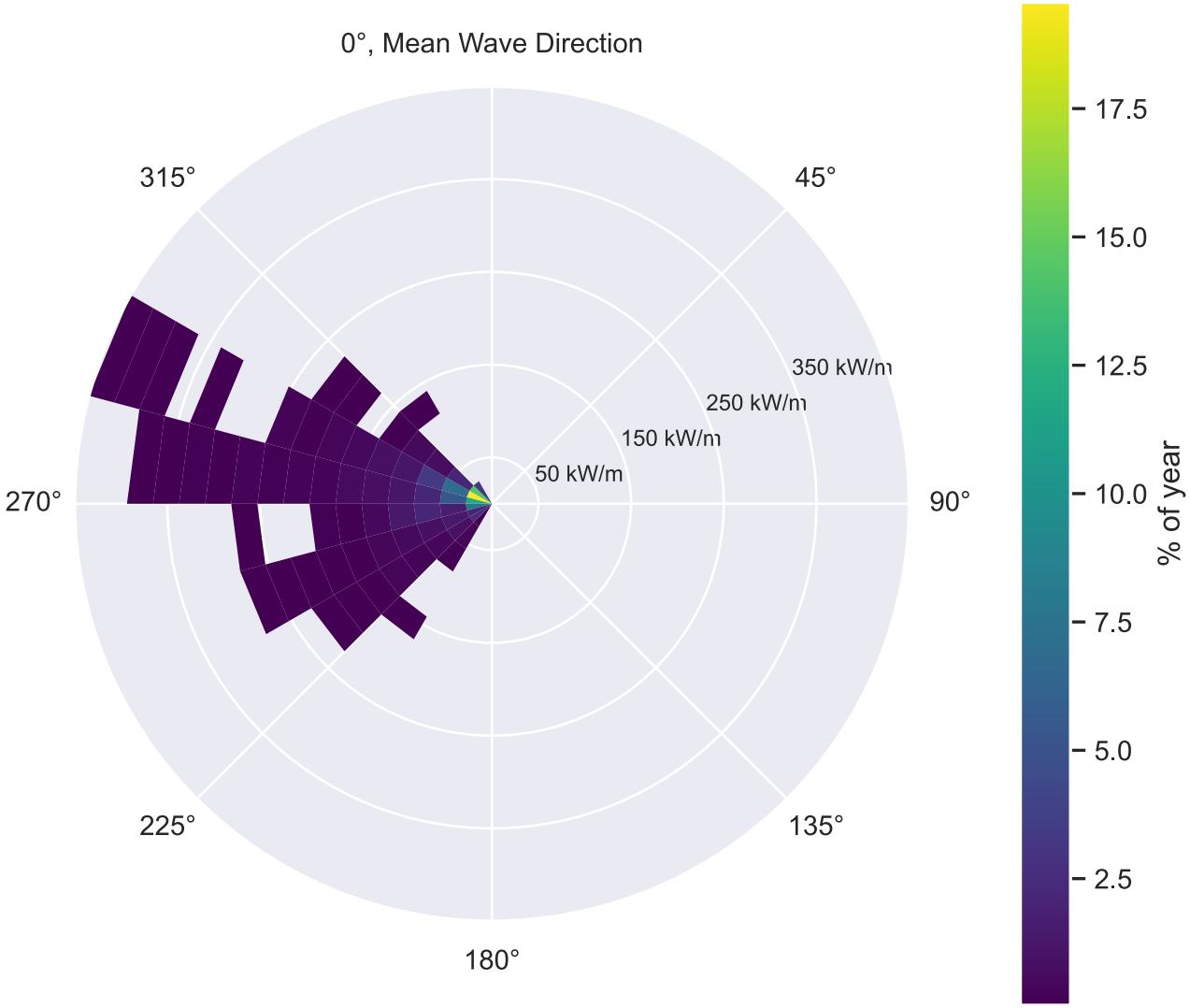

Figure 39: PacWave South (44.5670, −124.2290), joint probability (2020) of direction and omnidirectional wave energy flux (J). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
mean_sector = pd.cut(data.Dir, Dir_bins)
mean_J = data.J.groupby(mean_sector, observed=False).mean()
centers = np.deg2rad(Dir_bins[:-1] + 7.5)

fig, ax = plt.subplots(figsize=(6, 6), subplot_kw={"projection": "polar"})
ax.bar(centers, mean_J.fillna(0).values, width=np.deg2rad(15), alpha=0.7)
ax.set_theta_zero_location("N")
ax.set_theta_direction(-1)
savefig("directionality_mean_power")
```

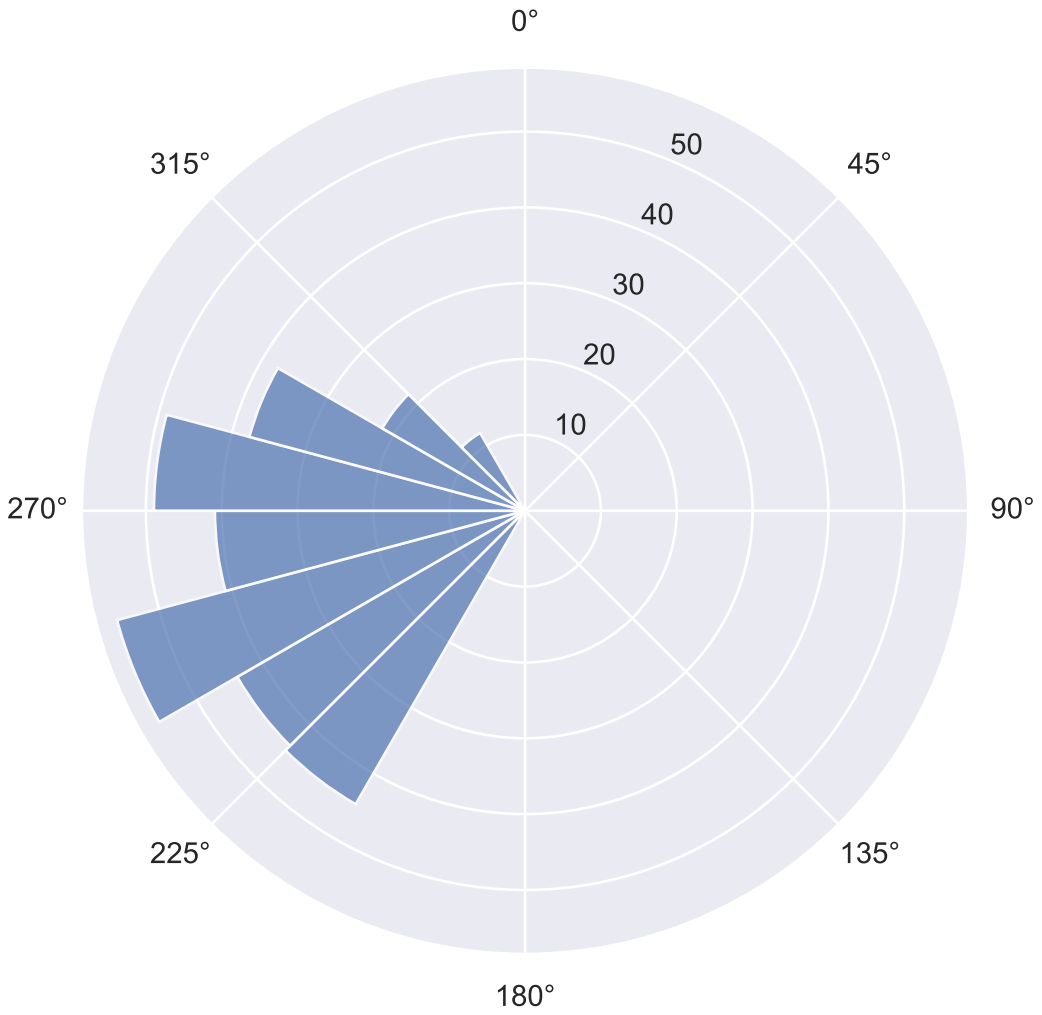

Figure 40: PacWave South (44.5670, −124.2290), mean omnidirectional wave energy flux (J) by direction sector (2020). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

12 Mean Annual Energy Production (MAEP)

Mean annual energy production (MAEP) is an estimate of electrical energy a device would generates at this point over an average year. It needs two matrices on the same sea-state bins: the resource side, which Section 8′s joint probability matrix already provides, and the device side, a power matrix from a modeled or measured Wave Energy Converter (WEC). In this example we are using `Wave Energy Converters.csv`, from the System Advisor Model (SAM) that provides power matrices from the wave energy reference models. Devices included here are RM3, a Wave Point Absorber, RM5 an Oscillating Surge Flap and RM6 and Oscillating Water Column.

12.1 Reading SAM Wave Reference Model Power Matricies

The library encodes each power matrix as a run of bracketed, semicolon-separated groups in one CSV cell. The first group is the T_e axis – a `0.0` placeholder for the corner, then one bin center per

column. Every later group is one row: its H_{m0} bin center, then one mean power value in kW per T_e column.

Both axes are bin centers, on 0.5 m and 1 s steps. Power matricies typically span a wider range of sea states (up to H_{m0} 9.75 m, T_e 20.5 s) and power matrix bins are not used where the matrices are not “aligned”.

```
def find_asset(filename):
    """The file next to the notebook, or wherever Kaggle mounted it."""
    local = Path(filename)
    if local.exists():
        return local
    root = Path("/kaggle/input")
    hit = next(root.rglob(filename), None) if root.exists() else None
    if hit is None:
        raise FileNotFoundError(f"{filename}: not found locally or under {root}")
    return hit


def parse_wec_power_matrix(cell):
    """One library cell -> a power matrix [kW] indexed by (Hm0, Te) bin centers."""
    groups = re.findall(r"\[([^\]]*)\]", cell)
    te_centers = [float(v) for v in groups[0].split(";")][1:]  # drop the corner
    hm0_centers, rows = [], []
    for group in groups[1:]:
        values = [float(v) for v in group.split(";")]
        hm0_centers.append(values[0])
        rows.append(values[1:])
    return pd.DataFrame(rows, index=hm0_centers, columns=te_centers)


# Rows 1 and 2 of the file are a units row and a variable-name row, not data.
wec_table = pd.read_csv(find_asset("Wave Energy Converters.csv"), skiprows=[1, 2])
POWER_MATRICES = {
    row["Name"]: parse_wec_power_matrix(row["Power"])
    for _, row in wec_table.iterrows()
}

for _name, _matrix in POWER_MATRICES.items():
    print(
        f"{_name}: {_matrix.shape[0]} Hm0 bins x {_matrix.shape[1]} Te bins, "
        f"rated {_matrix.to_numpy().max():.1f} kW"
    )
```

```
RM3: 20 Hm0 bins x 21 Te bins, rated 286.0 kW
RM5: 20 Hm0 bins x 21 Te bins, rated 360.0 kW
RM6: 20 Hm0 bins x 21 Te bins, rated 350.5 kW
```

12.2 Reference Model Mean Power Matricies

Each matrix is the device's mean electrical power per sea state (Figure 41 through Figure 43). Bins the device cannot operate in are zero and are left blank, so the shape of each plot is the device's operating envelope: where it produces, and where it saturates at rated power.

```
plot_wave_matrix(
    POWER_MATRICES["RM3"], zlabel="Device Power [kW]", annotate=False
)
savefig("power_matrix_rm3")
```

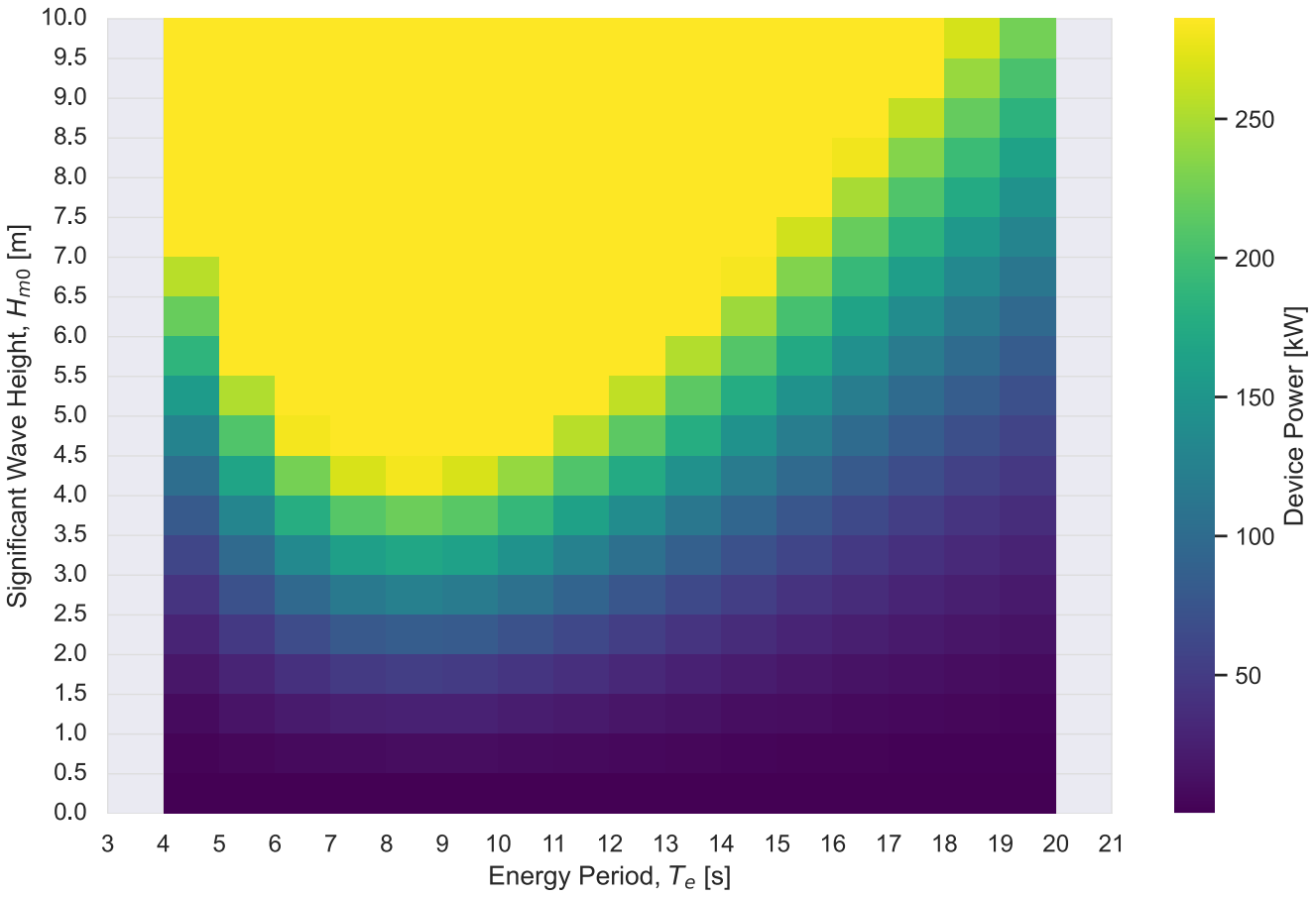

Figure 41: RM3 (heaving buoy, hydraulic drivetrain), modelled device power over (H_{m0}, T_e) sea states. Data Source: modelled reference-device power matrices, Wave Energy Converters.csv.

```
plot_wave_matrix(
    POWER_MATRICES["RM5"], zlabel="Device Power [kW]", annotate=False
```

```
)
savefig("power_matrix_rm5")
```

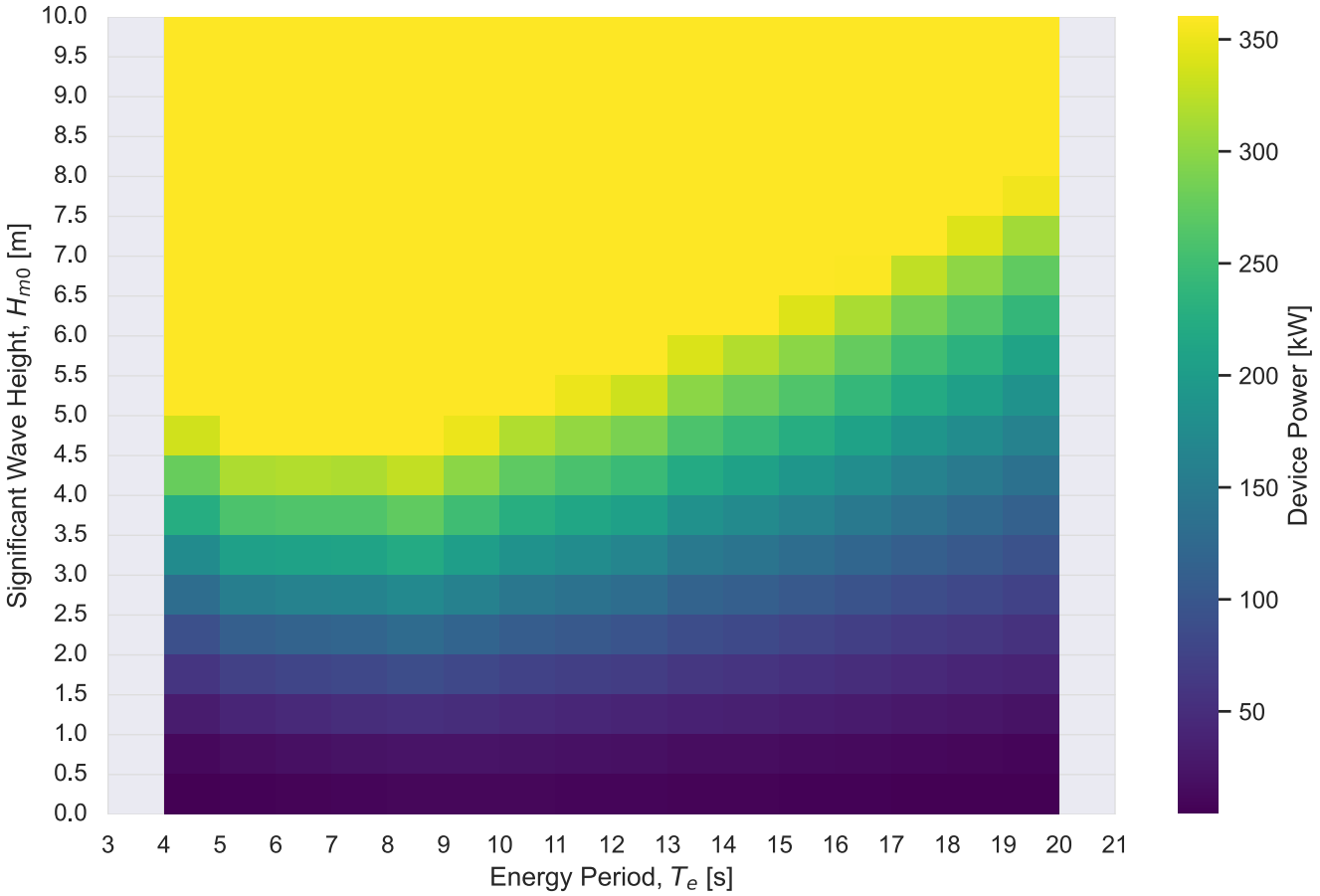

Figure 42: RM5 (oscillating surge wave converter, hydraulic drivetrain), modelled device power over (H_{m0}, T_e) sea states. Data Source: modelled reference-device power matrices, Wave Energy Converters.csv.

```
plot_wave_matrix(
    POWER_MATRICES["RM6"], zlabel="Device Power [kW]", annotate=False
)
savefig("power_matrix_rm6")
```

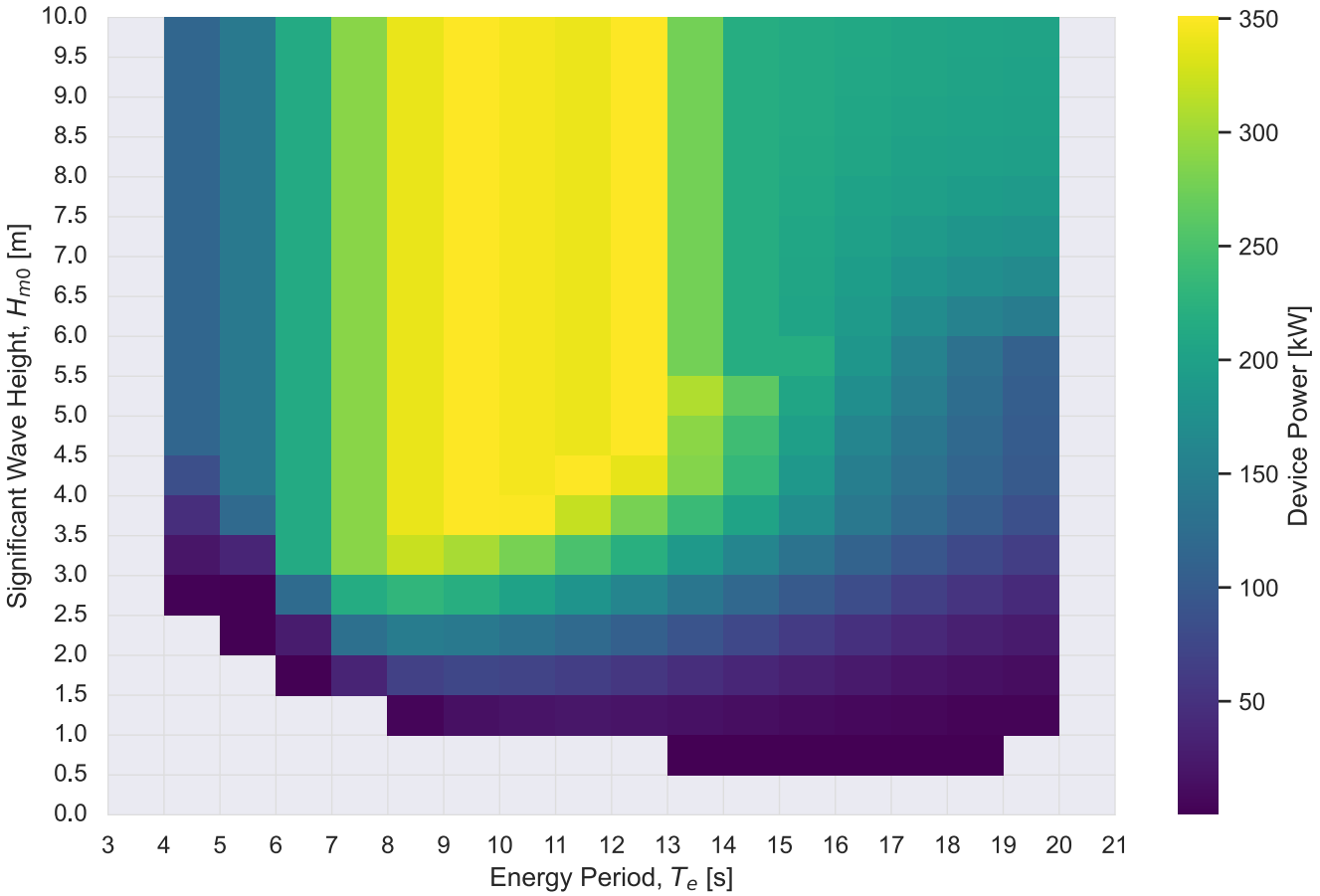

Figure 43: RM6 (backward bent duct buoy floating oscillating water column, Wells air turbine), modelled device power over (H_{m0}, T_e) sea states. Data Source: modelled reference-device power matrices, Wave Energy Converters.csv.

12.3 Mean Annual Energy Production (MAEP)

Each bin contributes (fraction of the year) x (power in that bin); summed and scaled to a year, that is MAEP. mhkit factors device performance as capture width x wave energy flux, so `performance.mean_annual_energy_production_matrix(CWM, JM, frequency)` multiplies three matrices and scales by 8766 h, the average length of a year. The library publishes the *product* of capture width and flux directly as a power matrix, so the flux term is unity here and the power matrix carries the whole kW value.

Note 8766 h is an average year, not this record: 2020 is a leap year and the record covers 8784 h. MAEP is deliberately the average-year figure, so the two differ by 0.2%.

12.3.a JPD Multiplied by Power Matrix

Figure 44 through Figure 46 show the intermediate: device power multiplied by the probability of the bin it sits in. This is the matrix mhkit sums internally, and its cells are deliberately odd quantities. Where RM3 makes 48.3 kW in a sea state that occupies 6.8% of the year, the cell reads 3.28 kW – not a power the device ever produces, but that bin's share of the annual average. Summed, the cells give the device's mean power; times 8766 h that is MAEP.

The greyed cells are where the two matrices do not overlap: either the point never produced that sea state, or the device makes nothing in it. A device only earns from bins that are populated *and* inside its operating envelope, and the grey is everything that falls outside that intersection.

```
def maep_contribution(power, frequency):
    """Per-bin contribution to mean power [kW]: device power x bin probability."""
    aligned = power.reindex(
        index=frequency.index, columns=frequency.columns
    ).fillna(0.0)
    return aligned * frequency
```

```
plot_wave_matrix(
    maep_contribution(POWER_MATRICES["RM3"], frequency),
    zlabel="Contribution to Mean Power [kW]",
    fmt=".2f", suffix="", blank_color="0.85",
)
savefig("maep_contribution_rm3")
```

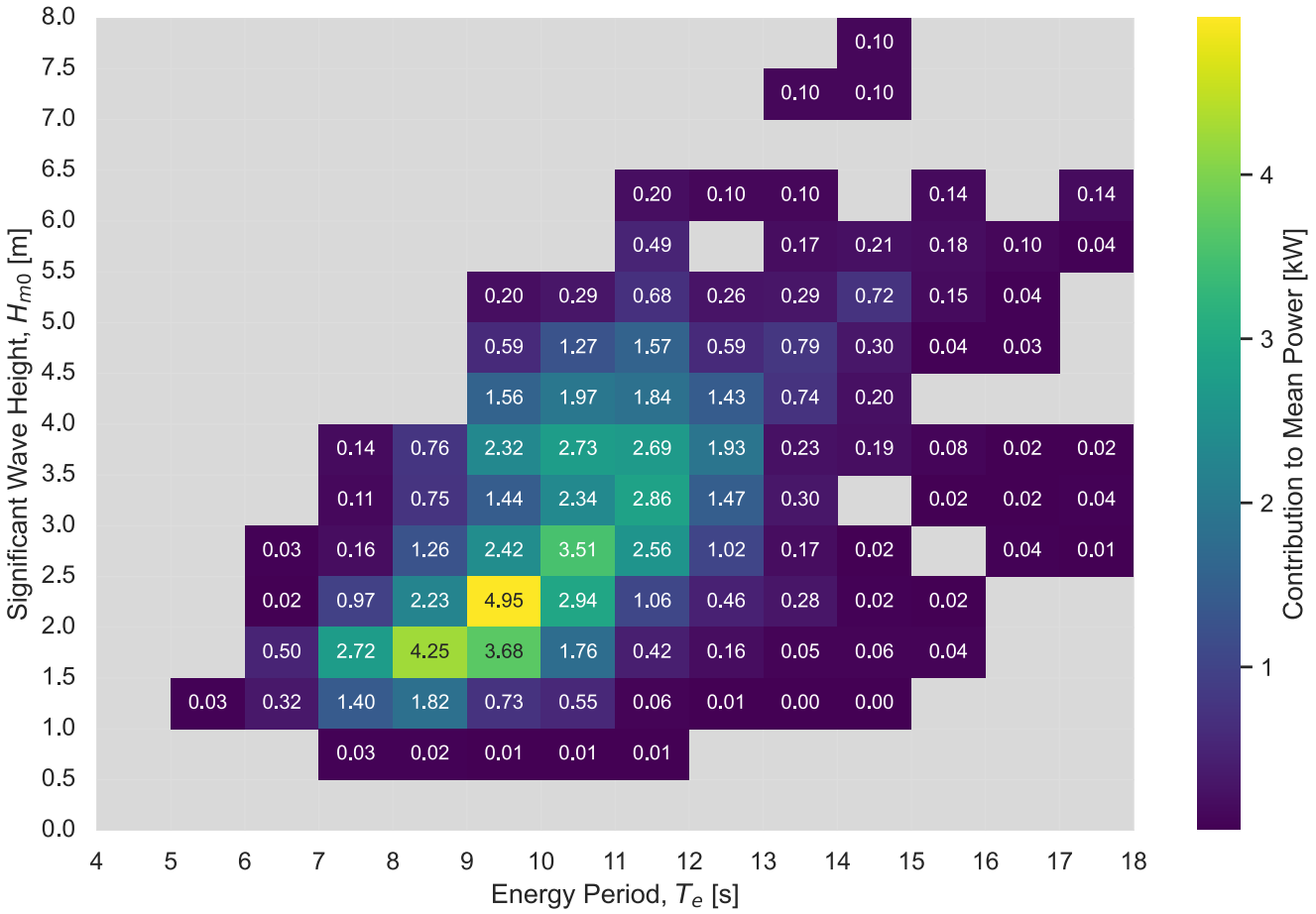

Figure 44: RM3, per-bin contribution to mean power: modelled device power multiplied by the 2020 joint probability of each (H_{m0}, T_e) sea state at PacWave South (44.5670, −124.2290). Cells sum to the device mean power; times 8766 h that is MAEP. Grey cells are sea states the point never produced or the device cannot use. Device Source: modelled reference-device power matrices, Wave Energy Converters.csv. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
plot_wave_matrix(
    maep_contribution(POWER_MATRICES["RM5"], frequency),
    zlabel="Contribution to Mean Power [kW]",
    fmt=".2f", suffix="", blank_color="0.85",
)
savefig("maep_contribution_rm5")
```

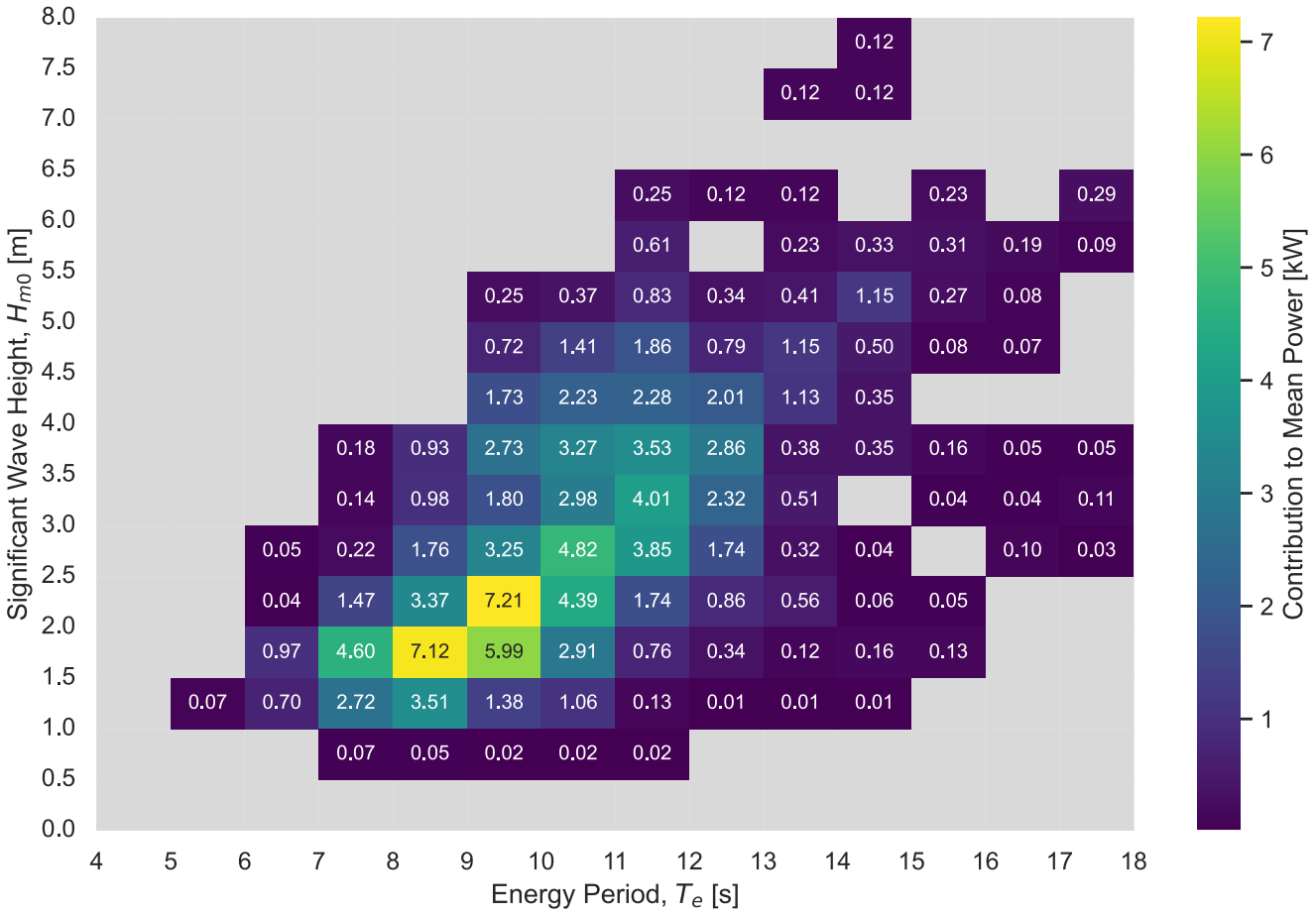

Figure 45: RM5, per-bin contribution to mean power: modelled device power multiplied by the 2020 joint probability of each (H_{m0}, T_e) sea state at PacWave South (44.5670, −124.2290). Cells sum to the device mean power; times 8766 h that is MAEP. Grey cells are sea states the point never produced or the device cannot use. Device Source: modelled reference-device power matrices, Wave Energy Converters.csv. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
plot_wave_matrix(
    maep_contribution(POWER_MATRICES["RM6"], frequency),
    zlabel="Contribution to Mean Power [kW]",
    fmt=".2f", suffix="", blank_color="0.85",
)
savefig("maep_contribution_rm6")
```

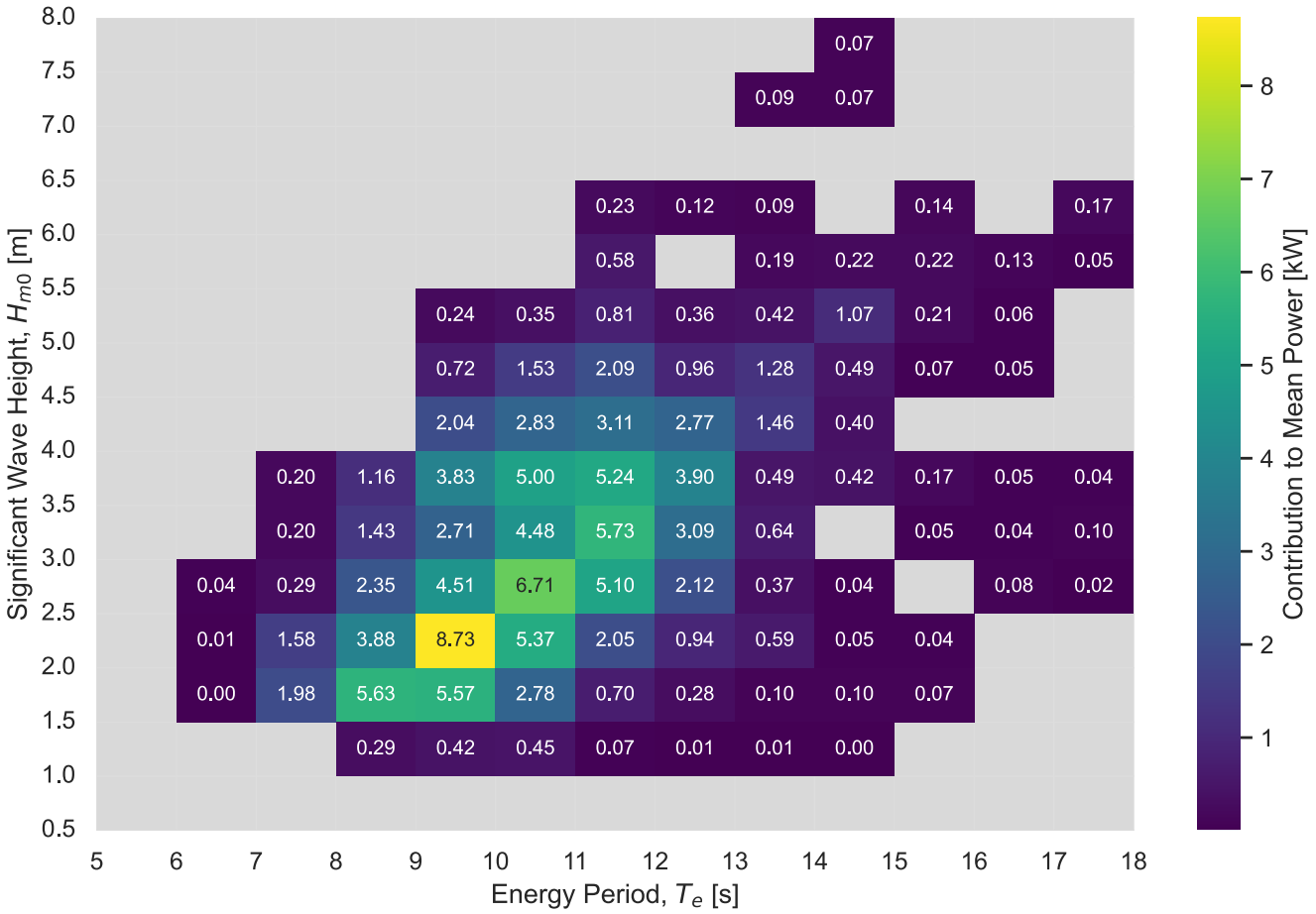

Figure 46: RM6, per-bin contribution to mean power: modelled device power multiplied by the 2020 joint probability of each (H_{m0}, T_e) sea state at PacWave South (44.5670, −124.2290). Cells sum to the device mean power; times 8766 h that is MAEP. Grey cells are sea states the point never produced or the device cannot use. Device Source: modelled reference-device power matrices, Wave Energy Converters.csv. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
def device_maep(power, frequency):
    """MAEP [kWh] for one device against one point's joint probability matrix.

    mhkit needs all three matrices on identical bins, so the device matrix is
    reindexed onto `frequency`'s. Both use the same bin-center convention, so
    this is a pure subset -- nothing is interpolated. `fillna(0)` is a guard for
    a device grid that does not reach a sea state the point produces; treating
    that as "produces nothing there" is the conservative reading.
    """
    aligned = power.reindex(
        index=frequency.index, columns=frequency.columns
```

```
    ).fillna(0.0)
    unit_flux = pd.DataFrame(
        1.0, index=frequency.index, columns=frequency.columns
    )
    return performance.mean_annual_energy_production_matrix(
        aligned, unit_flux, frequency
    )


_specs = wec_table.set_index("Name")
device_energy = pd.DataFrame(
    [
        {
            "Device": name,
            "Technology": _specs.loc[name, "Technology Type"],
            "Rated [kW]": matrix.to_numpy().max(),
            "MAEP [MWh/yr]": device_maep(matrix, frequency) / 1e3,
        }
        for name, matrix in POWER_MATRICES.items()
    ]
)
# Capacity factor against the matrix maximum, which is the rated power here.
device_energy["Capacity factor"] = device_energy["MAEP [MWh/yr]"] * 1e3 / (
    device_energy["Rated [kW]"] * 8766
)
show_table(device_energy.round({"Rated [kW]": 1, "MAEP [MWh/yr]": 1,
                                "Capacity factor": 3}))
```

Table 2: Mean annual energy production per modelled reference device at PacWave South, from the 2020 joint probability matrix. Device Source: modelled reference-device power matrices, Wave Energy Converters.csv. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

13 Multiple Points: Comparison

The single-point pipeline above characterizes one point; here we place several points side by side. `POINTS_TO_COMPARE` is set in the configuration section above. Every download is saved in the package cache, so re-runs skip S3 entirely. Every point keeps one color across every figure below.

Figure 47 visualizes the points being compared.

```
_ = render_points_map_outputs(
    {name: POINT_INFO[name] for name in POINTS_TO_COMPARE if name in POINT_INFO},
    make_points_map,
    output_dir=FIG_DIR,
```

```
    stem="compared_points_map",
    title="Compared points",
)
```

```
<folium.folium.Map at 0x1227d6010>
```

Figure 47: The points compared side by side below

```
point_data = {}
for name in POINTS_TO_COMPARE:
    try:
        point_data[name] = load_point(name)
    except Exception as e:  # e.g. a point outside its domain's grid
        print(f"skipping {name}: {type(e).__name__}: {e}")

# One color per point, shared across every figure below.
tab10 = plt.get_cmap("tab10")
point_colors = {name: tab10(i) for i, name in enumerate(point_data)}

{name: df.shape for name, df in point_data.items()}
```

```
{'US_Oregon_PacWave_South': (2928, 8),
 'US_California_San_Luis_Obispo': (2928, 8),
 'US_Hawaii_Oahu_WETS': (2928, 8),
 'US_Alaska_Kodiak': (2928, 8)}
```

13.1 Point Key Metrics

One row per point (Table 3), ranked by annual mean J. The annual energy column is the resource per meter of wave front, mean J integrated over the record. Figure 48 sets the mean against the 95th percentile on one linear axis; the gap between them separates what a device earns from (the mean) and what it must survive (the tail).

```
# Points ranked by annual mean J, richest first. Every figure below reuses this
# order so the ranking is the same wherever you look.
point_order = sorted(point_data, key=lambda n: point_data[n]["J"].mean(), reverse=True)

kpis = pd.DataFrame(
    {
        POINT_INFO[n]["label"]: {
            "mean Hm0 [m]": point_data[n].Hm0.mean(),
            "max Hm0 [m]": point_data[n].Hm0.max(),
            "mean Te [s]": point_data[n].Te.mean(),
            "mean J [kW/m]": point_data[n].J.mean(),
            "p95 J [kW/m]": point_data[n].J.quantile(0.95),
```

```
         "annual energy [MWh/m]": point_data[n].J.mean() * record_hours(point_data[n]) / 1000,
        }
        for n in point_order
    }
).T
show_table(kpis.round(2))
```

Table 3: Point Metrics for the loaded year (2020). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
annual_mean = pd.Series({n: point_data[n]["J"].mean() for n in point_order})
annual_p95 = pd.Series({n: point_data[n]["J"].quantile(0.95) for n in point_order})
rank = annual_mean.sort_values().index  # ascending: richest at the top of the axis

fig, ax = plt.subplots(figsize=(FIG_WIDTH, 4.5))
y = np.arange(len(rank))
ax.hlines(y, annual_mean[rank], annual_p95[rank], color="0.75", linewidth=2, zorder=1)
ax.scatter(annual_mean[rank], y, s=70, zorder=2,
           color=[point_colors[n] for n in rank], label="annual mean")
ax.scatter(annual_p95[rank], y, s=70, marker="D", facecolors="none", zorder=2,
           edgecolors=[point_colors[n] for n in rank], label="95th percentile")
for i, n in enumerate(rank):
    for value in (annual_mean[n], annual_p95[n]):
        ax.annotate(f"{value:.0f}", (value, i), xytext=(0, 8),
                    textcoords="offset points", ha="center", size=7, color="0.25")
ax.set_yticks(y)
ax.set_yticklabels([POINT_INFO[n]["label"] for n in rank])
ax.set_xlabel("$J$ [kW/m]")
ax.margins(x=0.08, y=0.09)
ax.legend(loc="lower right")
fig.tight_layout()
savefig("multipoint_mean_vs_p95")
```

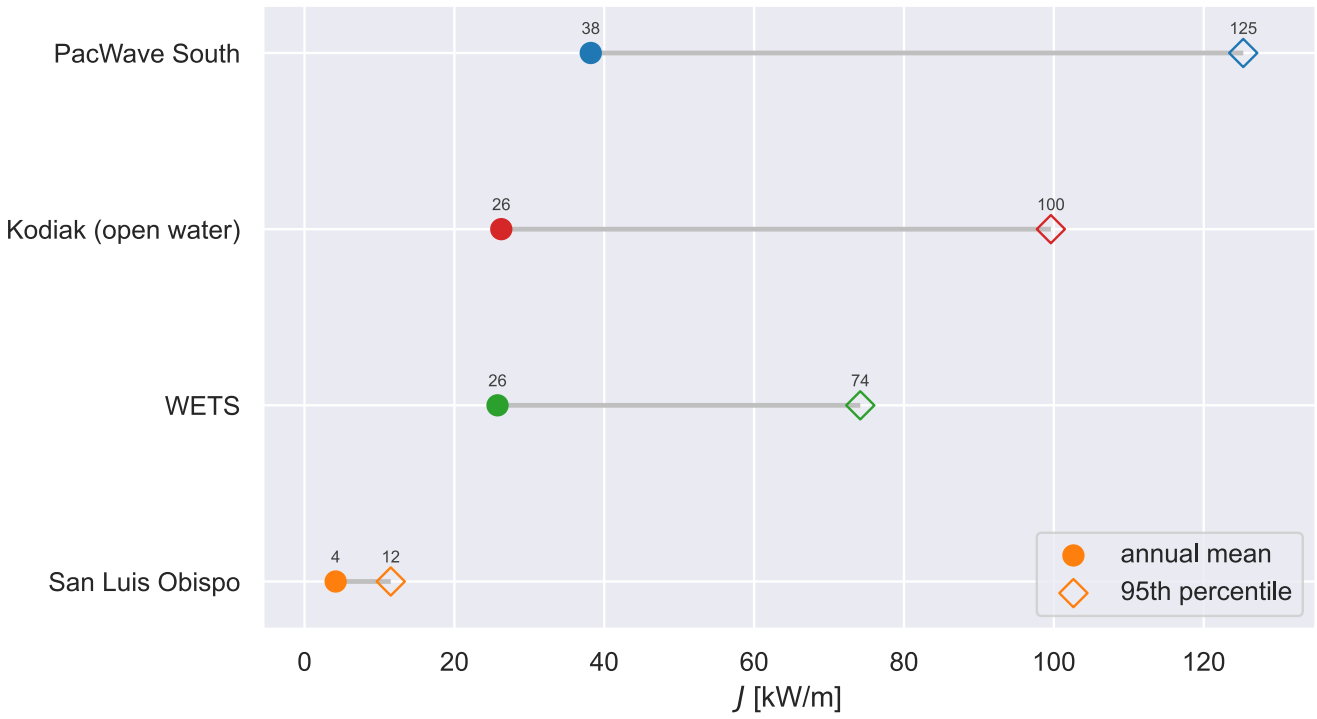

Figure 48: Annual mean against 95th percentile omnidirectional wave energy flux (J) per point (2020). Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

13.2 Moving Averages

A centered moving average smooths storm-scale noise and exposes each point's seasonal cycle which is helpful for comparison between sites. The 7-day window (Figure 49 through Figure 51) still tracks individual storms; the 30-day window (Figure 52 through Figure 54) leaves the seasonal signal. Seasonality follows the storm tracks a point is exposed to, not its latitude.

```
def multipoint_rolling(q, window, name):
    """`window` centered moving average of `q`, one line per point."""
    fig, ax = plt.subplots(figsize=(FIG_WIDTH, 4))
    for point, df in point_data.items():
        roll = df[q].rolling(window, center=True, min_periods=1).mean()
        ax.plot(roll.index, roll, color=point_colors[point],
                label=POINT_INFO[point]["label"], linewidth=1.5)
    ax.set(xlabel="Date", ylabel=QOIS[q])
    ax.margins(x=0)
    ax.legend(loc="upper left", bbox_to_anchor=(1.02, 1), borderaxespad=0)
    fig.autofmt_xdate()  # rotate/thin the date ticks; unrotated they overlap
    fig.tight_layout()
    savefig(name)
```

```
multipoint_rolling("Hm0", "7D", "multipoint_7d_hm0")
```

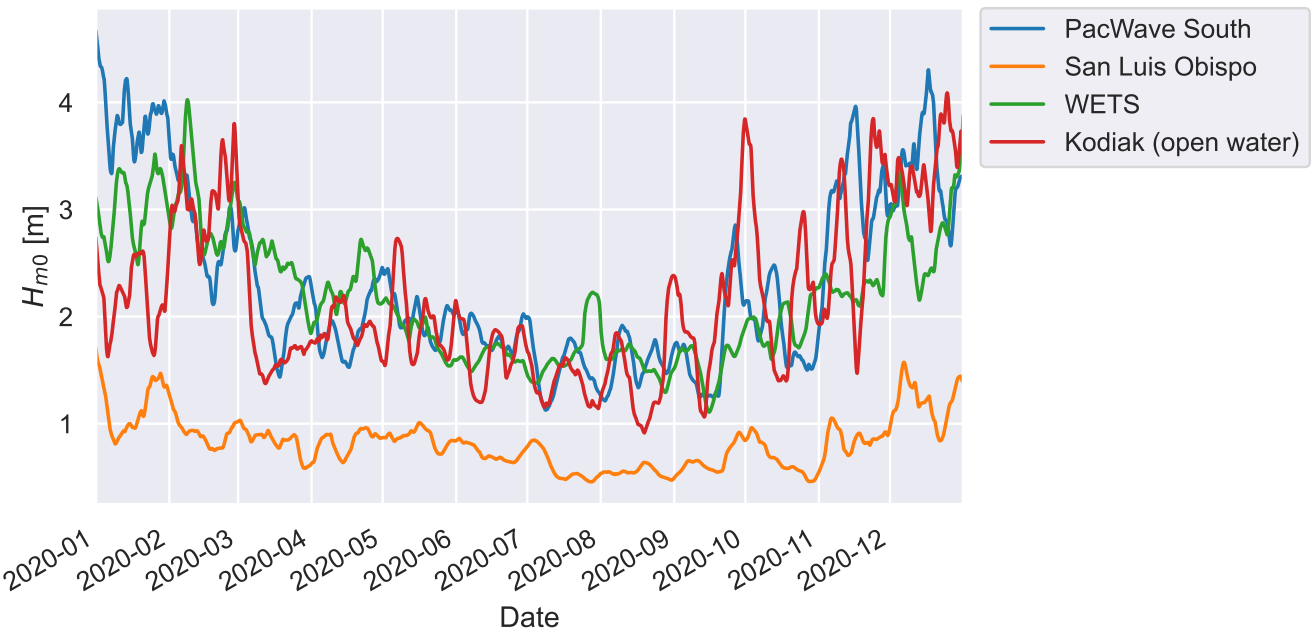

Figure 49: 7-day moving average (2020) of significant wave height (H_{m0}), all points. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
multipoint_rolling("Te", "7D", "multipoint_7d_te")
```

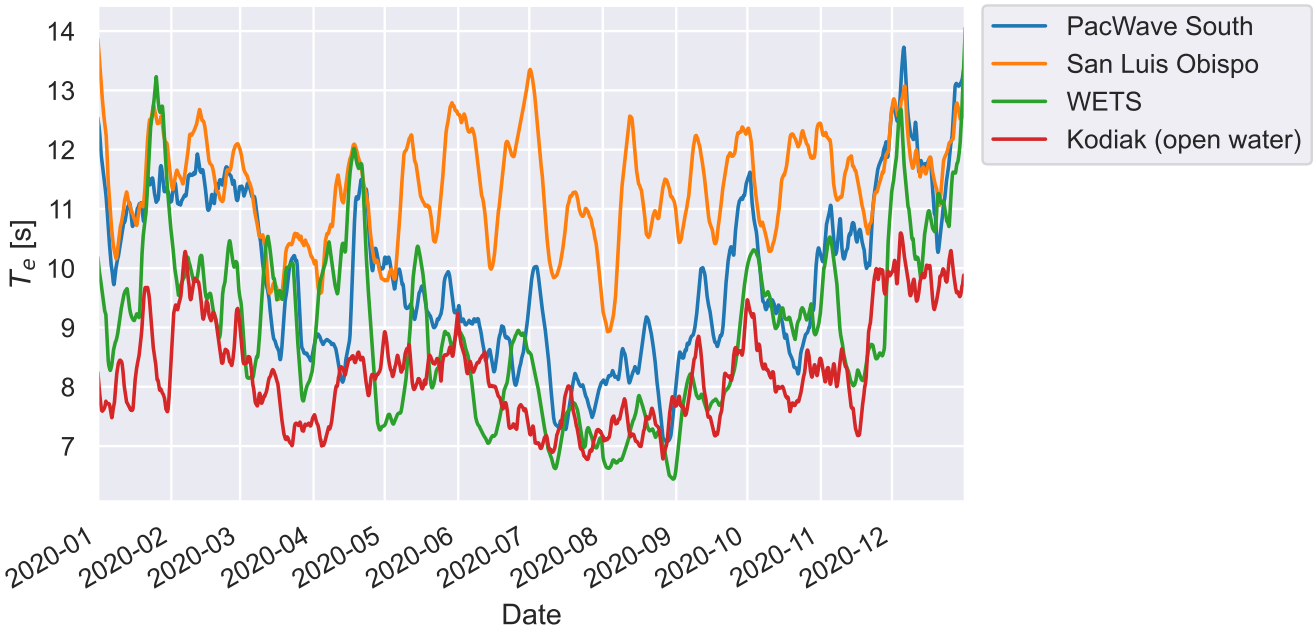

Figure 50: 7-day moving average (2020) of energy period (T_e), all points. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
multipoint_rolling("J", "7D", "multipoint_7d_j")
```

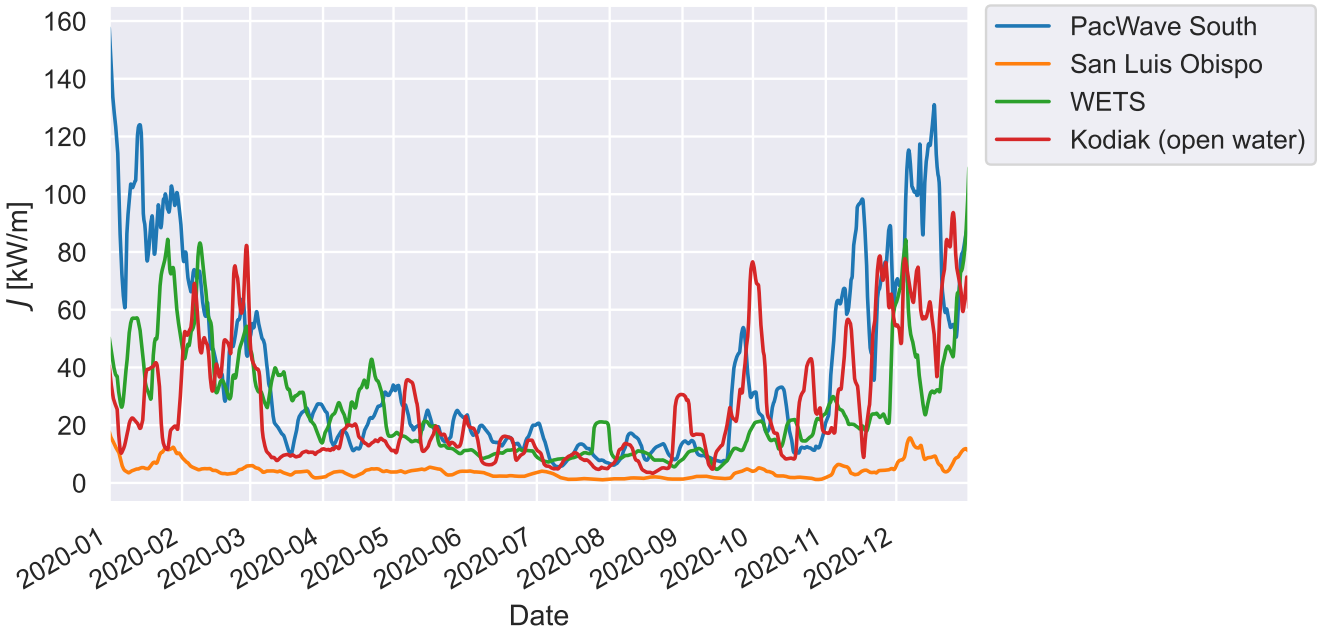

Figure 51: 7-day moving average (2020) of omnidirectional wave energy flux (J), all points. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
multipoint_rolling("Hm0", "30D", "multipoint_30d_hm0")
```

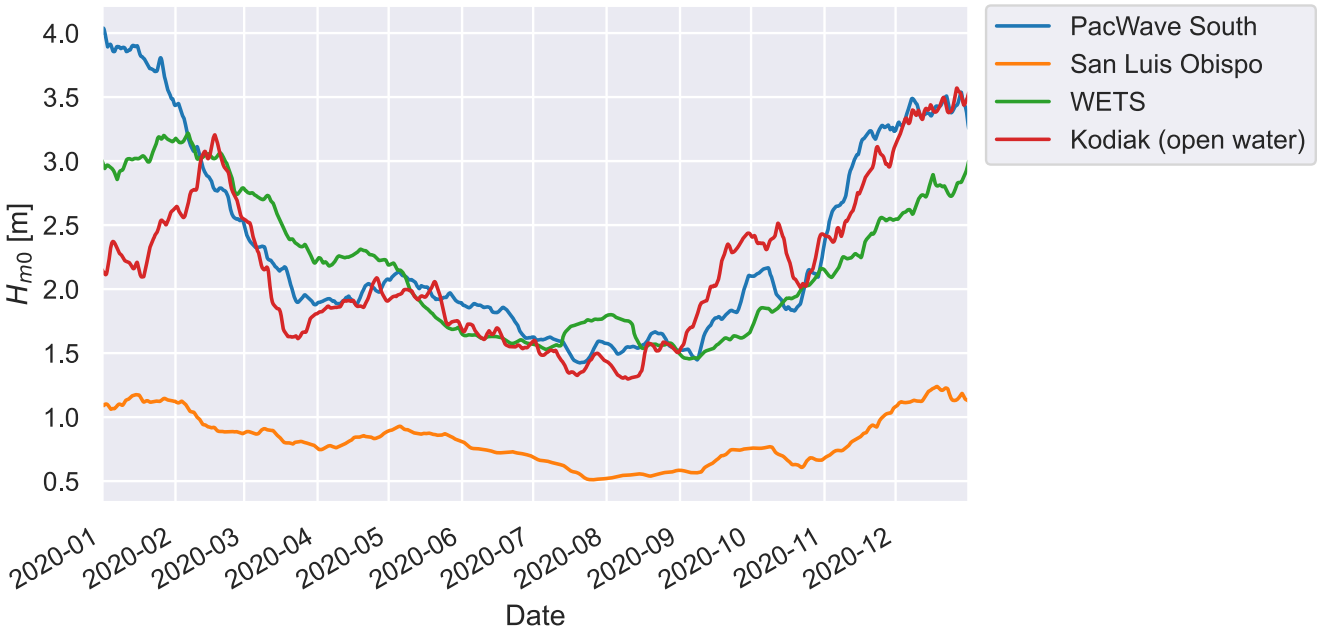

Figure 52: 30-day moving average (2020) of significant wave height (H_{m0}), all points. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
multipoint_rolling("Te", "30D", "multipoint_30d_te")
```

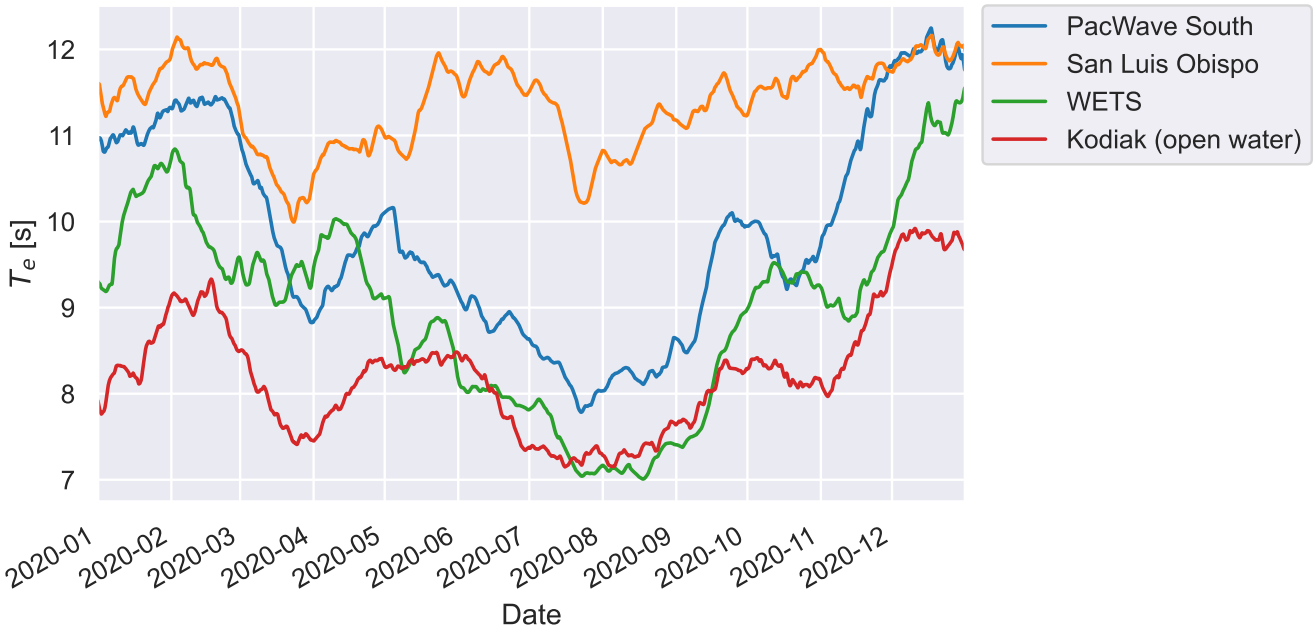

Figure 53: 30-day moving average (2020) of energy period (T_e), all points. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
multipoint_rolling("J", "30D", "multipoint_30d_j")
```

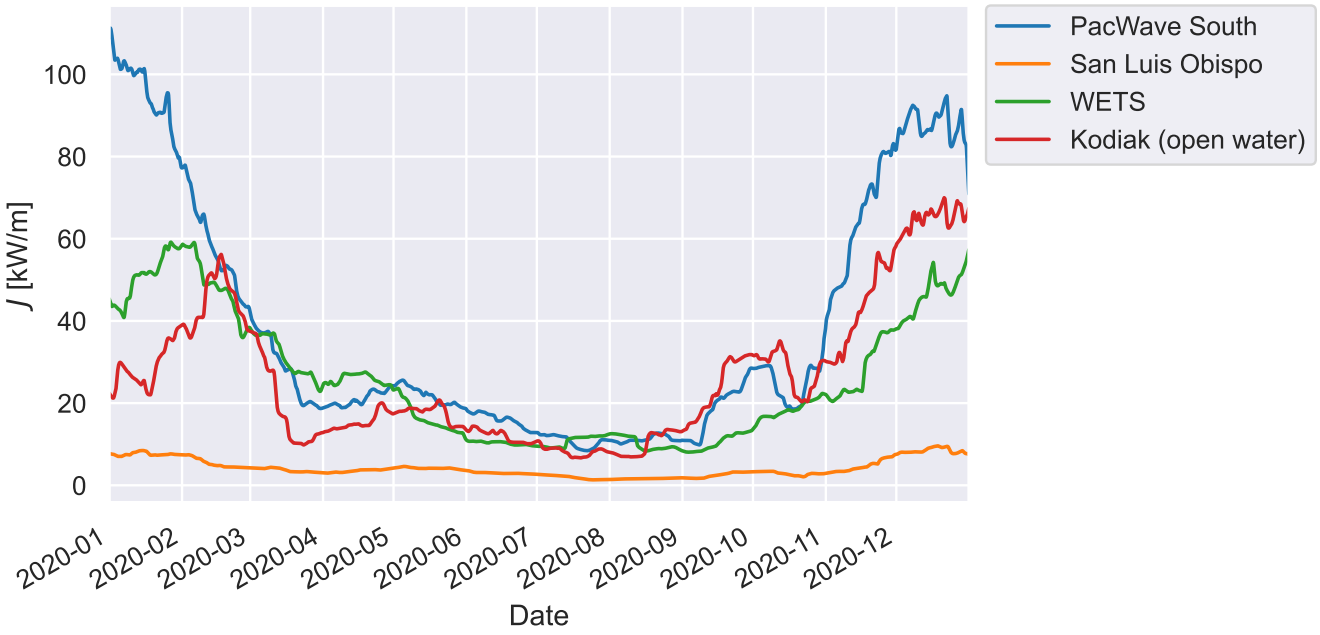

Figure 54: 30-day moving average (2020) of omnidirectional wave energy flux (J), all points. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

14 Environmental Contours: 1, 3 and 5 Year Records

A 100-year contour extrapolated from one year of data asks a few thousand 3-hour sea states to speak for a century. The size of that error is measurable. Refit the contour on more years and look at what moves. Here the 100-year PCA contour is refit per point on 1, 3 and 5 years of record.

`load_point` takes a list of years, so a longer record is one query: the package fetches the span in a single cached pull (five years times five variables still sits under the S3/API seam, so it stays on the free backend). Change `CONTOUR_YEARS` in the configuration section to widen or narrow the window.

```
# Nested windows, all ending at CONTOUR_YEARS[-1]: 1 -> [2020], 3 -> [2018..2020],
# 5 -> [2016..2020]. Each is a superset of the one before, so any change between
# them comes from added data and not from swapping which years are in play.
YEAR_WINDOWS = {n: CONTOUR_YEARS[-n:] for n in (1, 3, 5)}


def load_point_years(name, years=CONTOUR_YEARS):
    """One point's sea states over a window of years, sliced from the full record.

    The full CONTOUR_YEARS record is fetched (and cached) once per point; every
    window is a view of it, so widening the analysis costs no extra downloads.
    Only CONTOUR_VARIABLES are pulled -- the contour fits read Hm0 and Te and
    nothing else, so this stays a 2-variable request rather than an 8-variable one.
```

```
    """
    df = load_point(name, years=CONTOUR_YEARS, variables=CONTOUR_VARIABLES)
    return df[df.index.year.isin(years)]


# Fitting a contour is the expensive step in this example; each fit is a pure
# function of (point, years, method, return_period), so results are memoised to
# .cache/contours/. Set FORCE_CONTOURS = True to recompute everything.
FORCE_CONTOURS = False
CONTOUR_CACHE_DIR = CACHE_DIR / "contours"


# Per-point override of MHKiT's Hm0 bin width (`bin_val_size`, default 0.25).
# A point with a narrow, depth-limited Hm0 range leaves the upper bins nearly
# empty, and the fit through them goes singular (`LinAlgError`) -- a statement
# about the bin defaults, not the ocean. Narrower bins keep the point fittable.
# Points absent from this dict use MHKiT's default.
CONTOUR_BIN_SIZE = {
    "US_North_Carolina_Jenettes_Pier": 0.10,   # Atlantic; depth-limited at the pier
}


def contour(name, years, method="PCA", return_period=100, force=None):
    """100-year (default) contour for one point+record, memoised to disk.

    Returns a dict with 'x1' (Hm0) and 'x2' (Te), or both None if the fit does
    not converge.
    """
    bin_val_size = CONTOUR_BIN_SIZE.get(name)

    def fit():
        df = load_point_years(name, years=years)
        dt = (df.index[1] - df.index[0]).total_seconds()
        kwargs = {} if bin_val_size is None else {"bin_val_size": bin_val_size}
        try:
            cop = contours.environmental_contours(
                df.Hm0, df.Te, dt, return_period, method=method, **kwargs)
            return {"x2": np.asarray(cop[f"{method}_x2"]),
                    "x1": np.asarray(cop[f"{method}_x1"])}
        except (np.linalg.LinAlgError, ValueError):
            return {"x2": None, "x1": None}

    # The bin width goes in the cache key only when overridden, so existing
    # default-binned pickles stay valid rather than all missing at once.
    bin_key = "" if bin_val_size is None else f"__bin{bin_val_size}"
    key = f"{name}__{years[0]}-{years[-1]}__{method}__{return_period}yr{bin_key}.pkl"
```

```
    return memoize_pickle(
        CONTOUR_CACHE_DIR / key, fit,
        force=FORCE_CONTOURS if force is None else force,
    )


def contour_peak(name, years, method="PCA", return_period=100):
    """Peak Hm0 of a cached contour, or NaN if it did not converge."""
    hm0 = contour(name, years, method, return_period)["x1"]
    return np.nan if hm0 is None else float(np.nanmax(hm0))


record = pd.Series(
    {POINT_INFO[n]["label"]: len(load_point_years(n)) for n in point_order},
    name=f"sea states, {CONTOUR_YEARS[0]}–{CONTOUR_YEARS[-1]}",
)
print(record.to_string())
```

```
PacWave South          14616
Kodiak (open water)    14616
WETS                   14616
San Luis Obispo        14616
```

14.1 The extreme against the length of the record

For each point, the peak of the 100-year contour, the one number a designer takes away, is recomputed on 1, 3 and 5 years, each window a superset of the last (Figure 55). If a longer record simply refined the estimate, these lines would settle. They do not: within one to five years the estimate has not converged, so any single window is one draw from a distribution of answers, and a number quoted from one window without saying which window is not meaningful.

```
sweep = pd.DataFrame(
    {
        POINT_INFO[name]["label"]: {
            n: contour_peak(name, years) for n, years in YEAR_WINDOWS.items()
        }
        for name in point_order
    }
)

fig, ax = plt.subplots(figsize=(FIG_WIDTH, 5.5))
for name in point_order:
    label = POINT_INFO[name]["label"]
    ax.plot(sweep.index, sweep[label], marker="o", color=point_colors[name],
            linewidth=2, label=label)
```

```
ax.set(xlabel=f"Years of record (each window ends at {CONTOUR_YEARS[-1]})",
       ylabel="Peak $H_{m0}$ of the 100-year contour [m]")
ax.set_xticks(list(YEAR_WINDOWS))
ax.legend(loc="upper left", bbox_to_anchor=(1.02, 1), borderaxespad=0)
fig.tight_layout()
savefig("contour_peak_vs_record_length")
```

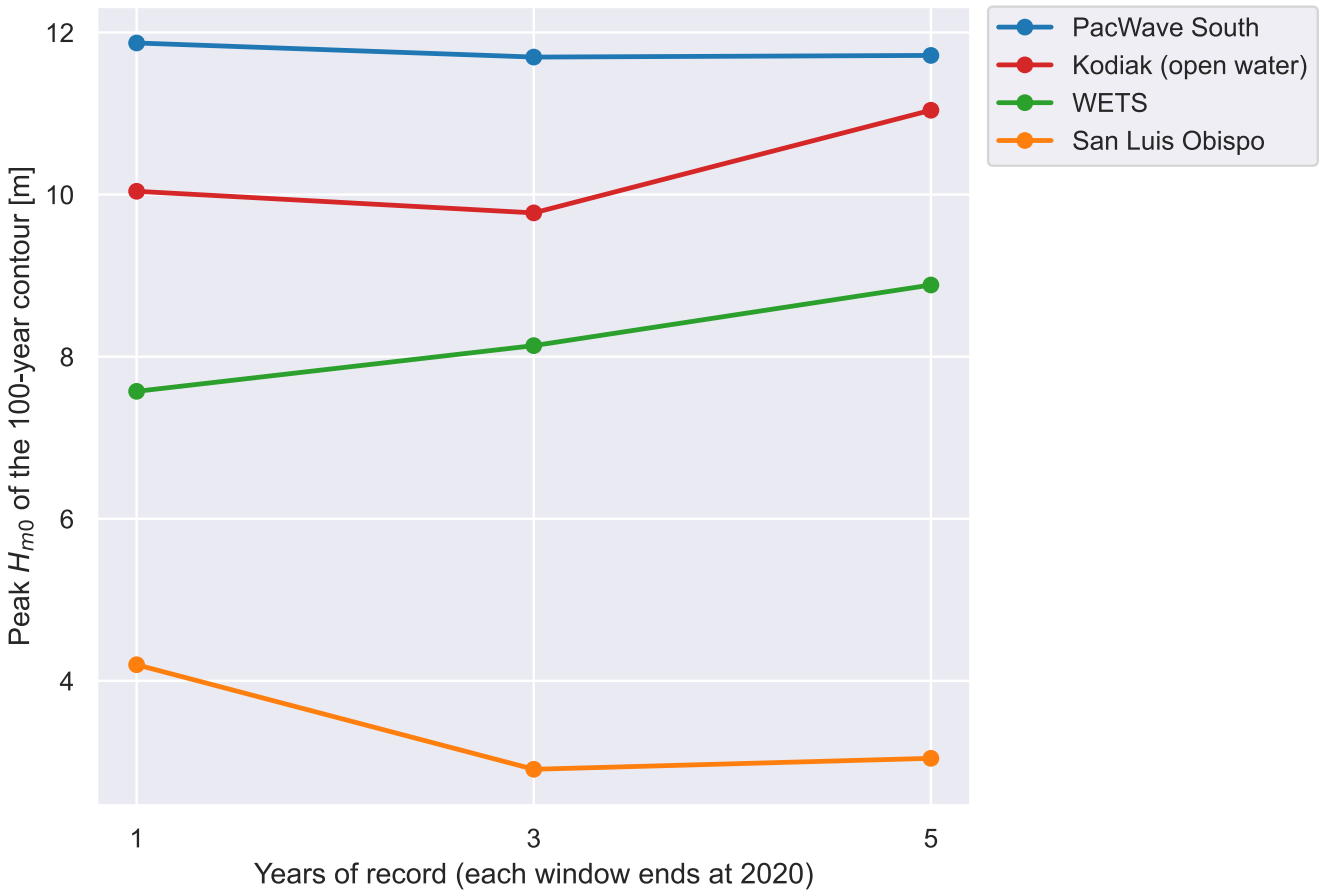

Figure 55: Peak 100-year significant wave height (H_{m0}) against record length (2016-2020), all points. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
table = sweep.T
table.columns = [f"{n} yr" for n in table.columns]
table["range [m]"] = (table.max(axis=1) - table.min(axis=1))
print(table.round(2).to_string())
```

```
                  1 yr   3 yr   5 yr  range [m]
PacWave South    11.87  11.70  11.72       0.17
```

```
Kodiak (open water)  10.04   9.77  11.04       1.27
WETS                  7.57   8.14   8.88       1.31
San Luis Obispo       4.20   2.91   3.04       1.29
```

The `range` column is the spread of answers a point gives depending only on how many years were used, a choice that has nothing to do with the ocean.

14.2 The contour at each record length

The same result as curves (Figure 56, Figure 57, Figure 58): every point's 100-year contour on one window per figure, on shared axes limits, so following one color across the figures shows the shape moving, not just its peak.

```
# Fit (or read back) every window's contours once, and frame all three figures
# to the same limits so they compare directly.
window_contours = {
    n: {name: contour(name, years) for name in point_order}
    for n, years in YEAR_WINDOWS.items()
}
_fitted = [c for by_point in window_contours.values()
           for c in by_point.values() if c["x1"] is not None]
te_max = max(np.nanmax(c["x2"]) for c in _fitted)
hm0_max = max(np.nanmax(c["x1"]) for c in _fitted)


def plot_window_contours(n):
    """All points' 100-year contours for one record-length window."""
    years = YEAR_WINDOWS[n]
    fig, ax = plt.subplots(figsize=(FIG_WIDTH, 5))
    for name in point_order:
        cop = window_contours[n][name]
        if cop["x1"] is None:
            continue
        ax.plot(cop["x2"], cop["x1"], color=point_colors[name], linewidth=1.8,
                label=POINT_INFO[name]["label"])
    ax.set(xlim=(0, te_max * 1.05), ylim=(0, hm0_max * 1.08),
           xlabel="Energy Period, $T_e$ [s]",
           ylabel="Significant Wave Height, $H_{m0}$ [m]")
    ax.legend(loc="upper left", bbox_to_anchor=(1.02, 1), borderaxespad=0,
              fontsize=8)
    fig.tight_layout()
    savefig(f"contour_100yr_{n}yr")
```

```
plot_window_contours(1)
```

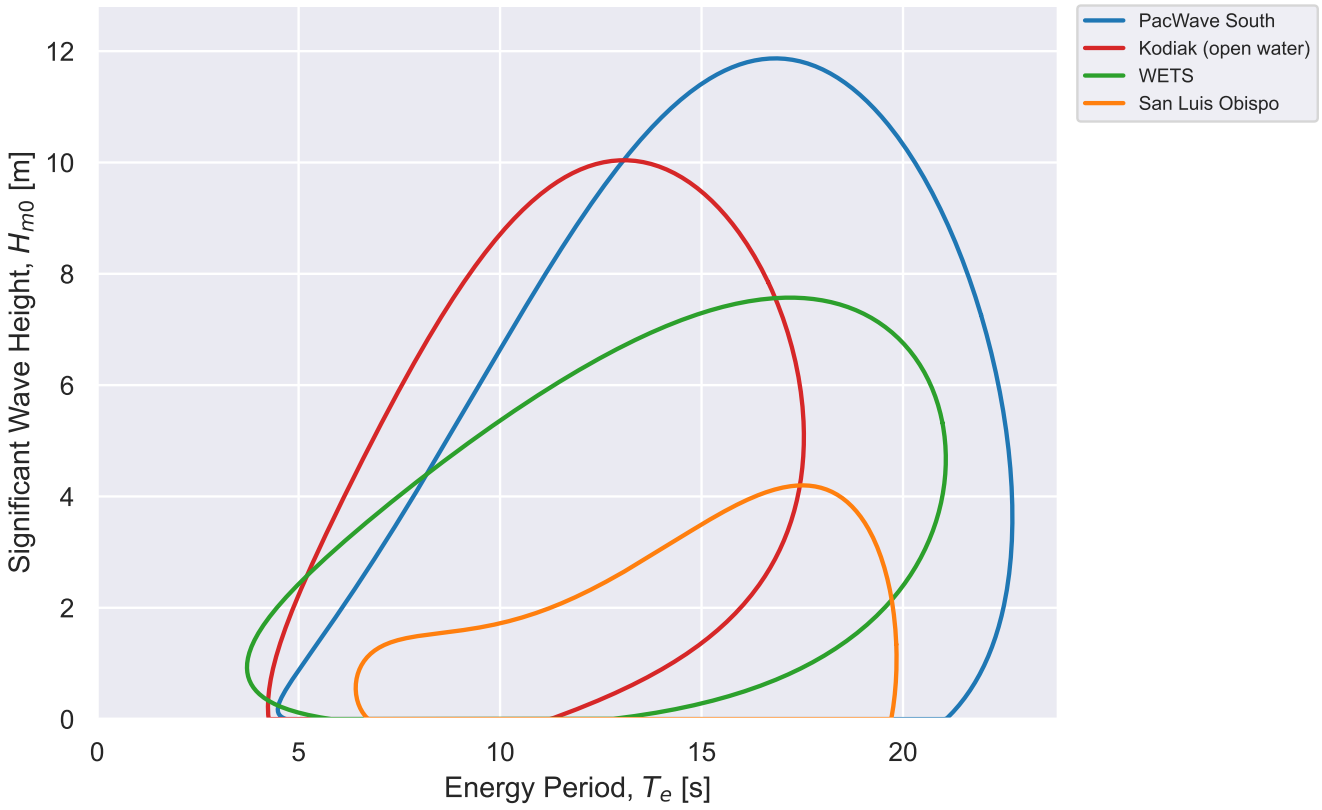

Figure 56: 100-year contours (H_{m0} vs T_e) from a 1-year record (2020), all points. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
plot_window_contours(3)
```

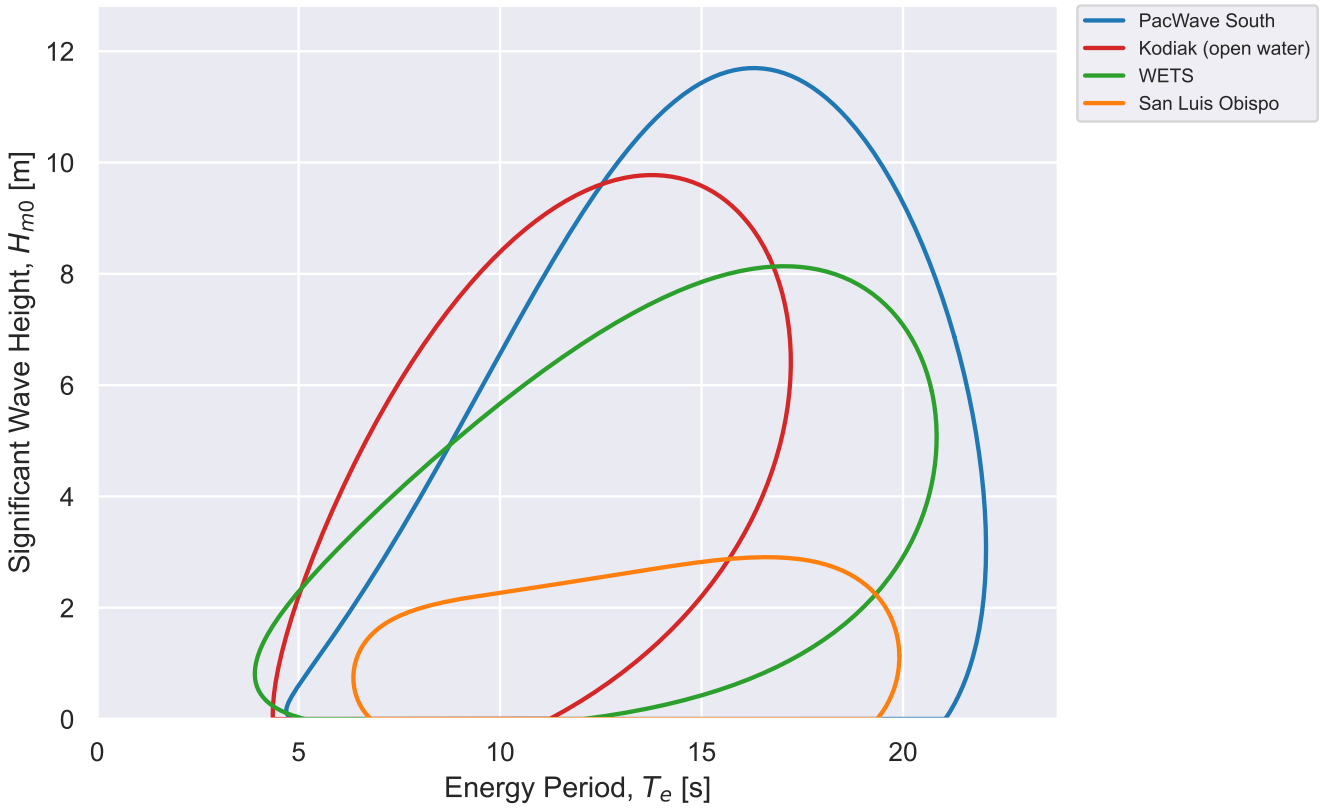

Figure 57: 100-year contours (H_{m0} vs T_e) from a 3-year record (2018-2020), all points. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
plot_window_contours(5)
```

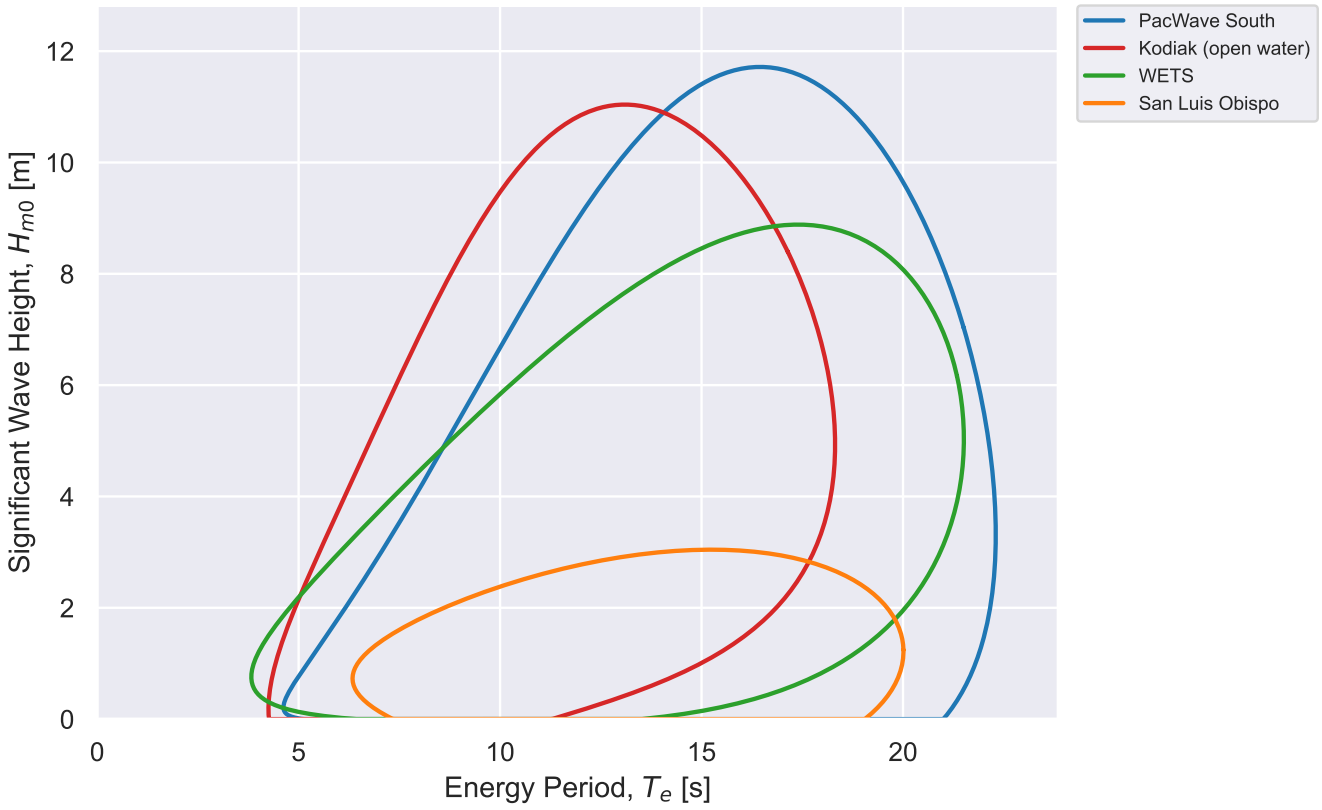

Figure 58: 100-year contours (H_{m0} vs T_e) from a 5-year record (2016-2020), all points. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

15 Multi-Point Device Comparison

Section 12 put three modelled reference devices against one point; Section 13 put several points against each other on resource alone. This section crosses the two: one MAEP per (point, device) pair, computed independently, with the RM3, RM5 and RM6 power matrices of Section 12 and the 2020 records already loaded in Section 13.

Each point's MAEP uses that point's *own* joint probability matrix, built exactly as Section 8 builds `frequency`: 0.5 m and 1 s IEC bins spanning that point's range. Sizing the grid to the point is what keeps the matrix a probability – every sea state the point produced lands in a bin, so the matrix sums to one, which mhkit requires. The device matrix is then reindexed onto whichever grid it is handed, as in Section 12.

```
def point_frequency(df):
    """Joint probability matrix for one point, on that point's own bins."""
    hm0_edges = np.arange(0, np.ceil(df.Hm0.max()) + 0.5, 0.5)
    te_edges = np.arange(0, np.ceil(df.Te.max()) + 1, 1)
    # Centers, not edges -- see the note in @sec-jpd.
```

```
    return performance.wave_energy_flux_matrix(
        df.Hm0, df.Te, df.J, "frequency",
        (hm0_edges[:-1] + hm0_edges[1:]) / 2,
        (te_edges[:-1] + te_edges[1:]) / 2,
    )


point_frequencies = {n: point_frequency(point_data[n]) for n in point_order}

# One MAEP per (point, device) pair, each its own calculation. Rows are points in
# point_order (richest resource first), columns are devices, values MWh/yr.
point_labels = [POINT_INFO[n]["label"] for n in point_order]
maep = pd.DataFrame(
    {
        device: [device_maep(matrix, point_frequencies[n]) / 1e3 for n in point_order]
        for device, matrix in POWER_MATRICES.items()
    },
    index=point_labels,
)

rated = pd.Series({d: m.to_numpy().max() for d, m in POWER_MATRICES.items()})
capacity_factor = maep * 1e3 / (rated * 8766)          # aligns on the device columns
best_point = maep.div(maep.max(axis=0), axis=1) * 100  # % of each device's best point

# --- Can the device physically go there? -------------------------------------
# A surge flap is referenced to the seabed and stands in the water column, so the
# water depth has to be close to the flap's own height -- a 25 m flap in 350 m of
# water is not a worse device, it is not a device. The library's Characteristic
# Diameter is that height, and DEPTH_TOLERANCE is how far from it a point may sit.
# The heaving buoy and the OWC float, so any depth that moors them will do; only
# technologies matched below carry the constraint.
DEPTH_TOLERANCE = 0.20
SEABED_REFERENCED = "Surge"  # matched against the library's Technology Type


def depth_band(device):
    """Workable water depth (min, max) [m], or None where depth does not constrain."""
    spec = _specs.loc[device]
    if SEABED_REFERENCED not in spec["Technology Type"]:
        return None
    size = float(spec["Characteristic Diameter"])
    return size * (1 - DEPTH_TOLERANCE), size * (1 + DEPTH_TOLERANCE)


def installable(device, point):
    """Can `device` physically be installed at `point`? Unknown depth is not a no."""
```

```
    band = depth_band(device)
    depth = POINT_INFO[point]["depth"]
    if band is None or depth is None:
        return True
    return band[0] <= depth <= band[1]


# Same shape as `maep`, so every figure can mask straight off it.
installable_at = pd.DataFrame(
    {d: [installable(d, n) for n in point_order] for d in POWER_MATRICES},
    index=point_labels,
)
point_depths = pd.Series([POINT_INFO[n]["depth"] for n in point_order],
                         index=point_labels)

# Points keep the colors they carry everywhere else in this notebook; devices get a
# separate palette so a device bar is never mistaken for a point bar.
label_colors = {POINT_INFO[n]["label"]: point_colors[n] for n in point_order}
device_colors = dict(
    zip(POWER_MATRICES, sns.color_palette("Set2", len(POWER_MATRICES)))
)
```

```
maep_table = maep.round(1).astype(str).mask(~installable_at, maep.round(1).astype(str) + " †")
maep_table.insert(0, "Depth [m]", point_depths.round(1))
show_table(maep_table)
```

Table 4: Mean annual energy production [MWh/yr] for each modelled reference device at each compared point, from each point's 2020 joint probability matrix, with the water depth at each point's grid node. A dagger marks a device that cannot physically be installed at that depth; its energy is still shown. Device Source: modelled reference-device power matrices, Wave Energy Converters.csv. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

15.1 MAEP by Point and by Device

The same numbers, grouped two ways. Figure 59 puts one group per point and answers "at this point, which device earns most". Figure 60 puts one group per device and answers "for this device, which point earns most". Both axes are linear: a log axis would flatten exactly the magnitude gap between points that this comparison exists to show.

Grey, hatched bars are combinations that the water depth rules out. RM5 is an oscillating surge flap: it is referenced to the seabed and stands in the water column, so it needs a depth close to its own 25 m height (the library's Characteristic Diameter), and `DEPTH_TOLERANCE` allows 20 m to 30 m. None of the four points sit in that band – San Luis Obispo is too shallow at 14 m, and PacWave South, Kodiak

and WETS are 67.7 m, 143.9 m and 353.1 m – so every RM5 bar here is greyed. RM3 and RM6 both float, so depth only has to moor them and neither is constrained.

The greyed bars still carry their computed height, deliberately. The arithmetic is sound – that is genuinely the energy the flap's power matrix and the point's sea states imply – and showing it keeps the resource comparison legible while making clear that the number is not a deployment. A device that cannot be installed earns nothing, whatever its matrix says.

> **Note**
>
> The reference-model power matrices stop at H_{m0} 9.75 m and T_e 20.5 s, and a point that produces sea states past that earns nothing from them – the grey, non-overlapping region of Figure 44. That limit does not bind for the points and the year compared here: every point's bins fall inside the device grid, so the numbers below are set by where each device *produces*, not by where its matrix runs out. It would bind on a longer or more energetic record, and a low MAEP is worth checking against the device's envelope before reading it as a statement about the resource.

```
def grouped_bars(frame, colors, ylabel, name, fmt="%.0f", height=4.5, mask=None):
    """Grouped vertical bars: one x group per row of `frame`, one bar per column.

    `mask` is a boolean frame shaped like `frame`; False draws the bar grey and
    hatched instead of in its own color. The bar keeps its height -- the energy is
    real arithmetic, it is the installation that is not possible.
    """
    groups, series = list(frame.index), list(frame.columns)
    x = np.arange(len(groups))
    width = 0.8 / len(series)
    fig, ax = plt.subplots(figsize=(FIG_WIDTH, height))
    for i, name_ in enumerate(series):
        offset = (i - (len(series) - 1) / 2) * width
        ok = (np.ones(len(groups), bool) if mask is None
              else mask[name_].to_numpy().astype(bool))
        bars = ax.bar(
            x + offset, frame[name_].to_numpy(), width, label=name_,
            color=[colors[name_] if good else "0.88" for good in ok],
            edgecolor=["none" if good else "0.55" for good in ok],
        )
        for bar, good in zip(bars, ok):
            if not good:
                bar.set_hatch("///")
        ax.bar_label(bars, fmt=fmt, padding=2, fontsize=7, color="0.25")
    ax.set_xticks(x)
    # Wrap at spaces rather than rotating: point labels like "Kodiak (open water)"
    # collide at this width, and a rotated categorical axis reads worse than two lines.
    ax.set_xticklabels(["\n".join(textwrap.wrap(str(g), 14)) for g in groups])
```

```
    ax.set_ylabel(ylabel)
    ax.margins(y=0.16)  # headroom so the tallest bar's label is not clipped
    # Proxy handles, so a series keeps its own color in the legend even when its
    # first bar happens to be a greyed one.
    handles = [Patch(facecolor=colors[s]) for s in series]
    labels_ = list(series)
    if mask is not None and not mask.to_numpy().all():
        handles.append(Patch(facecolor="0.88", edgecolor="0.55", hatch="///"))
        labels_.append("not installable\nat this depth")
    ax.legend(handles, labels_, loc="upper left", bbox_to_anchor=(1.02, 1),
              borderaxespad=0)
    fig.tight_layout()
    savefig(name)
```

```
grouped_bars(maep, device_colors, "MAEP [MWh/yr]", "multipoint_maep_by_point",
             mask=installable_at)
```

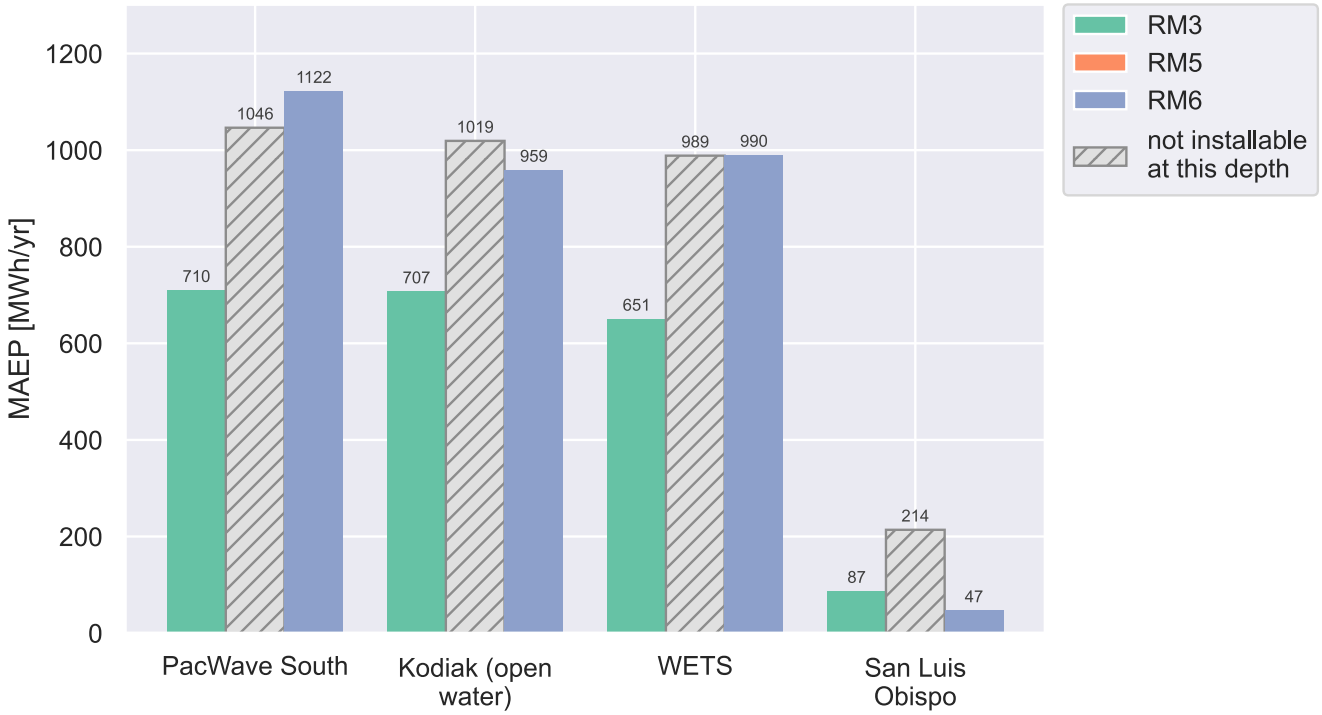

Figure 59: Mean annual energy production per point, one bar per modelled reference device (2020). Device Source: modelled reference-device power matrices, Wave Energy Converters.csv. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
grouped_bars(maep.T, label_colors, "MAEP [MWh/yr]", "multipoint_maep_by_device",
             mask=installable_at.T)
```

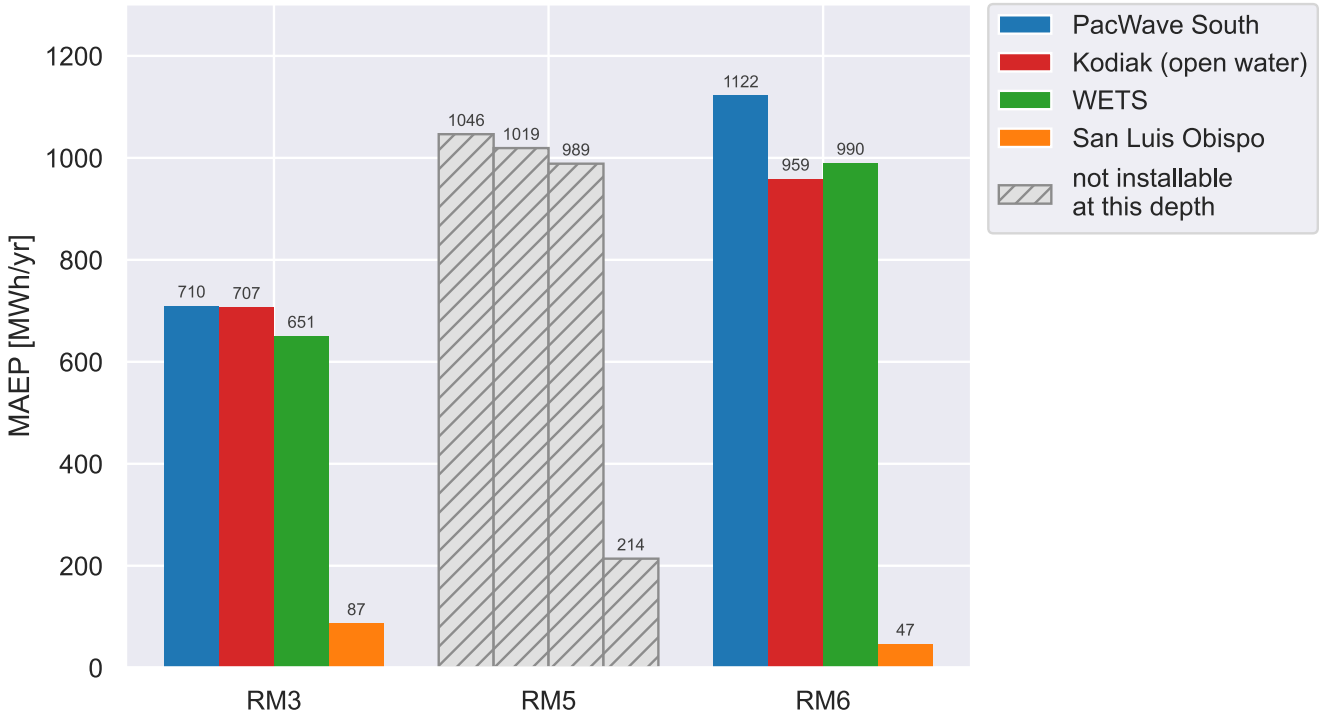

Figure 60: Mean annual energy production per modelled reference device, one bar per point (2020). Device Source: modelled reference-device power matrices, Wave Energy Converters.csv. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

15.2 Normalized Comparisons

MAEP mixes resource with device size, so the two figures above rank a large device above a small one wherever it is placed. Figure 61 divides that out: MAEP against what the device would make running at its rated power for the whole average year (the same 8766 h and the same rating – the matrix maximum – used in Table 2). It reads as utilization, and is comparable across devices of different size.

Figure 62 normalizes the other way, expressing each device's MAEP as a percentage of its own best point. Device scale drops out entirely, leaving only how much a device gives up by moving.

```
grouped_bars(capacity_factor.T, label_colors, "Capacity factor",
             "multipoint_capacity_factor", fmt="%.2f", mask=installable_at.T)
```

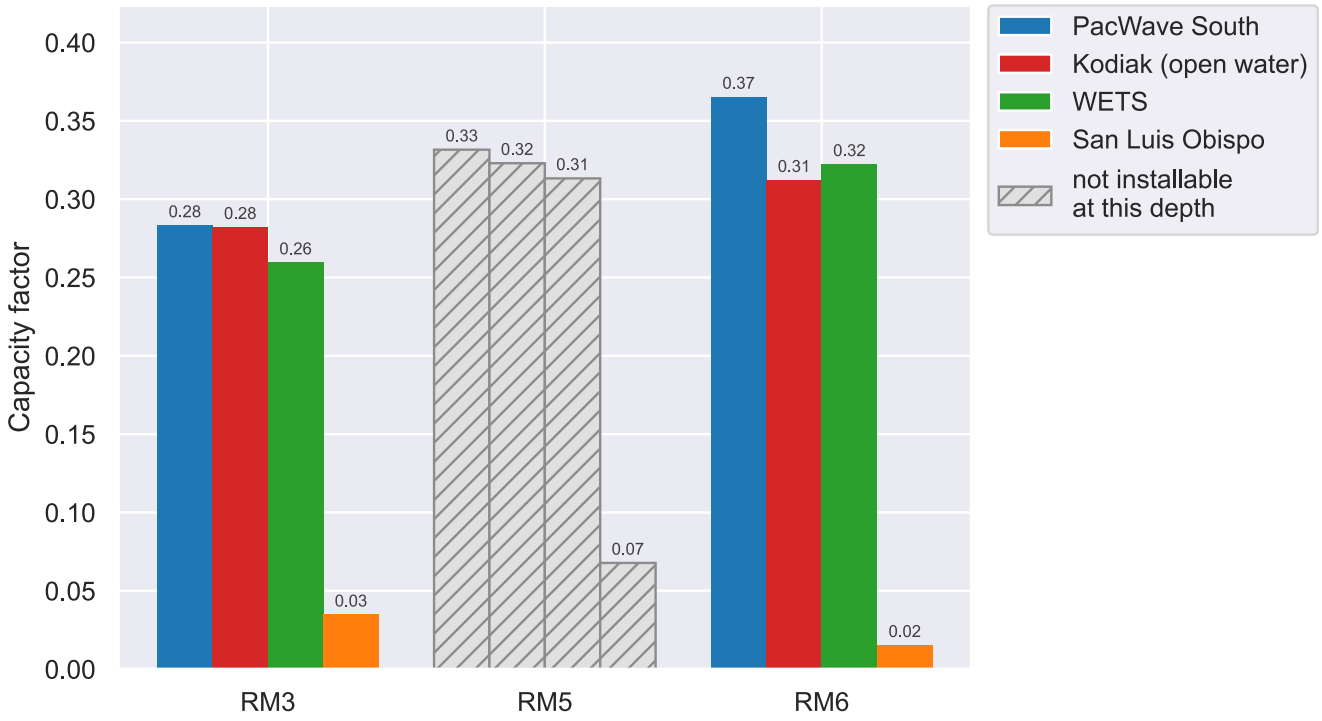

Figure 61: Capacity factor – MAEP against rated power over an 8766 h average year – per modelled reference device, one bar per point (2020). Device Source: modelled reference-device power matrices, Wave Energy Converters.csv. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

```
grouped_bars(best_point.T, label_colors, "MAEP [% of device best point]",
             "multipoint_maep_normalized", mask=installable_at.T)
```

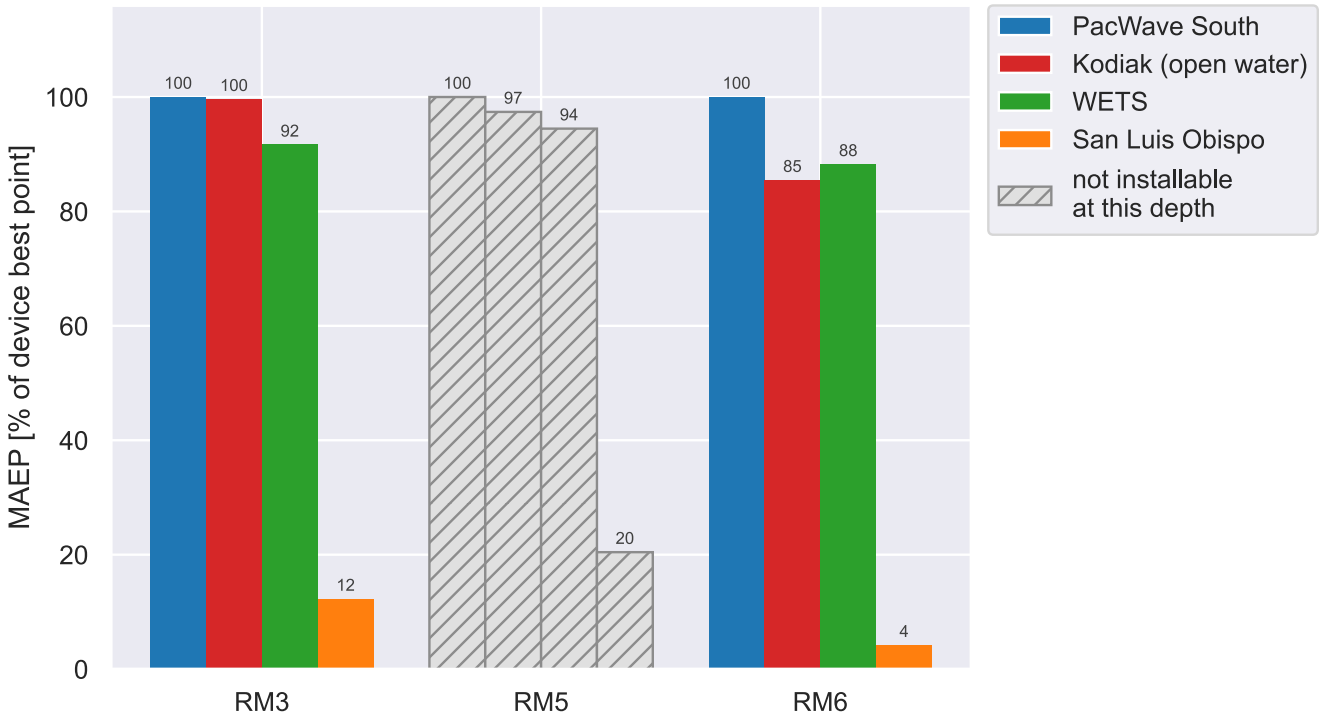

Figure 62: Mean annual energy production as a percentage of each modelled reference device's own best point, one bar per point (2020). Device Source: modelled reference-device power matrices, Wave Energy Converters.csv. Data Source: Yang, Zhaoqing, Vince Neary, Levi Kilcher, and Mike Lawson. 2020. High Resolution Ocean Surface Wave Hindcast (US Wave) Data. Marine and Hydrokinetic Data Repository, National Renewable Energy Laboratory. https://doi.org/10.15473/1647329

References

Bibliography

[1] Z. Yang, V. Neary, L. Kilcher, and M. Lawson, "High Resolution Ocean Surface Wave Hindcast (US Wave) Data." [Online]. Available: https://mhkdr.openei.org/submissions/326

[2] R. Coe, C. Michelen, A. Eckert-Gallup, Y.-H. Yu, and J. Van Rij, "WEC Design Response Toolbox v. 1.0." [Online]. Available: https://www.osti.gov//servlets/purl/1312743

[3] A. C. Eckert-Gallup, C. J. Sallaberry, A. R. Dallman, and V. S. Neary, "Application of principal component analysis (PCA) and improved joint probability distributions to the inverse first-order reliability method (I-FORM) for predicting extreme sea states," *Ocean Engineering*, vol. 112, pp. 307–319, 2016, doi: 10.1016/j.oceaneng.2015.12.018.